

List of Files

```
using Printf
include("src/printmat.jl");
```

```
files = filter(x → endswith(lowercase(x),".ipynb"), readdir()) #all .ipynb files
filter!(x → x != "Ch00_ListOfFiles.ipynb",files)
```

```
printmat(files)
```

```
Ch01_Basics.ipynb
Ch02_PortfolioChoice1.ipynb
Ch02_StatsAppendix.ipynb
Ch03_MeanVariance.ipynb
Ch03b_MV_NoShortSales.ipynb
Ch04_Betas.ipynb
Ch05_PortfolioChoice2.ipynb
Ch06_CAPM.ipynb
Ch07_RiskMeasures.ipynb
Ch08_UtilityTheory.ipynb
Ch09_MultiFactorM.ipynb
Ch10_EfficientMarkets.ipynb
Ch11_PerfEval.ipynb
Ch12_LongRun.ipynb
Ch13_DynamicPortfolios.ipynb
```

Basics

of return calculations: returns, average returns and volatilities of portfolios.

Load Packages and Extra Functions

The notebook uses the functions `printmat()` and `printlnPs()` for formatted printing of matrices and numbers. These functions are in the included `src/printmat.jl` file and call on the `Printf` package.

Also, the `lag()` function (from `src/UtilityFnMFT.jl`) lags a vector/matrix.

```
using Printf

include("src/printmat.jl")
include("src/UtilityFnMFT.jl");           #; to suppress printing of last command in cell
```

Return Calculations

The return of holding the asset between $t - 1$ and t is

$$R_t = (P_t + D_t)/P_{t-1} - 1,$$

where P_t is the price (measured after dividends) and D_t is the dividend.

We can calculate the returns by a loop or by a more compact notation, see below.

A Remark on the Code

For the vectorized version, notice that `- lag(P)` creates a vector `[NaN,P[1],...,P[end-1]]`, that is, the previous (lagged) value of `P`. - we use `[a,b]./[c,d]` to do element-by-element division. Also, use `[a,b] .- 1` to subtract 1 from each element. In contrast, `[a,b]-[c,d]` (or `+`) needs no dot since that is a traditional mathematical operation.

```
P = [100,108,109]          #prices (after dividends) for t=1,2,3
D = [0,2,0]               #dividends, could also use [0;2;0]

R = zeros(length(P))      #where to store the results
for t in 2:length(P)      #P[2] is the 2nd element of P
    R[t] = (P[t] + D[t])/P[t-1] - 1
end
R = R[2:end]              #get rid of R[1] since we have no return there

R_alt = (P + D)./lag(P) .- 1    #vectorized alternative, notice the ./ and .- 1

printmat(R*100,R_alt[2:end]*100,colNames=["return, %","return (alt), %"],rowNames=2:3,cell00="period")
```

period	return, %	return (alt), %
2	10.000	10.000
3	0.926	0.926

Excess Returns

Subtract a reference return, often a safe return, to get an excess return

$$R_t^e = R_t - R_{f_t}$$

```
Rf = 0.01

R^e = R .- Rf          #R\^e [TAB] to get R^e
printmat(R^e)
```

0.090
-0.001

Descriptive Statistics

We need some extra packages for reading in data.

Data is monthly so we annualize means by multiplying by 12 and standard deviations by $\sqrt{12}$.

```
using Statistics, DelimitedFiles
```

```
x = readlm("Data/FFmFactorsPs.csv",',',skipstart=1)
ym = x[:,1]
x = x[:,2:4];    #market, small minus big, high minus low

println("Sample period: ",ym[1],"-",ym[end])
```

Sample period: 197001.0-202412.0

```
μ = mean(x,dims=1)*12
σ = std(x,dims=1)*sqrt(12)
SR = μ./σ

xx = vcat(μ,σ,SR)          #to print
printmat(xx;rowNames=["mean","std","SR"],colNames=["Rme","SMB","HML"])
```

	Rme	SMB	HML
mean	7.296	1.176	3.445
std	15.931	10.666	10.736
SR	0.458	0.110	0.321

Cumulating Returns

Net returns can be cumulated to calculate portfolio values as

$$V_t = V_{t-1}(1 + R_t)$$

where we need a starting value (initial investment) for the portfolio (a common choice is to normalise to $V_0 = 1$).

With log returns, $r_t = \log(1 + R_t)$, we instead do

$$\ln V_t = \ln V_{t-1} + r_t$$

If the return series is an excess return, add the riskfree rate to convert it to get net returns - and then cumulate as described above.

A Remark on the Code

- Use `cumprod([a,b])` to calculate $[a, a*b]$ and `cumsum([a,b])` to calculate $[a, a+b]$.
- To add 1 to each element of an array R , do `1 .+ R` (Notice the dot and the space before the dot.)
- To calculate the logarithm of each value in a matrix X , do `log.(X)`. Again, notice the dot. In general, a function $fn(x)$ that is defined for a scalar can be called with a dot `fn.(X)` to do the calculation for each element in an array (vector, matrix,...).

```
R = [20,-35,25]/100          #returns for t=1,2,3
V = cumprod(1 .+ R)          #V(t) = V(t-1)*(1+R(t)), starting at 1 in t=0
r = log.(1 .+ R)             #log returns
lnV = cumsum(r)              #lnV(t) = lnV(t-1) + r(t)

printmat(R,V,lnV,colNames=["R","V","lnV"],rowNames=1:3,cell00="period")

printred("Check that lnV really equals log.(V). Also, try a loop instead")
```

period	R	V	lnV
1	0.200	1.200	0.182
2	-0.350	0.780	-0.248
3	0.250	0.975	-0.025

Check that `lnV` really equals `log.(V)`. Also, try a loop instead

Portfolio Return

We form a portfolio by combining n assets: v is the vector of n portfolio weights. The portfolio return is

$$R_v = v'R,$$

where R is a vector of returns of the n assets.

```
v = [0.8,0.2]
R = [10,5]/100          #returns of asset 1 and 2
Rv = v'R                #R\v[TAB] to get Rv

printblue("Portfolio weights:")
printmat(v,rowNames=["asset 1","asset 2"])

printblue("Returns:")
```

```
printmat(R;rowNames=["asset 1","asset 2"])

printblue("Portfolio return: ")
printlnPs(Rv)
```

Portfolio weights:

```
asset 1    0.800
asset 2    0.200
```

Returns:

```
asset 1    0.100
asset 2    0.050
```

Portfolio return:

```
0.090
```

Portfolio Return with a Riskfree Asset

The previous expression for a portfolio return is still correct when one of the returns (in R) is riskfree. However, we will often use an alternative form

$$R_p = v'R + (1 - 1'v)R_f,$$

or (equivalently) as

$$R_p = v'R^e + R_f,$$

where v now are the weights on the n risky assets and no longer required to sum to 1.

In Julia $1'v$ can be calculated as `ones(n)'v` or (perhaps easier) `sum(v)`.

```
Rf = 0.01

Re = R .- Rf

v = [0.55,0.2]

printlnPs("weight on riskfree asset: ",1-sum(v))

Rp = v'R + (1-sum(v))*Rf
Rp_alt = v'Re + Rf

printlnPs("Portfolio return, two versions: ",Rp," and ",Rp_alt)
```

weight on riskfree asset: 0.250
Portfolio return, two versions: 0.068 and 0.068

The Basics of Portfolio Choice

This notebook analyses the effect of leverage and diversification on the portfolio performance.

Load Packages and Extra Functions

```
using Printf  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)  #or :svg
```

Portfolio Return: Expected Value and Variance

We form a portfolio by combining n assets: v is the vector of n portfolio weights. The portfolio return is

$$R_v = v'R,$$

where R is a vector of returns of the n assets.

```
v = [0.8,0.2]  
printblue("Portfolio weights:")  
printmat(v;rowNames=["asset 1","asset 2"])
```

```
Portfolio weights:  
asset 1    0.800  
asset 2    0.200
```


The expected portfolio return and the portfolio variance can be computed as

$$ER_v = v' \mu \text{ and}$$

$$\text{Var}(R_v) = v' \Sigma v,$$

where μ is a vector of expected (average) returns of the n assets and Σ the $n \times n$ variance-covariance matrix.

```

μ = [9,6]/100                                #\mu[TAB] to get μ
Σ = [256 96;                                #\Sigma[TAB]
     96 144]/100^2

printblue("expected returns*100: ")
printmat(μ*100;rowNames=["asset 1","asset 2"],prec=2)

printblue("covariance matrix*100^2:")
printmat(Σ*100^2;rowNames=["asset 1","asset 2"],colNames=["asset 1","asset 2"],prec=2)

```

expected returns*100:

asset 1	9.00
asset 2	6.00

covariance matrix*100^2:

	asset 1	asset 2
asset 1	256.00	96.00
asset 2	96.00	144.00

```

ER_v = v'μ
VarR_v = v'Σ*v

printlnPs("Expected portfolio return: ",ER_v)
printlnPs("Portfolio variance and std:",VarR_v,sqrt(VarR_v))

```

Expected portfolio return:	0.084	
Portfolio variance and std:	0.020	0.142

Leverage: A Risky and a Riskfree Asset

Suppose you can invest in a risky asset (with return R , expected return μ and standard deviation σ) and also in a riskfree asset (at the rate R_f).

With the portfolio weight v on the risky asset, the portfolio return is

$$R_p = vR + (1 - v)R_f.$$

$v > 1$ is called leverage (and it is financed by borrowing at the rate R_f).

The average and standard deviation of the portfolio are

$$ER_p = v\mu + (1 - v)R_f$$

and

$$\text{Std}(R_p) = |v|\sigma,$$

where (μ, σ) are the average return and standard deviation of the risky asset.

By considering different values of v , we can show what sort of combinations of ER_p and $\text{Std}(R_p)$ that can be achieved.

A Remark on the Code

1. if $v=0.2$, then `println("hello is $v")` will print `hello is 0.2`
2. if $v_range=[0.5,0.25]$, then `1.0 .- v_range` will give `[1-0.5,1-0.25]`.

```
μ = 9.5/100 #expected return of the risky asset
σ = 8/100   #std of the risky asset
Rf = 3/100  #risk free return (interest rate)

v = 0.5     #try a single value of v

ERp = v*μ + (1 - v)*Rf
StdRp = abs(v)*σ

printlnPs("ERp and Std(Rp) when v=$v:", ERp, StdRp)
```

ERp and Std(Rp) when v=0.5: 0.062 0.040

```
v_range = [-2,-1,-0.5,0,0.5,1,2] #try different values of v
ERp      = v_range*μ + (1 .- v_range)*Rf #a vector of the same length as v_range, notice the .-
StdRp    = abs.(v_range)*σ #notice the dot.

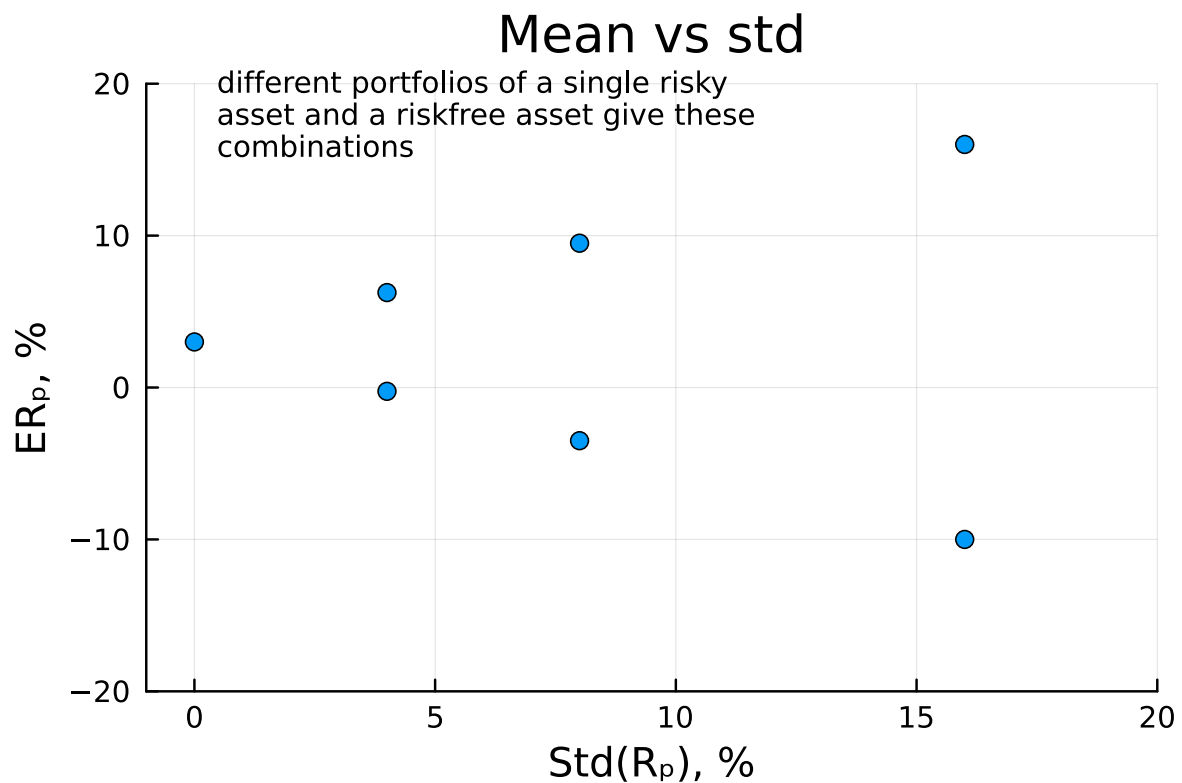
txt = text("different portfolios of a single risky\nasset and a riskfree asset give these\ncombinat

p1 = scatter( StdRp*100, ERp*100,
              legend = false,
```

```

ylim = (-20,20),
xlim = (-1,20),
title = "Mean vs std",
xlabel = "Std(Rp), %",
ylabel = "ERp, %",
annotation = (0.5,18,txt) )
display(p1)

```



Diversification

The Correlations

The variance of an *equally weighted* portfolio of *two assets* is

$$\text{Var}(R_p) = \sigma_{11}/4 + \sigma_{22}/4 + \sigma_{12}/2,$$

where σ_{ii} the variance of asset i and σ_{ij} is the covariance of assets i and j . Notice that $\sigma_{12} = \rho \sqrt{\sigma_{11} \sigma_{22}}$, where ρ is the correlation.

We here use σ_{ii} to denote a variance since it is tricky to write σ_i^2 in the code (it looks ugly, like σ^2_1).

```
 $\sigma_{11}$  = 256/100^2
 $\sigma_{22}$  =  $\sigma_{11}$           #assume the same variance of the two assets
 $\rho$     = 0.5

VarRp =  $\sigma_{11}/4$  +  $\sigma_{22}/4$  +  $\rho*\text{sqrt}(\sigma_{11}*\sigma_{22})/2$ 

println("With a correlation of $ $\rho$ , we get")
printmat([ $\sigma_{11}$ ;VarRp];rowNames=["Individual variance","portfolio variance"])
```

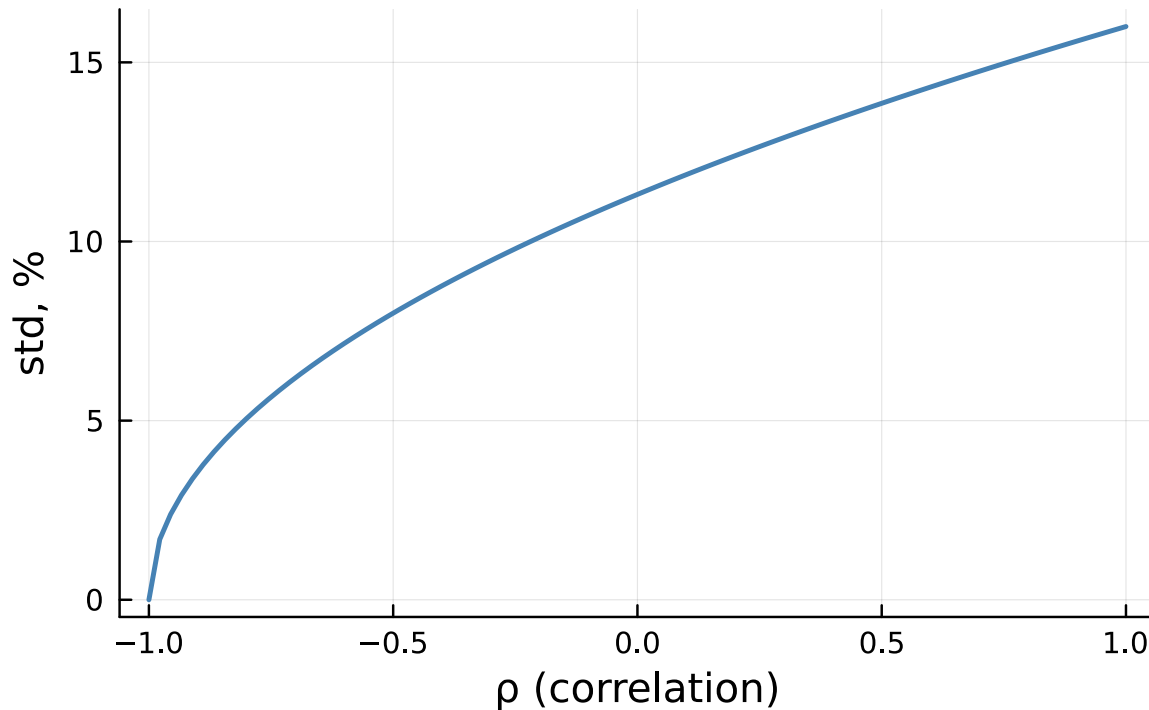
With a correlation of 0.5, we get
Individual variance 0.026
portfolio variance 0.019

```
L = 91
 $\rho_{\text{range}}$  = range(-1,1,length=L)    #repeat calculations for different correlations

VarRp = fill(NaN,L)
for i in 1:L
    VarRp[i] =  $\sigma_{11}/4$  +  $\sigma_{22}/4$  +  $\rho_{\text{range}}[i]*\text{sqrt}(\sigma_{11}*\sigma_{22})/2$ 
end

p1 = plot(  $\rho_{\text{range}}$ ,100*sqrt.(VarRp),
           linecolor = :steelblue,
           linewidth = 2,
           ylabel = "std, %",
           title = "Std of EW portfolio",
           xlabel = " $\rho$  (correlation)",
           label = false)
display(p1)
```

Std of EW portfolio



The Number of Assets

More generally, the variance of an equally weighted portfolio of n assets is

$$\text{Var}(R_p) = (\bar{\sigma}_{ii} - \bar{\sigma}_{ij})/n + \bar{\sigma}_{ij},$$

where $\bar{\sigma}_{ii}$ is the average variance (across the assets) and $\bar{\sigma}_{ij}$ is the average covariance. (In the code, we use σ_{ii_avg} to denote this.)

```
σii_avg = σ11                #average variance, for simplicity same as σ11
σij_avg = ρ*sqrt(σii_avg*σii_avg)  #average covariance
n_range = 1:49                #try all these values of n

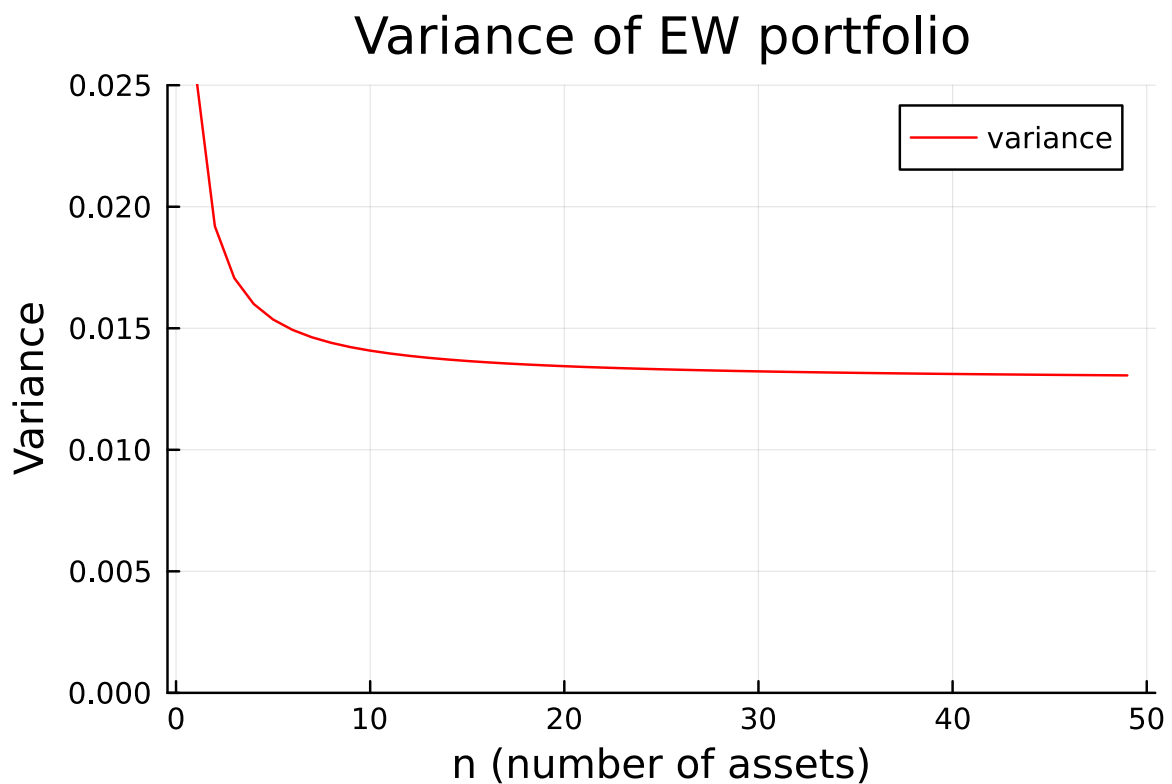
VarRp = (σii_avg-σij_avg)/n_range .+ σij_avg    #variance of equally weighted portfolio

printblue("Portfolio variance and std, n = 1 to 3")
printmat([VarRp[1:3] sqrt.(VarRp[1:3])];colNames=["var","std"],rowNames=1:3,cell00="n")
```

Portfolio variance and std, n = 1 to 3

n	var	std
1	0.026	0.160
2	0.019	0.139
3	0.017	0.131

```
p1 = plot( n_range,VarRp,
           linecolor = :red,
           linewidth = 1,
           ylim = (0,0.025),
           label = "variance",
           title = "Variance of EW portfolio",
           xlabel = "n (number of assets)",
           ylabel = "Variance" )
display(p1)
```



Calculate the Average Correlation (extra)

A Remark of the Code

`tril(C,-1)` creates a new matrix where all $n(n-1)/2$ elements below the main diagonal are kept unchanged, and all other elements are replaced by zeros.

```
using LinearAlgebra

C = [1      0.50  0.25;      #a correlation matrix
     0.50  1      0.10;
     0.25  0.10  1      ]

n = size(C,1)
c_avg = sum(tril(C,-1))/(n*(n-1)/2)  #average of the n(n-1)/2
printlnPs(c_avg)                    #numbers below the main diagonal
```

0.283

Mean Variance Frontiers

This notebook calculates (a) the mean-variance frontiers when there are no portfolio restrictions; (b) the tangency portfolio.

Load Packages and Extra Functions

```
using Printf, LinearAlgebra
include("src/printmat.jl");

prec = 2;    #default number of decimal digits in printmat()
```

```
using Plots
default(size = (480,320),fmt = :png)    #or :svg
```

The MV Frontier

Mean variance (MV) analysis starts with providing the vector of expected returns μ and the covariance matrix Σ of the investable assets.

Then, it plots the “mean variance” frontier: it is a scatter plot showing the lowest possible portfolio standard deviation ($\text{Std}(R_p)$) on the horizontal axis (yes, the standard deviation, not the variance) at a required average return ($ER_p = \mu^*$) on the vertical axis. We consider many different μ^* values to create the scatter. In most figures we connect the dots to form a curve.

Remember: to calculate the expected return and the variance of a portfolio with portfolio weights in the vector w , use

$$ER_p = w' \mu \text{ and}$$

$$\text{Var}(R_p) = w' \Sigma w.$$

Also, the sum of the portfolio weights should equal 1.

MV Frontier with Two Assets

With only two investable assets, all portfolios of them are on the MV frontier. We can therefore trace out the entire MV frontier by calculating the means and standard deviations of a range of portfolios with different weights (w_1 , $1 - w_1$) on the two assets.

```
 $\mu$  = [11.5, 6]/100          #expected returns
 $\Sigma$  = [166  58;          #covariance matrix
        58 100]/100^2

printblue("expected returns, %:")
printmat( $\mu$ *100;rowNames=["asset 1","asset 2"],prec)      #prec is defined above

printblue("covariance matrix, bp:")
printmat( $\Sigma$ *100^2;rowNames=["asset 1","asset 2"],colNames=["asset 1","asset 2"],prec)
```

```
expected returns, %:
asset 1      11.50
asset 2       6.00
```

```
covariance matrix, bp:
           asset 1  asset 2
asset 1    166.00    58.00
asset 2     58.00   100.00
```

Remarks on the Code

For the the code in the next cell, notice the following:

1. $(ER_p, StdR_p) = (fill(NaN,L), fill(NaN,L))$ creates two L -vectors filled with NaNs. (This is the same as having two lines of code.)
2. local w makes sure that w inside the loop does not affect any previously assigned w . In contrast, a global w would make the value created in the loop overwrite any previous w . In a notebook, global is implicitly assumed. In a script (.jl) file, you typically have to explicitly indicate local/global.

```
L      = 41                #number of  $w_1$  values to try
w1_range = range(1.5,-0.5,length=L)  #different possible weights on asset 1

(ER_p, StdR_p) = (fill(NaN,L), fill(NaN,L))  #to put the results in
for i in 1:L
```

```

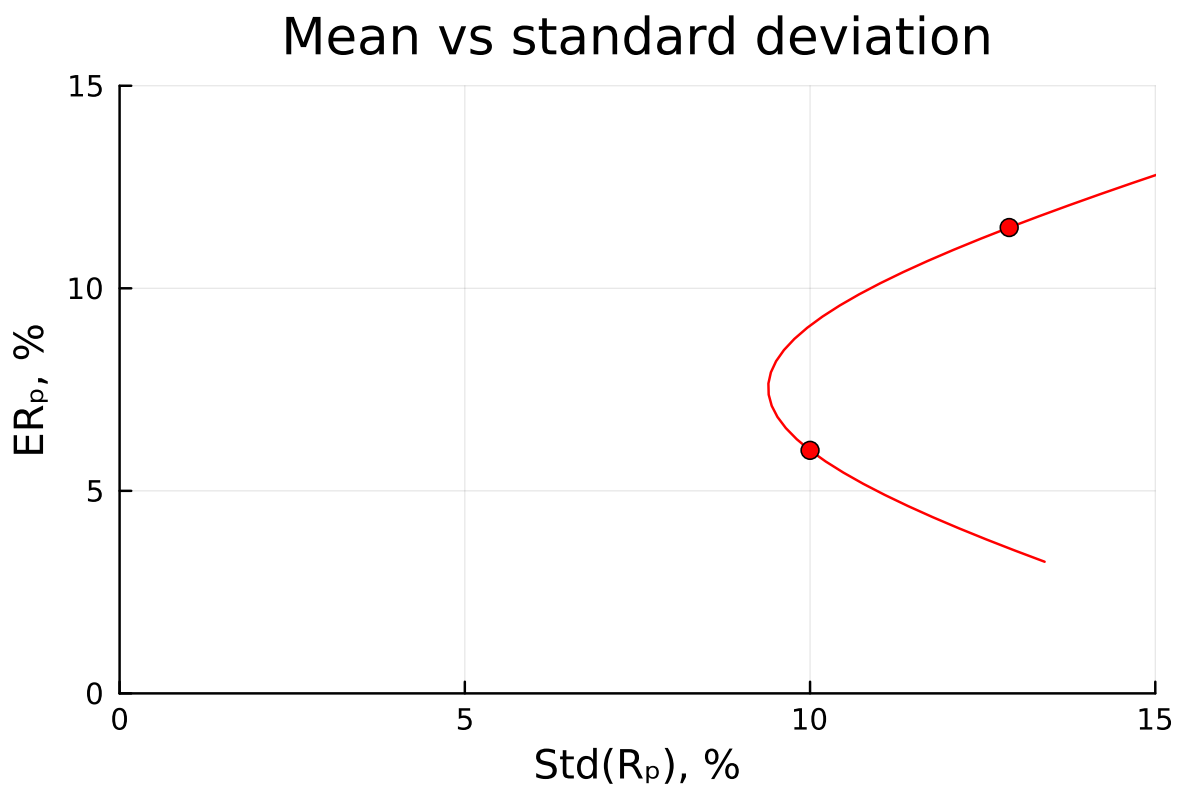
#local w                                #local/global is needed in script
w      = [w1_range[i],1-w1_range[i]]    #weights on asset 1 and 2
ERp[i] = w'*μ
StdRp[i] = sqrt(w'*Σ*w)
end

```

```

p1 = plot( StdRp*100,ERp*100,
           legend = nothing,
           linecolor = :red,
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std(Rp), %",
           ylabel = "ERp, %" )
scatter!(sqrt.(diag(Σ))*100,μ*100,markercolor=:red)
display(p1)

```



Portfolios of 3 or More (Risky) Assets

The next few cells define the average returns and the covariance matrix for 3 assets and illustrate a few portfolios.

```

μ = [11.5, 9.5, 6]/100          #expected returns
Σ = [166 34 58;                 #covariance matrix
     34 64 4;
     58 4 100]/100^2
Rf = 0.03                       #riskfree return

assetNames = ["A","B","C"]
printblue("μ and Rf in %:")
printmat(μ*100;rowNames=assetNames,prec)
printmat(Rf*100;prec)

printblue("Σ in bp:")
printmat(Σ*100^2;rowNames=assetNames,colNames=assetNames,prec)

```

μ and Rf in %:

A	11.50
B	9.50
C	6.00

3.00

Σ in bp:

	A	B	C
A	166.00	34.00	58.00
B	34.00	64.00	4.00
C	58.00	4.00	100.00

```

wM = [0 0.22 0.02 0.25;        #different portfolios (one in each column)
      1 0.30 0.63 0.68;
      0 0.48 0.35 0.07]
K = size(wM,2)                  #number of different portfolios

(ERp,StdRp) = (fill(NaN,K),fill(NaN,K))
for i in 1:K                    #loop over columns in wM
    #local w
    w = wM[:,i]

```

```

    ERp[i]    = w'*μ
    StdRp[i]  = sqrt(w'*Σ*w)
end

printblue("mean and std (in %) of the portfolios defined by vM: ")
printmat([ERp';StdRp']*100;colNames=["A","1","2","3"],rowNames=["mean","std"],prec)

```

mean and std (in %) of the portfolios defined by vM:

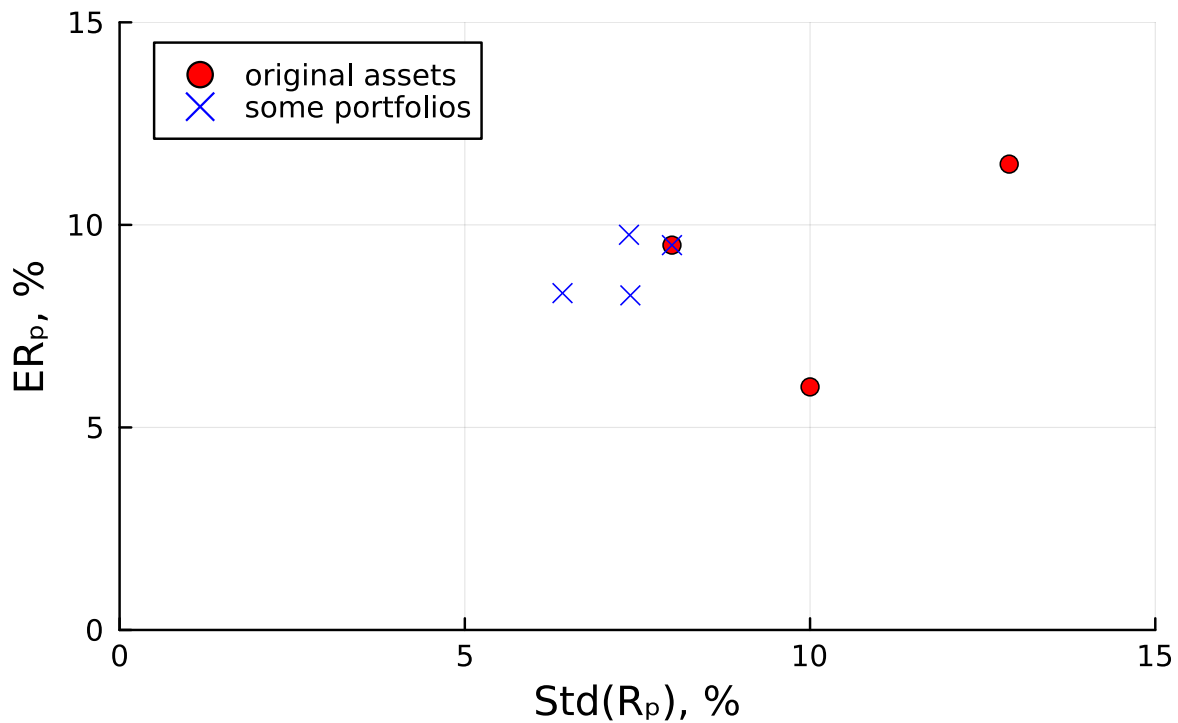
	A	1	2	3
mean	9.50	8.26	8.31	9.75
std	8.00	7.40	6.41	7.38

```

p1 = scatter( sqrt.(diag(Σ))*100,μ*100,
              markercolor = :red,
              label = "original assets",
              xlim = (0,15),
              ylim = (0,15),
              title = "Mean vs standard deviation",
              xlabel = "Std(Rp), %",
              ylabel = "ERp, %",
              legend = :topleft )
scatter!(StdRp*100,ERp*100,marker=:x,markercolor=:blue,label="some portfolios")
display(p1)

```

Mean vs standard deviation



Calculating the MV Frontier: 3 or More (Risky) Assets

To find the MV frontier with 3 or more assets we have to solve the optimization problem:

$$\min \text{Var}(R_p) \text{ s.t. } ER_p = \mu^* \text{ and } \sum_{i=1}^n w_i = 1.$$

This can be done with a numerical minimization routine or by linear algebra (at least when we do not put any further restrictions on the portfolio weights). The next cells use the linear algebra approach: it solves for w from the following linear equations (first order conditions):

$$\begin{bmatrix} \Sigma & \mu & \mathbf{1}_n \\ \mu' & 0 & 0 \\ \mathbf{1}_n' & 0 & 0 \end{bmatrix} \begin{bmatrix} w \\ \lambda \\ \delta \end{bmatrix} = \begin{bmatrix} \mathbf{0}_n \\ \mu^* \\ 1 \end{bmatrix}$$

```
"""
```

```
    MVCalc( $\mu^*$ , $\mu$ , $\Sigma$ )
```

```
Calculate the std and weights of a portfolio (with mean return  $\mu^*$ ) on MVF of risky assets.
We use  $\mu^*$  to denote the required average return, si
```

```

# Remark
- The code could be made quicker by calculating 'Af = factorize(A)' once and then loop
over different elements in a vector of ' $\mu^x$ ' values as in ' $w\lambda\delta = \dots$ ' but using 'Af' instead of 'A'

"""
function MVCalc( $\mu^x, \mu, \Sigma$ ) #the std of a portfolio on MVF of risky assets
    n = length( $\mu$ )
    A = [ $\Sigma$        $\mu$  ones(n);    #A is just a name of this matrix, it's not asset A
           $\mu'$       0 0;
          ones(n)' 0 0];
    w $\lambda\delta$  = A\[zeros(n); $\mu^x$ ;1]
    w = w $\lambda\delta$ [1:n]
    StdRp = sqrt(w' $\Sigma$ *w)
    return StdRp,w
end

```

MVCalc

```

(StdAt10,wAt10) = MVCalc(0.1, $\mu, \Sigma$ )
printblue("Testing: the MV portfolio with a mean return of 10%")
printlnPs("\nstd: ",StdAt10)

printblue("\nw and its sum: ")
printmat([wAt10;sum(wAt10)];rowNames=[assetNames;"sum"],prec)

```

Testing: the MV portfolio with a mean return of 10%

std: 0.077

w and its sum:

A	0.29
B	0.69
C	0.02
sum	1.00

A Remark on the Code

- $MVCalc(\mu^x, \mu, \Sigma)[1]$ picks out the first output from the MVCalc function.
- $StdRp = [MVCalc(\mu^x, \mu, \Sigma)[1] \text{ for } \mu^x \text{ in } \mu^x_range]$ is a loop over the different μ^x values in μ^x_range and creates a vector of outputs.

It is the same as writing an explicit loop as in

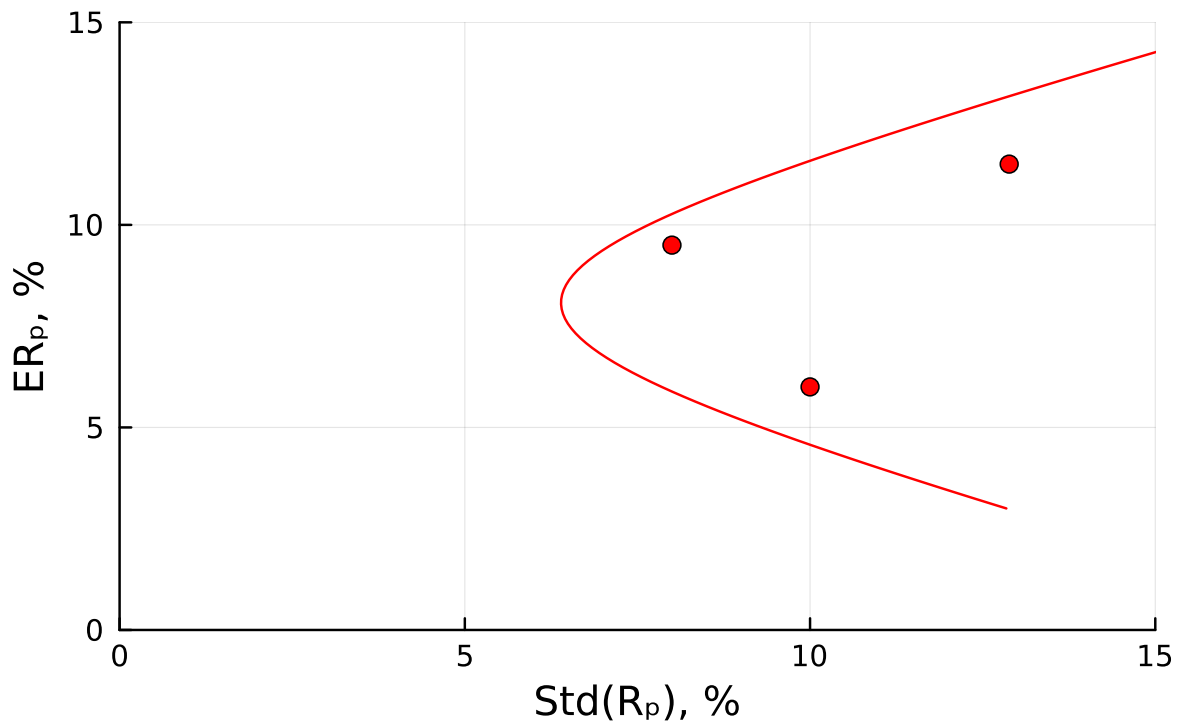
```
L      = length( $\mu^x$ _range)
StdRp = fill(NaN,L)
for i in 1:L
    StdRp[i] = MVCalc( $\mu^x$ _range[i], $\mu$ , $\Sigma$ )[1]
end
```

```
 $\mu^x$ _range = range(Rf,0.15,length=101)

StdRp = [MVCalc( $\mu^x$ , $\mu$ , $\Sigma$ )[1] for  $\mu^x$  in  $\mu^x$ _range] #loop over different required average returns, ( $\mu^x$ _

p1 = plot( StdRp*100, $\mu^x$ _range*100,
           legend = nothing,
           linecolor = :red,
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std(Rp), %",
           ylabel = "ERp, %" )
scatter!(sqrt.(diag( $\Sigma$ ))*100, $\mu$ *100,markercolor=:red)
display(p1)
```

Mean vs standard deviation



Calculating the MV Frontier (of Risky and Riskfree Assets)

All portfolios on the MV frontier of both risky and riskfree have (a vector of) portfolio weights on the risky assets as

$$w = \frac{\mu^* - R_f}{\mu^e' \Sigma^{-1} \mu^e} \Sigma^{-1} \mu^e,$$

where μ^* is the required average return.

The weight of the riskfree asset is $1 - \mathbf{1}'w$.

The cell below also contains an alternative formulation based on the first order conditions and solving them numerically.

```
"""
Calculate the std of a portfolio (with mean  $\mu^*$ ) on MVF of (Risky,Riskfree)

# Remark
- This code could be speeded up by calculating  $(\Sigma^{-1}\mu^e)/(\mu^e'\Sigma^{-1}\mu^e)$  once and then
multiply with different values of  $(\mu^* - R_f)$ .
```



```

"""
function MVCalcRf( $\mu^x$ , $\mu$ , $\Sigma$ ,Rf)
     $\mu^e$     =  $\mu$  .- Rf                #expected excess returns
     $\Sigma_{-1}$  = inv( $\Sigma$ )
    w      = ( $\mu^x$ -Rf)/( $\mu^e'$  $\Sigma_{-1}\mu^e$ ) *  $\Sigma_{-1}\mu^e$ 
    StdRp  = sqrt(w' $\Sigma$ w)
    return StdRp,w
end

"""
Alternative calculation of the std of a portfolio (with mean  $\mu^x$ ) on MVF of (Risky,Riskfree)
"""
function MVCalcRfX( $\mu^x$ , $\mu$ , $\Sigma$ ,Rf)                #calculates the std of a portfolio
    n      = length( $\mu$ )                          #on MVF of (Risky,Riskfree)
     $\mu^e$     =  $\mu$  .- Rf
    A      = [ $\Sigma$   $\mu^e$ ;
               $\mu^e'$  0]
    w $\lambda$  = A\[zeros(n);( $\mu^x$ -Rf)]
    w      = w $\lambda$ [1:n]
    StdRp  = sqrt(w' $\Sigma$ w)
    return StdRp,w
end

```

MVCalcRfX

```

(Std,w) = MVCalcRf(0.1, $\mu$ , $\Sigma$ ,Rf)
(StdX,wX) = MVCalcRfX(0.1, $\mu$ , $\Sigma$ ,Rf)

printblue("Testing: the portfolio with a mean return of 10%")
printlnPs("\nstd: ",Std)

printlnPs("\nw and its sum: ")
printmat(hcat([w;sum(w)],[wX;sum(wX)]);rowNames=[assetNames;"sum"],colNames=["explicit","alt"],prec=5)

printlnPs("weight on riskfree:",1-sum(w))

```

Testing: the portfolio with a mean return of 10%

std: 0.076

w and its sum:

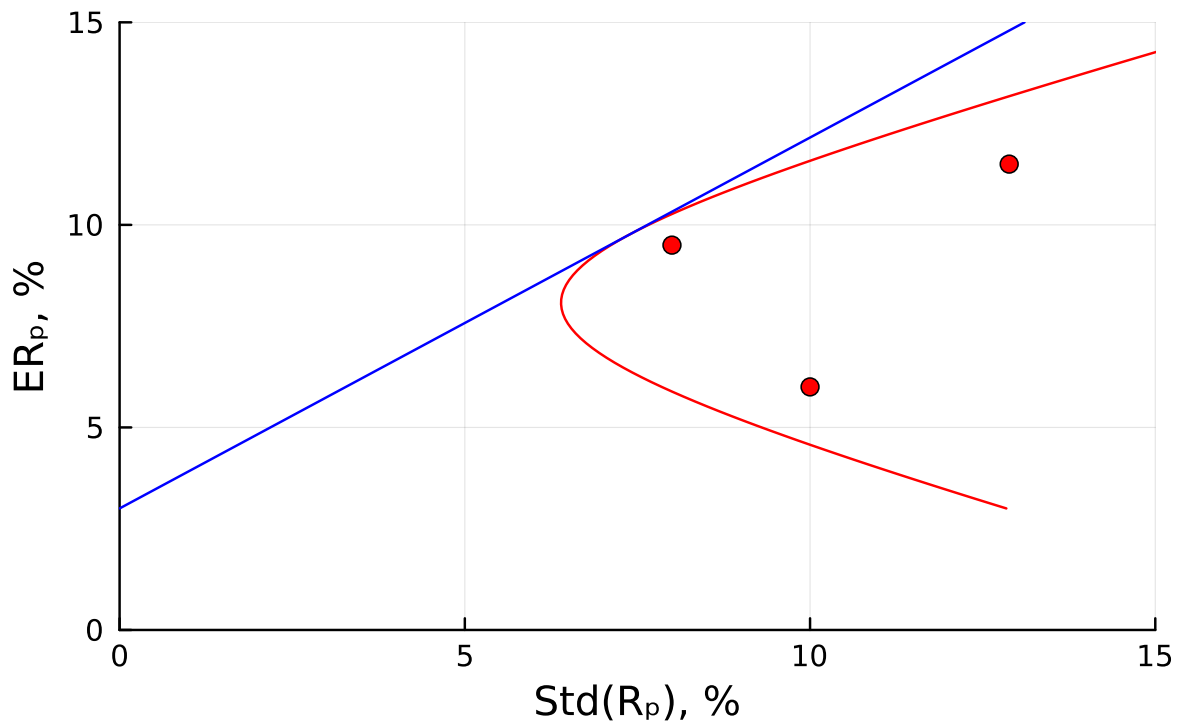
	explicit	alt
A	0.26	0.26
B	0.71	0.71
C	0.07	0.07
sum	1.04	1.04

weight on riskfree: -0.037

```
StdRpRf = [MVCalcRf( $\mu^x$ , $\mu$ , $\Sigma$ ,Rf)[1] for  $\mu^x$  in  $\mu^x$ _range]

p1 = plot( [StdRp StdRpRf]*100, $\mu^x$ _range*100,
           legend = nothing,
           linecolor = [:red :blue],
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std( $R_p$ ), %",
           ylabel = "ER $_p$ , %" )
scatter!(sqrt.(diag( $\Sigma$ ))*100, $\mu$ *100,markercolor=:red)
display(p1)
```

Mean vs standard deviation



The Tangency Portfolio

The tangency portfolio is a particular portfolio on the MV frontier of risky and riskfree, where the weights on the risky assets sum to 1. It is therefore also on the MV frontier of risky assets only. The vector of portfolio weights is

$$w_T = \frac{\Sigma^{-1} \mu^e}{\mathbf{1}' \Sigma^{-1} \mu^e}$$

```
function MVTangencyP(mu,Sigma,Rf)           #calculates the tangency portfolio
    n      = length(mu)
    mu^e   = mu .- Rf                        #expected excess returns
    Sigma_1 = inv(Sigma)
    w      = Sigma_1 * mu^e / (ones(n)' Sigma_1 * mu^e)
    muT    = w' mu^e + Rf
    StdT   = sqrt(w' Sigma * w)
    return w,muT,StdT
end
```

MVTangencyP (generic function with 1 method)

```
(wT,μT,σT) = MVTangencyP(μ,Σ,Rf)

printblue("Tangency portfolio: ")
printmat([wT;sum(wT)];rowNames=[assetNames;"sum"],prec)
printlnPs("mean and std of tangency portfolio, %: ",[μT σT]*100)
```

Tangency portfolio:

A	0.25
B	0.68
C	0.07
sum	1.00

mean and std of tangency portfolio, %:	9.750	7.372
--	-------	-------

By mixing the tangency portfolio and the riskfree, we can create any point on the MV frontier of risky and riskfree (also called the Capital Market Line, CML).

The code below shows the expected return and standard deviation of several portfolio (different v values) of the form

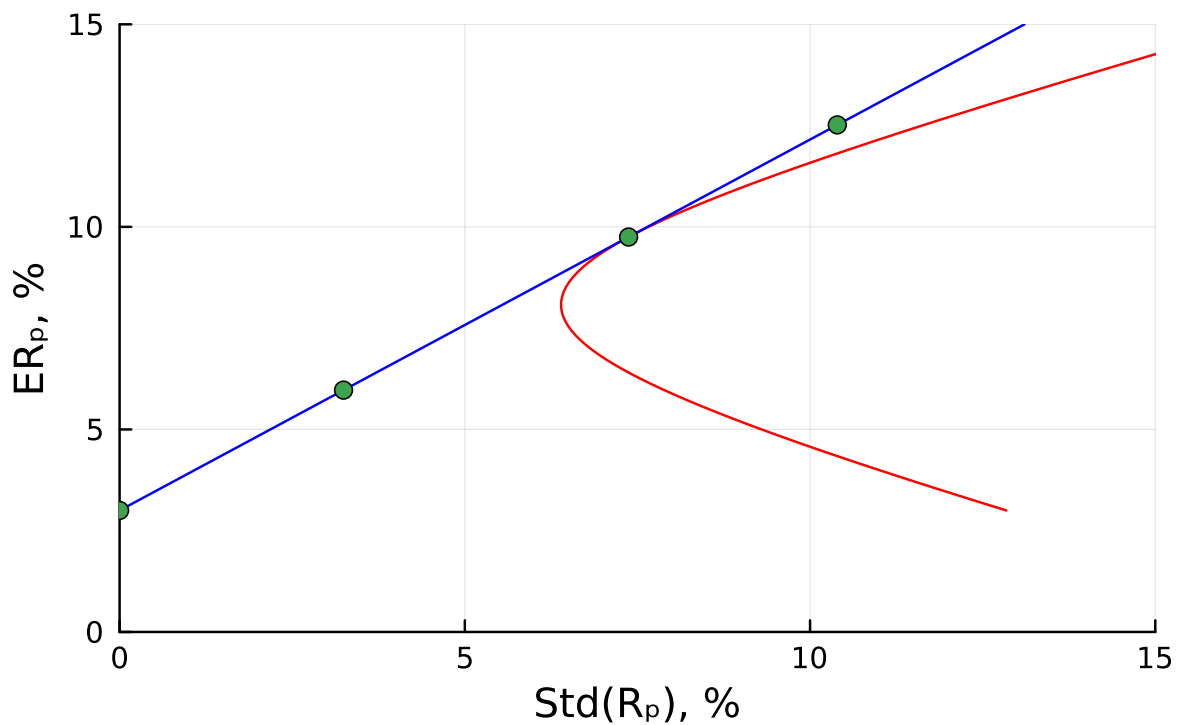
$$R_v = vR_T + (1 - v)R_f \text{ where } R_T = w_T'R$$

```
v_range = [0;0.44;1;1.41] #try different mixes of wT and Rf

ER_v = v_range*μT + (1.- v_range)*Rf
StdR_v = abs.(v_range)*σT

p1 = plot( [StdR_p StdRpRf]*100,μ*_range*100,
           legend= nothing,
           linecolor = [:red :blue],
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std(R_p), %",
           ylabel = "ER_p, %" )
scatter!(StdR_v*100,ER_v*100)
display(p1)
```

Mean vs standard deviation



Examples of Tangency Portfolios

```

μe = [8, 5]/100                                #means
Σ = [ 256  0;
      0   144]/100^2
wT, = MVTangencyP(μe,Σ,0)                        #use μe and set Rf=0
printmat(wT;rowNames=["asset 1","asset 2"],prec)

wT, = MVTangencyP([12; 5]/100,Σ,0)
printmat(wT;rowNames=["asset 1","asset 2"],prec)

Σ = [ 1  -0.8;
      -0.8  1]
wT, = MVTangencyP(μe,Σ,0)
printmat(wT;rowNames=["asset 1","asset 2"],prec)

Σ = [ 1  0.8;
      0.8  1]

```

```
wT, = MVTangencyP( $\mu^e$ , $\Sigma$ ,0)
printmat(wT;rowNames=["asset 1","asset 2"],prec)
```

```
asset 1      0.47
asset 2      0.53
```

```
asset 1      0.57
asset 2      0.43
```

```
asset 1      0.51
asset 2      0.49
```

```
asset 1      1.54
asset 2     -0.54
```

Drawing the MV Frontier Revisited (extra)

Recall: with only two investable assets, all portfolios of them are on the MV frontier.

We apply this idea by using the global minimum variance portfolio (see below for code) and the tangency portfolios (see above).

```
"""
Calculate the global minimum variance portfolio
"""
function MVMinimumVarP( $\mu$ , $\Sigma$ ,Rf)
    n      = length( $\mu$ )
     $\mu^e$    =  $\mu$  .- Rf
     $\Sigma_{-1}$  = inv( $\Sigma$ )
    w      =  $\Sigma_{-1} \times \text{ones}(n) / (\text{ones}(n)' \Sigma_{-1} \times \text{ones}(n))$ 
    mu     = w'  $\mu^e$  + Rf
    Std    = sqrt(w'  $\Sigma$  * w)
    return w,mu,Std
end
```

MVMinimumVarP

```

μ = [11.5, 9.5, 6]/100          #expected returns
Σ = [166 34 58;                #covariance matrix
     34 64 4;
     58 4 100]/100^2
Rf = 0.03                      #riskfree return

```

```

(wT,μT,σT) = MVTangencyP(μ,Σ,Rf)
(wg,mug,Stdg) = MVMinimumVarP(μ,Σ,Rf)
printblue("Tangency and minimum variance portfolios: ")
printmat([wT wg];colNames=["tangency","min var"],rowNames=assetNames,prec)

```

Tangency and minimum variance portfolios:

	tangency	min var
A	0.25	-0.02
B	0.68	0.62
C	0.07	0.40

```

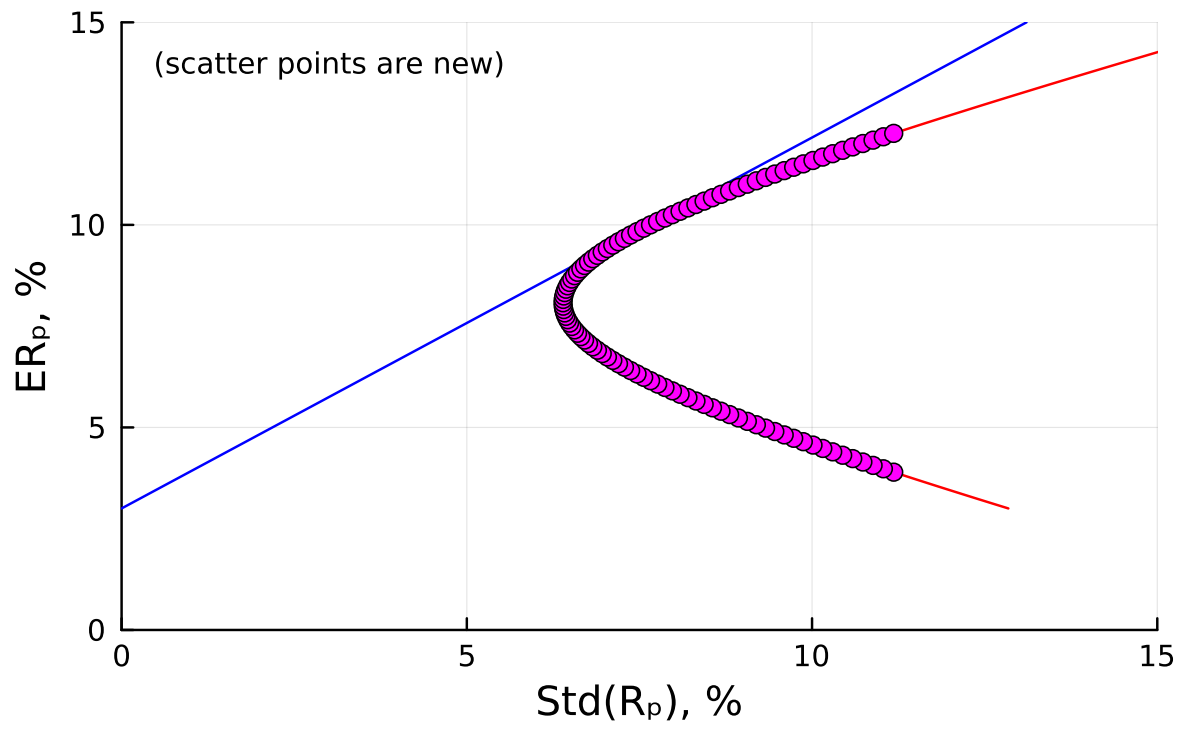
L = 101
v_range = range(-2.5,2.5,length=L)          #try different mixes of wT and wg

(ERv,StdRv) = (fill(NaN,L),fill(NaN,L))
for i in 1:L
    #local w                                #local/global is needed in script
    w = v_range[i]*wT + (1-v_range[i])*wg
    ERv[i] = w'*μ
    StdRv[i] = sqrt(w'*Σ*w)
end

p1 = plot( [StdRp StdRpRf]*100,μ*_range*100,
           legend = nothing,
           linecolor = [:red :blue],
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std(Rp), %",
           ylabel = "ERp, %",
           annotation = (0.5,14,text("(scatter points are new)",8,:left)) )
scatter!(StdRv*100,ERv*100,markercolor=:magenta)
display(p1)

```

Mean vs standard deviation



Mean Variance Frontier with Short Sales Constraints

This notebook calculates mean variance frontiers for two cases (1) when there are no restrictions on the portfolio weights and (2) when we impose the restriction that no weights can be negative.

We use the package [OSQP.jl](#) which solves problems of the type:

$$\min 0.5\theta'P\theta + q'\theta \text{ subject to } l \leq A\theta \leq u.$$

As an alternative, consider the [EfficientFrontier.jl](#) package.

Load Packages and Utility Functions

```
using Printf, LinearAlgebra, SparseArrays, OSQP

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Inputs to MV Calculations

```
μ = [11.5, 9.5, 6]/100      #expected returns
Σ = [166 34 58;             #covariance matrix
     34 64 4;
     58 4 100]/100^2
Rf = 0.03

assetNames = ["A","B","C"]
```

```

printblue("μ in %:")
printmat(μ*100;rowNames=assetNames,prec=2)

printblue("Σ in bp:")
printmat(Σ*10000;rowNames=assetNames,colNames=assetNames,prec=2)

printblue("Rf in %:")
printlnPs(Rf*100)

```

```

μ in %:
A      11.50
B       9.50
C       6.00

```

```

Σ in bp:
      A      B      C
A  166.00  34.00  58.00
B   34.00  64.00   4.00
C   58.00   4.00 100.00

```

```

Rf in %:
    3.000

```

Traditional MV Calculations

(when there are no constraints) from the chapter on MV analysis.

The file included below contains the function `MVCalc()` from the chapter on MV analysis. It calculates the (MV efficient) portfolio standard deviation for a given required return (μ^*), for the case of no restrictions on the portfolio except that the weights sum to one.

A Remark on the Code

- `[MVCalc( $\mu^*$ , $\mu$ , $\Sigma$ )[1] for  $\mu^*$  in  $\mu^*_\text{range}$ ]` is the same as writing a loop over all values in μ^*_range and extracting the first output from `MVCalc()` for each iteration.

```

include("src/MvCalculations.jl");

```

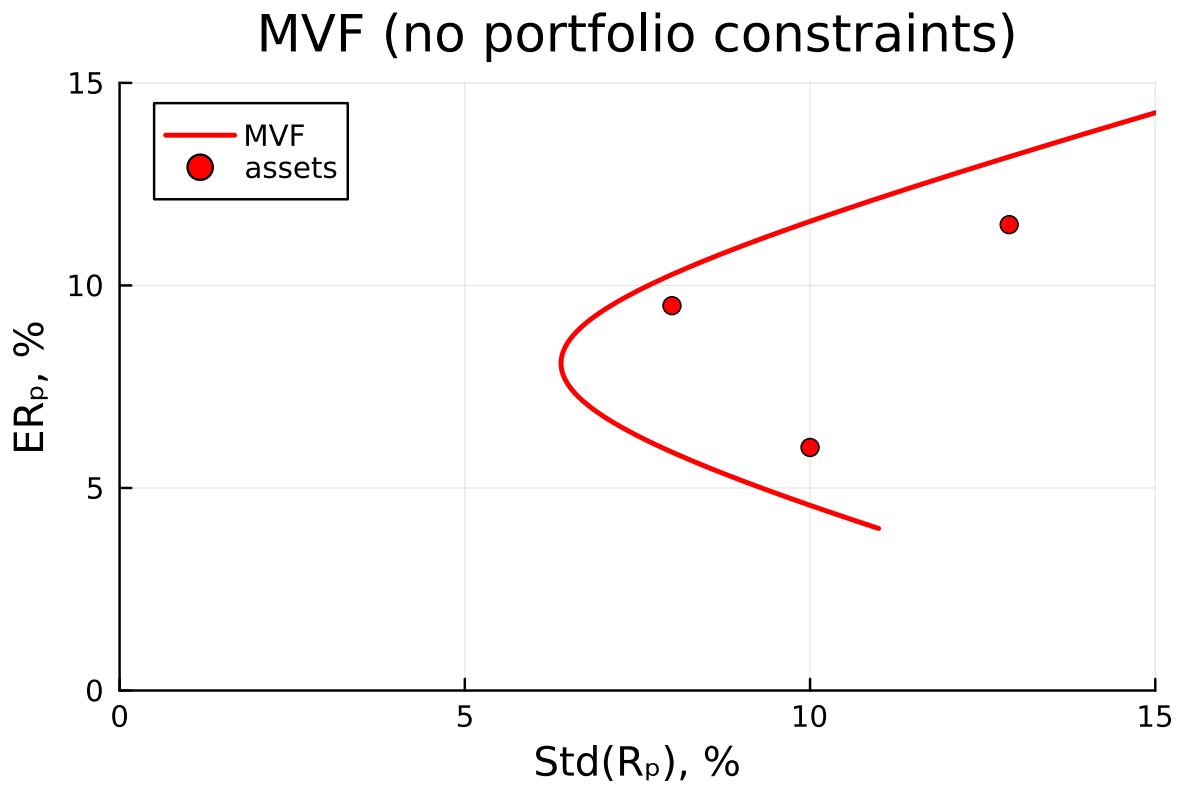
```

μx_range = range(0.04,0.15,length=201)

StdRp = [MVCalc(μx,μ,Σ)[1] for μx in μx_range] #MV calculations without portfolio restrictions

p1 = plot( StdRp*100,μx_range*100,
           linecolor = :red,
           linewidth = 2,
           label = "MVF",
           legend = :topleft,
           xlim = (0,15),
           ylim = (0,15),
           title = "MVF (no portfolio constraints)",
           xlabel = "Std(Rp), %",
           ylabel = "ERp, %" )
scatter!(sqrt.(diag(Σ))*100,μ*100,color=:red,label="assets")
display(p1)

```



MV Frontier when Short Sales are Not Allowed

The code below solves the problem

$$\min \text{Var}(R_p) \text{ s.t. } ER_p = \mu^*,$$

where we also require $w_i \geq 0$ and $\sum_{i=1}^n w_i = 1$.

It is straightforward to add other constraints, for instance, that all weights are between 0 and 0.5.

A Remark on the Code

- The function `MeanVarNoSS( $\mu, \Sigma, \mu^*$ )` takes a vector μ^* as input and loops over those $\mu^*[i]$ values that are feasible (must be between the lowest and highest values in μ).

```
"""
    MeanVarNoSS( $\mu, \Sigma, \mu^*$ _range_range)

Calculate mean variance frontier when short sales are not allowed.

# Input
- ' $\mu::\text{Vector}$ ':      n vector, mean returns
- ' $\Sigma::\text{Matrix}$ ':   nxn, covariance matrix of returns, can contain riskfree assets
- ' $\mu^*\text{\_range}::\text{Vector}$ ': K vector, mean returns to calculate results for

# Output
- ' $\text{StdRp}::\text{Vector}$ ':    K vector, standard deviation of mean-variance portfolio (risky only) with mean
- ' $w_p::\text{Matrix}$ ':      Kxn, portfolio weights of      ""

# Requires
- LinearAlgebra, SparseArrays, OSQP

"""
function MeanVarNoSS( $\mu, \Sigma, \mu^*\text{\_range}$ ) #MV with no short-sales, numerical minimization

    (K,n) = (length( $\mu^*\text{\_range}$ ),length( $\mu$ ))
    vv     = findall( minimum( $\mu$ ) .<=  $\mu^*\text{\_range}$  .<= maximum( $\mu$ ) ) #solve only if feasible

    P = sparse(2* $\Sigma$ )                #P and A must be sparse
    q = zeros(n)
    A = sparse([ $\mu'$ ;ones(1,n);I])
    l = [NaN;1;zeros(n)]
```

```

u = [NaN;1;ones(n)]

prob = OSQP.Model()           #initial set up of problem
settings = Dict(:verbose => false)
OSQP.setup!(prob;P=P,q=q,A=A,l=l,u=u,settings...)

(w_p,StdRp) = (fill(NaN,K,n),fill(NaN,K))
for i in vv                   #loop over (feasible)  $\mu^*$ _range elements
    (l[1],u[1]) = ( $\mu^*$ _range[i], $\mu^*$ _range[i])
    OSQP.update!(prob;l=l,u=u) #update problem
    result = OSQP.solve!(prob)
    w = result.info.status == :Solved ? result.x : NaN
    if !any(isnan,w)
        w_p[i,:] = w
        StdRp[i] = sqrt(w'\Sigma*w)
    end
end

return StdRp, w_p

end

```

MeanVarNoSS

```

StdRp_no_ss = MeanVarNoSS( $\mu$ , $\Sigma$ , $\mu^*$ _range)[1];    #MV calculations with no short sales

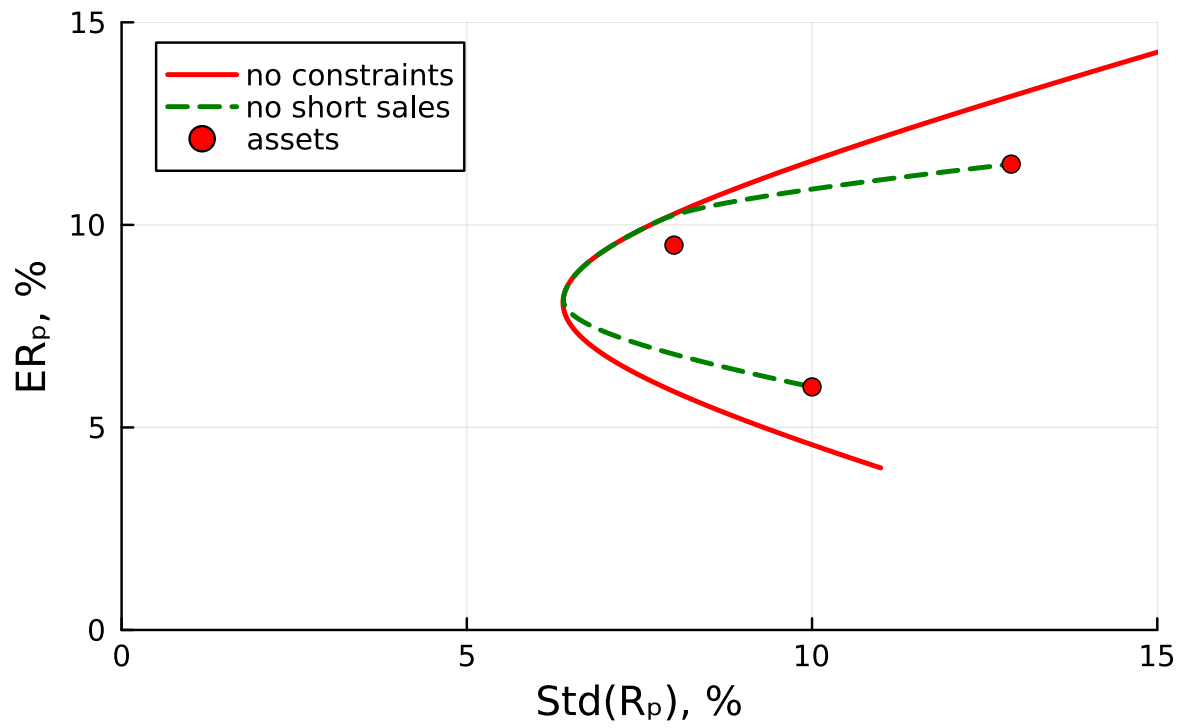
```

```

p1 = plot( [StdRp StdRp_no_ss]*100, $\mu^*$ _range*100,
    linecolor = [:red :green],
    linestyle = [:solid :dash],
    linewidth = 2,
    label = ["no constraints" "no short sales"],
    xlim = (0,15),
    ylim = (0,15),
    legend = :topleft,
    title = "MVF (with/without constraints)",
    xlabel = "Std( $R_p$ ), %",
    ylabel = "ER $_p$ , %" )
scatter!(sqrt.(diag( $\Sigma$ ))*100, $\mu$ *100,color=:red,label="assets")
display(p1)

```

MVF (with/without constraints)



The Inputs to MV Calculations

This notebook estimates (single and multi-) index models. It uses those to construct (alternative) estimates of the covariance matrix of the asset returns.

Load Packages and Extra Functions

```
using Printf, LinearAlgebra, Statistics, DelimitedFiles

include("src/printmat.jl")
include("src/OlsM.jl");
```

Loading Data

We load data from two data files: for the returns on Fama-French equity factors and then also for the 25 Fama-French portfolios. To keep the output simple (easy to display...), we use only 5 of those portfolios.

```
x = readlm("Data/FFmFactorsPs.csv",',',skipstart=1)
Rme = x[:,2]           #market excess return
RSMB = x[:,3]          #small minus big firms
RHML = x[:,4]          #high minus low book-to-market ratio
Rf = x[:,5]            #interest rate

x = readlm("Data/25_Portfolios_5x5.CSV",',',skipstart=1)
R = x[:,2:end]         #returns for 25 FF portfolios
Re = R .- Rf           #excess returns for the 25 FF portfolios
Re = Re[:,[1,7,13,19,25]] #use just 5 assets to make the printing easier

(T,n) = size(Re)       #no. obs and no. test assets

AssetNames = string("Asset ",[1,7,13,19,25])
```

5-element Vector{String}:

```
"Asset 1"  
"Asset 7"  
"Asset 13"  
"Asset 19"  
"Asset 25"
```

Betas

```
x          = [ones(T) Rme]          #regressors  
(b,σ)      = OlsM(Re,x)  
(β,VarRes) = (b[2:2,:],vec(σ).^2)  
  
printblue("β for $n assets, from OLS of Re on constant and Rme:")  
printmat(β,colNames=AssetNames,rowNames=["β on Rme"])
```

β for 5 assets, from OLS of Re on constant and Rme:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
β on Rme	1.398	1.177	1.010	0.989	0.978

Covariance Matrix with Average Correlations

The next cell contains two functions that will help us construct a covariance matrix from (a) estimated standard deviations and (b) a common (average) number for all correlations.

```
"""  
    vech(x,k=0)  
  
Stack the matrix elements on and below the principal diagonal into a vector (k=0).  
With k=-1, instead stacks just the elements below the diagonal.  
"""  
  
function vech(x,k=0)  
    vv = tril(trues(size(x)),k)  
    y = x[vv]  
    return y  
end
```



```

"""
Cov_withSameCorr( $\sigma$ , $\rho$ )

Compute covariance matrix from vector of standard deviations ( $\sigma$ )
and a single correlation (assumed to be the same across all pairs).
"""
function Cov_withSameCorr( $\sigma$ , $\rho$ )
     $\sigma$  = vec( $\sigma$ )                #to make sure it is a vector
    n = length( $\sigma$ )
    CorrMat = ones(n,n)* $\rho$  + (1- $\rho$ )*I    #corr matrix, correlation =  $\rho$ 
    CovMat = ( $\sigma$ * $\sigma'$ ).*CorrMat
    return CovMat
end

```

Cov_withSameCorr

```

S = cov(Re) #Covariance matrix calculated from data (to compare with)
#printblue("Covariance matrix calculated from data (to compare with):")
#printmat(S)

C = cor(Re)                #nxn correlation matrix
#printblue("Correlation matrix:")
#printmat(C;rowNames=AssetNames,colNames=AssetNames)



$\rho$ _avg = mean(vech(C,-1))


#printblue("Average correlation:")
#printlnPs( $\rho$ _avg)

```

Correlation matrix:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	1.000	0.902	0.772	0.694	0.561
Asset 7	0.902	1.000	0.909	0.833	0.658
Asset 13	0.772	0.909	1.000	0.920	0.770
Asset 19	0.694	0.833	0.920	1.000	0.796
Asset 25	0.561	0.658	0.770	0.796	1.000

Average correlation:

0.781

```

Σ1 = Cov_withSameCorr(std(Re,dims=1),ρ_avg)

printblue("Covariance matrix assuming same correlation:")
printmat(Σ1;rowNames=AssetNames,colNames=AssetNames)

printblue("Difference to data:")
printmat(Σ1-S;rowNames=AssetNames,colNames=AssetNames)

```

Covariance matrix assuming same correlation:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	65.852	39.506	33.010	32.899	37.032
Asset 7	39.506	38.811	25.342	25.256	28.429
Asset 13	33.010	25.342	27.097	21.104	23.755
Asset 19	32.899	25.256	21.104	26.915	23.675
Asset 25	37.032	28.429	23.755	23.675	34.101

Difference to data:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	0.000	-6.104	0.404	3.684	10.451
Asset 7	-6.104	-0.000	-4.122	-1.677	4.483
Asset 13	0.404	-4.122	-0.000	-3.733	0.359
Asset 19	3.684	-1.677	-3.733	0.000	-0.445
Asset 25	10.451	4.483	0.359	-0.445	0.000

Covariance Matrix from a Single-Index Model

Step 1: Do Regressions

A Remark on the Code

- The function `Olsm` is included in the file `Olsm.jl` (see the first cell of the notebook). It does OLS estimation and reports the point estimates and standard deviations of the residuals.
- The functions reports for several dependent variables (different columns in the first function input).

```

x          = [ones(T) Rme]          #regressors
(b,σ)      = Olsm(Re,x)
(β,VarRes) = (b[2:2,:],vec(σ).^2)

```

```
printblue("β for $n assets, from OLS of Re on constant and Rme:")
printmat(β,colNames=AssetNames,rowNames=["β on Rme"])
```

β for 5 assets, from OLS of Re on constant and Rme:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
β on Rme	1.398	1.177	1.010	0.989	0.978

Step 2: Use OLS Estimates to Calculate the Covariance Matrix

The single index model implies that the covariance of assets i and j is

$$\sigma_{ij} = \beta_i \beta_j \text{Var}(R_{mt}) + \text{Cov}(\varepsilon_{it}, \varepsilon_{jt}),$$

but where we *assume* that $\text{Cov}(\varepsilon_{it}, \varepsilon_{jt}) = 0$ if $i \neq j$

The betas are typically estimated by a linear regression (OLS), as above.

A Remark on the Code

- The `CovFromIndexModel()` function can handle both the case with one factor (as used here) and with several factors (a multi-index model, as will be used further down). In the code β is a $K \times n$ matrix, where K is the number of factors ($K = 1$ for the single index model) and n is the number of assets.

```
"""
    CovFromIndexModel(b,VarRes,Ω)

Calculate covariance matrix from a multi-factor model.

b is Kxn where K is the number of factors and n is the number of assets.

Cov(Ri,Rj) = b_i'Ω*b_j, where Ω is the Cov(indices) and b_i is the vector of regression
coefficients when regressing Ri on a constant and the indices (and b_j is for asset j).
"""
function CovFromIndexModel(b,VarRes,Ω)    #coefs for regression i is in b[:,i]
    n = length(VarRes)
    CovR = fill(NaN,n,n)
    for i in 1:n, j in 1:n                #loop over both i and j
        CovR[i,j] = i == j ? b[:,i]'Ω*b[:,i] + VarRes[i] : b[:,i]'Ω*b[:,j]
    end
    return CovR
end
```

CovFromIndexModel

```
 $\sigma_{mm} = \text{var}(R_{me})$ 
 $\Sigma_2 = \text{CovFromIndexModel}(\beta, \text{VarRes}, \sigma_{mm})$ 

printblue("Covariance matrix calculated from betas:")
printmat( $\Sigma_2$ ; rowNames=AssetNames, colNames=AssetNames)

printblue("Difference to data:")
printmat( $\Sigma_2 - S$ ; rowNames=AssetNames, colNames=AssetNames)
```

Covariance matrix calculated from betas:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	65.852	34.786	29.860	29.238	28.908
Asset 7	34.786	38.811	25.137	24.613	24.336
Asset 13	29.860	25.137	27.097	21.128	20.890
Asset 19	29.238	24.613	21.128	26.915	20.455
Asset 25	28.908	24.336	20.890	20.455	34.101

Difference to data:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	0.000	-10.824	-2.746	0.022	2.328
Asset 7	-10.824	-0.000	-4.326	-2.320	0.390
Asset 13	-2.746	-4.326	0.000	-3.709	-2.505
Asset 19	0.022	-2.320	-3.709	0.000	-3.665
Asset 25	2.328	0.390	-2.505	-3.665	-0.000

Covariance Matrix from a Multi-Index Model

A multi-index model is based on

$$R_{it} = a_i + b_i' I_t + \varepsilon_{it},$$

where b_i is a K -vector of slope coefficients.

If Ω is the covariance matrix of the indices I_t , then the covariance of assets i and j is

$$\sigma_{ij} = b_i' \Omega b_j + \text{Cov}(\varepsilon_{it}, \varepsilon_{jt}),$$

but where we assume that $\text{Cov}(\varepsilon_{it}, \varepsilon_{jt}) = 0$ if $i \neq j$

```

x          = [ones(T) Rme RSMB RHML]          #regressors

(b,σ)      = OLSM(Re,x)
(β,VarRes) = (b[2:end,:],vec(σ).^2)

printblue("OLS slope coefficients:")
printmat(β,colNames=AssetNames,rowNames=["β on Rme", "β on RSMB", "β on RHML"])

```

OLS slope coefficients:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
β on Rme	1.098	1.019	0.989	1.030	1.129
β on RSMB	1.386	0.929	0.437	0.215	-0.146
β on RHML	-0.273	0.117	0.425	0.571	0.861

```

Ω = cov(x[:,2:end])          #covariance matrix of the (non-constant) factors

Σ3 = CovFromIndexModel(β,VarRes,Ω)

printblue("Covariance matrix calculated from betas:")
printmat(Σ3;rowNames=AssetNames,colNames=AssetNames)

printblue("Difference to data:")
printmat(Σ3-S;rowNames=AssetNames,colNames=AssetNames)

```

Covariance matrix calculated from betas:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	65.852	45.817	33.659	29.746	23.868
Asset 7	45.817	38.811	28.735	26.468	23.366
Asset 13	33.659	28.735	27.097	23.863	23.402
Asset 19	29.746	26.468	23.863	26.915	24.597
Asset 25	23.868	23.366	23.402	24.597	34.101

Difference to data:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	0.000	0.207	1.052	0.531	-2.713
Asset 7	0.207	-0.000	-0.729	-0.465	-0.579
Asset 13	1.052	-0.729	0.000	-0.974	0.006
Asset 19	0.531	-0.465	-0.974	0.000	0.477
Asset 25	-2.713	-0.579	0.006	0.477	-0.000

A Shrinkage Estimator

is

$$\Sigma = \delta F + (1 - \delta)S,$$

where $0 < \delta < 1$, F is the “target covariance matrix” (for instance, from the constant correlation approach or one of the index models) and S is the sample variance-covariance matrix.

This approach will (by definition) be worse in-sample, but may well be better out-of-sample (a forecast for the coming period).

```
 $\delta = 0.6$   
 $\Sigma_4 = \delta * \Sigma_1 + (1 - \delta) * S$   
  
printmat( $\Sigma_4$ ; rowNames=AssetNames, colNames=AssetNames)
```

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
Asset 1	65.852	41.947	32.849	31.425	32.851
Asset 7	41.947	38.811	26.990	25.927	26.636
Asset 13	32.849	26.990	27.097	22.597	23.611
Asset 19	31.425	25.927	22.597	26.915	23.853
Asset 25	32.851	26.636	23.611	23.853	34.101

Portfolio Choice 2

This notebook solves a portfolio optimization problem. The objective function of the investor trades off the portfolio expected return and variance. There are no restrictions on the portfolio weights, except that they must sum to 1 (across all assets, risky and riskfree).

Load Packages and Extra Functions

```
using Printf, LinearAlgebra

include("src/printmat.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)  #OR :svg
```

From Chapter on Mean-Variance Analysis

The file included below contains functions from the chapter on MV analysis, in particular, `MVTangencyP()` which calculates the tangency portfolio, `MVCalc()` which calculates the MV frontier from risky assets, and `MVCalcRf()` which does the same but from both risky assets and a riskfree asset.

```
include("src/MvCalculations.jl");
```

Optimal Portfolio Choice: A Risky and a Riskfree Asset

Consider the objective function (“utility”) $ER_p - k/2 \times \text{Var}(R_p)$.

It depends on the weight v on the risky asset (and $1 - v$ on the riskfree asset), since both terms do (see the previous cells).

The optimal portfolio weight is

$$v = \frac{\mu^e}{k\sigma^2}$$

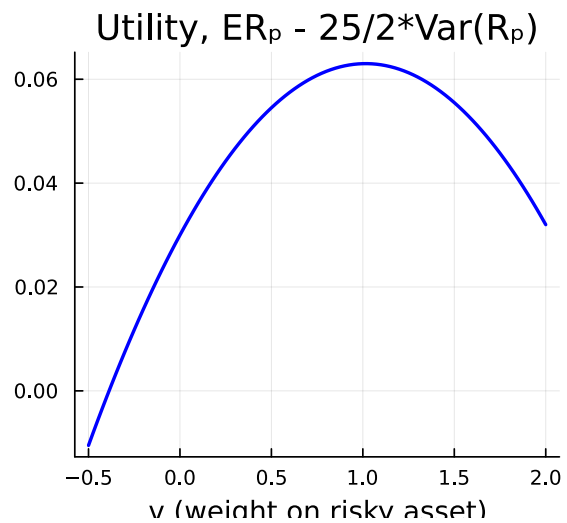
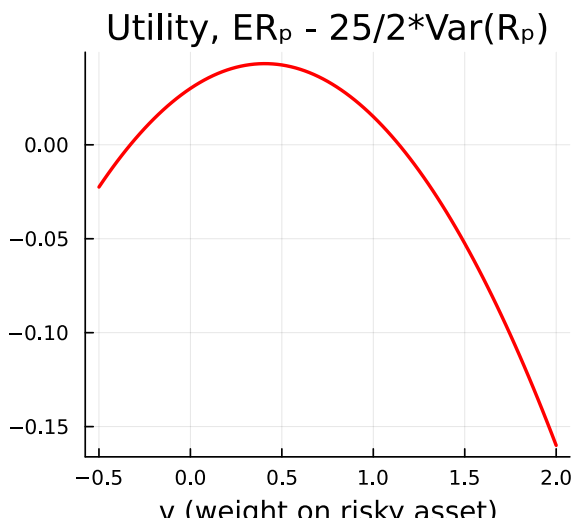
```
μ = 9.5/100 #expected return of the risky asset
σ = 8/100   #std of the risky asset
Rf = 3/100   #risk free return (interest rate)

k1 = 25      #risk aversion
k2 = 10
μe = μ - Rf  #expected excess return

v_range = range(-0.5,2,length=101)      #trying different portfolio weights
ERp = v_range*μe .+ Rf                  #a vector of the same length as v_range
VarRp = v_range.^2*σ^2                  #v_range.^2 to square each element of v_range
Util1 = ERp - k1/2*VarRp
Util2 = ERp - k2/2*VarRp

p1 = plot( v_range,Util1,
            linecolor = :red,
            linewidth = 2,
            legend = false,
            title = "Utility, ERp - $k1/2*Var(Rp)",
            xlabel = "v (weight on risky asset)" )
p2 = plot( v_range,Util2,
            linecolor = :blue,
            linewidth = 2,
            legend = false,
            title = "Utility, ERp - $k1/2*Var(Rp)",
            xlabel = "v (weight on risky asset)" )

pAll = plot( p1,p2,                        #combine the subplots
             layout = @layout[a a],
             size = (700,300) )
display(pAll)
```

```
vopt1 = μe/(k1*σ2)      #optimal solution
vopt2 = μe/(k2*σ2)

printblue("Optimal weights on risky and riskfree assets: ")
xut = hcat([vopt1;1-vopt1],[vopt2;1-vopt2])
printmat(xut;rowNames=["Risky","Riskfree"],colNames=["k=$k1","k=$k2"])

printred("compare with the figure")
```

Optimal weights on risky and riskfree assets:

	k=25	k=10
Risky	0.406	1.016
Riskfree	0.594	-0.016

compare with the figure

Optimal Portfolio Choice: Several Risky Assets and a Riskfree

An investor who maximizes

$$ER_p - \frac{k}{2} \text{Var}(R_p),$$

subject to

$$R_p = v' R^e + R_f$$

will pick the portfolio weights (on the risky assets)

$$v = \frac{1}{k} \Sigma^{-1} \mu^e$$

The portfolio weight on the riskfree asset is $1 - \mathbf{1}'v$

```

"""
Calculate optimal portfolio weights
"""
function OptimalPortfolio(μ,Σ,Rf,k)
    μe    = μ .- Rf           #expected excess returns
    v      = inv(Σ) * μe/k    #optimal weights risky assets, 1-sum(v) in riskfree
    ERp    = v'μe .+ Rf      #expected return and std of optimal portfolio
    StdRp  = sqrt(v'Σ*v)
    return v,ERp,StdRp
end

```

OptimalPortfolio

```

"""
    μΣRf(TwoAssets=true)

The parameter values for the 2- and 3-asset examples
"""
function μΣRf(TwoAssets=true)
    if TwoAssets
        μe = [8.5, 6.5]/100           #excess returns
        Σ  = [166 34;                #covariance matrix
              34 64]/100^2
        rowNames = ["Asset 1","Asset 2"]
    else
        μe = [11.5, 9.5, 6]/100
        Σ  = [166 34 58;
              34 64 4;
              58 4 100]/100^2
        rowNames = ["Asset A","Asset B","Asset C"]
    end
    Rf = 3/100                       #riskfree rate
    return μe, Σ, Rf, rowNames
end;

```

An Example with 2 Risky Assets and a Riskfree Asset

We first solve an example with 2 risky assets and a riskfree asset. In this case, we can plot how the objective function depends on the portfolio weights.

```
(μ,Σ,Rf,rowNames) = μΣRf(true)    #parameter values, two assets
μe = μ .- Rf;                      #average excess returns

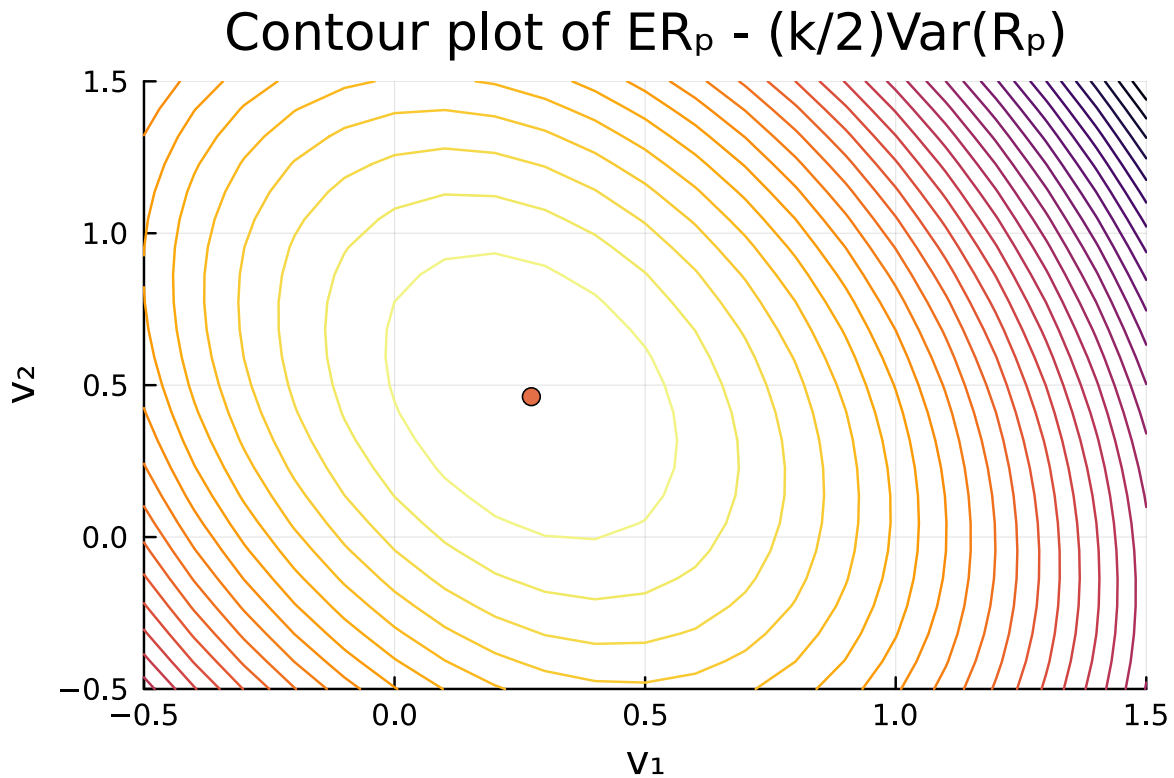
#println("μ*100 and Σ*100^2")
#printmat(μ*100)
#printmat(Σ*10_000)

k = 9

(n1,n2) = (21,23)
v1_range = range(-0.5,1.5,length=n1)    #portfolio weights asset 1
v2_range = range(-0.5,1.5,length=n2)    # ""                                2

Util = fill(NaN,n1,n2)
for i in 1:n1, j in 1:n2                #loop across portfolio weights
    #local v                                #local/global is needed in script
    v = [v1_range[i];v2_range[j]]
    Util[i,j] = v'μe + Rf - (k/2)*v'Σ*v
end

p1 = contour( v1_range,v2_range,Util',    #notice the transpose
              xlims = (-0.5,1.5),
              ylims = (-0.5,1.5),
              legend = false,
              levels = 31,
              title = "Contour plot of ERp - (k/2)Var(Rp)",
              xlabel = "v1",
              ylabel = "v2")
scatter!([0.273],[0.462])
display(p1)
```



The next cell calls on `OptimalPortfolio()` to calculate the optimal weights - and also compares with the tangency portfolio.

```
vOpt, = OptimalPortfolio(μ,Σ,Rf,k)           #find optimal portfolio

printblue("optimal portfolio weights on risky assets and riskfree when k = $k:")
printmat([vOpt;(1-sum(vOpt))];rowNames=[rowNames;"Riskfree"])
```

optimal portfolio weights on risky assets and riskfree when k = 9:

Asset 1	0.273
Asset 2	0.462
Riskfree	0.264

MV Preferences Give a Portfolio on the MV Frontier

...illustrated by a case with three risky assets and a riskfree. We also compare the optimal portfolios for different investors (with different risk aversions).

We then show that the optimal portfolios are on the mean-variance frontier.

```

(μ,Σ,Rf,rowNames) = μΣRf(false)          #parameter values, 3 assets
μe = μ .- Rf

(vD,muD,StdD) = OptimalPortfolio(μ,Σ,Rf,28)      #high risk aversion
(vE,muE,StdE) = OptimalPortfolio(μ,Σ,Rf,8.8)     #low risk aversion

(wT,muT,StdT) = MVTangencyP(μ,Σ,Rf)             #tangency portfolio

printblue("optimal portfolio weights:")
printmat([vD;1-sum(vD)],[vE;1-sum(vE)],[wT;1-sum(wT)];colNames=["D (high k)","E (low k)","T"],
rowNames=[rowNames;"Riskfree"])

printblue("optimal weights/tangency portfolio:")
printmat(vD./wT,vE./wT;colNames=["D (high k)","E (low k)"],rowNames)

printblue("optimal portfolios are effectively c*tangency portfolio (+ a position in the riskfree)")

```

optimal portfolio weights:

	D (high k)	E (low k)	T
Asset A	0.110	0.350	0.248
Asset B	0.302	0.962	0.682
Asset C	0.031	0.099	0.070
Riskfree	0.556	-0.411	0.000

optimal weights/tangency portfolio:

	D (high k)	E (low k)
Asset A	0.444	1.411
Asset B	0.444	1.411
Asset C	0.444	1.411

optimal portfolios are effectively c*tangency portfolio (+ a position in the riskfree)

```

L = 101
μstar_range = range(Rf,0.15,length=L)

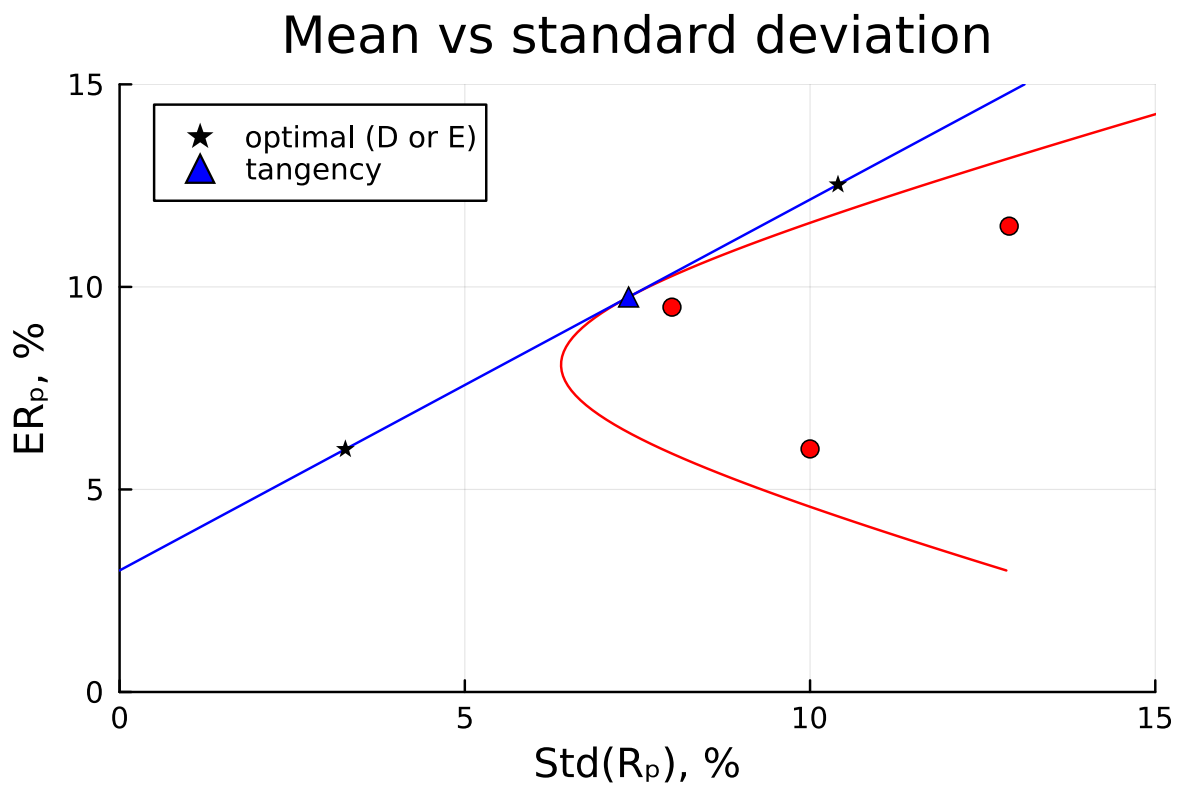
StdRp = fill(NaN,L,2)          #loop over required average returns (μstar)
for i in 1:L
    StdRp[i,1] = MVCalc( μstar_range[i],μ,Σ)[1]      #risky only
    StdRp[i,2] = MVCalcRf(μstar_range[i],μ,Σ,Rf)[1]  #risky and riskfree
end

```

```

p1 = plot( StdRp*100,μstar_range*100,
           label = "",
           linecolor = [:red :blue],
           xlim = (0,15),
           ylim = (0,15),
           title = "Mean vs standard deviation",
           xlabel = "Std(Rp), %",
           ylabel = "ERp, %" )
scatter!(sqrt.(diag(Σ))*100,μ*100,color=:red,label="")
scatter!([StdD,StdE]*100,[μD,μE]*100,color=:black,marker=:star,label="optimal (D or E)")
scatter!([StdT]*100,[μT]*100,color=:blue,marker=:utriangle,label="tangency",legend=:topleft)
display(p1)

```



CAPM

This notebook summarizes the implications of CAPM and then tests them by OLS.

Load Packages and Extra Functions

```
using Printf, DelimitedFiles, Statistics

include("src/printmat.jl")
include("src/OlsGMNW.jl");           #contains a function for OLS
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

The Theoretical Predictions of CAPM

The following section illustrates the theoretical predictions of CAPM by taking the following steps:

1. define a set of investable assets
2. find the tangency portfolio
3. calculate the betas of each asset against the tangency portfolio
4. check whether the average returns are in accordance with CAPM: $ER_i = R_f + \beta_i(\mu_T - R_f)$ or equivalently $ER_i^e = \beta_i\mu_T^e$, where superscript e indicates an excess return.

Characteristics of Three Assets: Means, Variances and Covariances

```

μ = [0.115, 0.095, 0.06]    #expected returns
Σ = [166  34  58;           #covariance matrix
     34  64   4;
     58   4 100]/100^2
Rf = 0.03

assetNames = ["A","B","C"]

printblue("expected returns, in %:")
printmat(μ*100;rowNames=assetNames)

printblue("covariance matrix, in bp:")
printmat(Σ*10_000;colNames=assetNames,rowNames=assetNames)

```

expected returns, in %:

A	11.500
B	9.500
C	6.000

covariance matrix, in bp:

	A	B	C
A	166.000	34.000	58.000
B	34.000	64.000	4.000
C	58.000	4.000	100.000

The Tangency Portfolio

The file included below contains, among other things, the function `MVTangencyP()` from the chapter on MV analysis. It calculates the tangency portfolio.

```
include("src/MvCalculations.jl");
```

```

(wT,μT,σT) = MVTangencyP(μ,Σ,Rf)    #tangency portfolio weights and implied mean return and s
printblue("Tangency portfolio weights:")
printmat(wT;rowNames=assetNames)

printblue("mean and std of tangency portfolio:")
printmat([μT,σT];rowNames=["μT","σT"])

```

Tangency portfolio weights:

A 0.248
 B 0.682
 C 0.070

mean and std of tangency portfolio:

μ_T 0.097
 σ_T 0.074

of the Assets (Theory)

1. The foc for optimal portfolio choice says: $ER^e = ck\Sigma w_T$, where w_T is the tangency portfolio.
2. The covariance of R_i and R_T (σ_{iT}) is the i th row of Σw_T . (Alternatively, $w_i \Sigma w_T$, where w_i has 1 in position i and 0 elsewhere.)
3. Rewrite and calculate $\beta_i = \sigma_{iT} / \sigma_T^2$.
4. CAPM: $ER_i^e = \beta_i R_T^e$.

```
β = Σ*wT/σT^2          #vector of betas
ER_CAPM = Rf .+ β*(μT-Rf)  #ER_CAPM is a vector since β is a vector

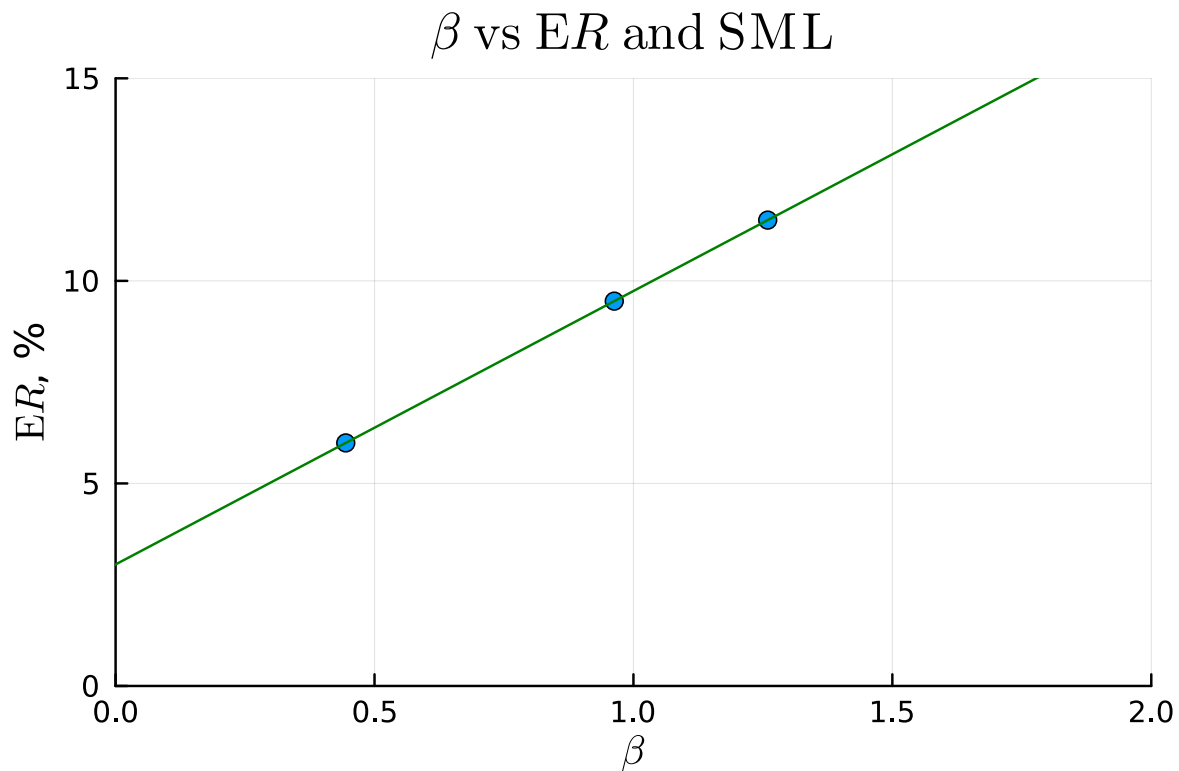
b = range(0,2,length=5)
SML = Rf .+ b*(μT-Rf)      #SML

printblue("β and ER according to CAPM: ")
printmat(β,ER_CAPM;rowNames=assetNames,colNames=["β","ERe (CAPM)"],width=15)
```

β and ER according to CAPM:

	β	ER ^e (CAPM)
A	1.259	0.115
B	0.963	0.095
C	0.444	0.060

```
p1 = scatter( β,ER_CAPM*100,          #points on the security market line
             xlim = (0,2),
             ylim = (0,15),
             legend = false,
             title = L"\beta \mathrm{\ } vs\ E\}R \mathrm{\ } and \ SML}",
             xlabel = L"\beta",
             ylabel = L"$\mathrm{E\}R$, %" )
Plots.abline!((μT-Rf)*100,Rf*100,linecolor=:green,line=:solid)
display(p1)
```



An Empirical Test of CAPM

The next section performs an empirical test of CAPM. First, we load data. Then, we run linear regressions and test whether the intercepts are zero (the CAPM prediction) or not. (We test each asset separately.)

Loading Data

We load data from two data files: for the returns on Fama-French equity factors and then also for the 25 Fama-French portfolios. To keep the output simple, we use only 5 of those return series.

```
x = readlm("Data/FFmFactorsPs.csv",',',skipstart=1)
Rme = x[:,2]           #market excess return
RSMB = x[:,3]          #small minus big firms
RHML = x[:,4]          #high minus low book-to-market ratio
Rf = x[:,5]            #interest rate
```

```

x = readlm("Data/25_Portfolios_5x5.CSV",',',skipstart=1)
R = x[:,2:end]                #returns for 25 FF portfolios
Re = R .- Rf                  #excess returns for the 25 FF portfolios
Re = Re[:,[1,7,13,19,25]]     #use just 5 assets to make the printing easier

(T,n) = size(Re)              #no. obs and no. test assets

AssetNames = string("Asset ",[1,7,13,19,25]);

```

OLS Estimation and Testing $\alpha = 0$

We now use the market return as a proxy for the tangency portfolio - and test if CAPM holds for the test assets.

To do that, we estimate (α_i, β_i) in the CAPM regression

$$R_{it}^e = \alpha_i + \beta_i R_{mt}^e + \varepsilon_{it},$$

where R_{it}^e and R_{mt}^e are excess return, and then test if $\alpha_i = 0$. This corresponds to the CAPM prediction that $ER_i^e = \beta_i \mu_m^e$. Compared with the theoretical expression above, we use the market excess return as a measure of the excess return of the tangency portfolio.

A Remark on the Code

- The function `olsNW` is included in the file `ols.jl` (see the first cell of the notebook). It does OLS estimation and reports the point estimates, residuals and more. The variance-covariance matrix of the coefficients is based on the Gauss-Markov assumptions.
- In the call on `printmat()`, `;rowNames` (notice: semicolon) is the same as `,rowNames=rowNames`.

```

x = [ones(T) Rme]                #regressors

(alpha,tstat) = (fill(NaN,n),fill(NaN,n))
for i in 1:n                      #loop over the different test assets
    #local b_i, residual, Covb          #local/global is needed in script
    (b_i,_,_,Covb,) = olsNW(Re[:,i],x,1)
    alpha[i] = b_i[1]                #estimated alpha
    tstat[i] = (alpha[i]-0)/sqrt(Covb[1,1]) #t-stat of H0: true alpha=0
end

printblue("OLS intercepts and t-stats:")

```

```
rowNames = [" $\alpha$ ", "t-stat"]
printmat([" $\alpha$ "; tstat']; colNames=AssetNames, rowNames)
```

OLS intercepts and t-stats:

	Asset 1	Asset 7	Asset 13	Asset 19	Asset 25
α	-0.687	0.062	0.112	0.218	0.149
t-stat	-3.447	0.525	1.200	2.061	0.971

Risk Measures

This notebook illustrates the *Value at Risk* (VaR), *Expected Shortfall* and other downside risk measures. To improve the performance, a model for time-varying risk is estimated and incorporated into the calculations. The model is backtested on a data set.

We use the [Distributions.jl](#) package to calculate probability values and quantiles.

Load Packages and Extra Functions

```
using Printf, Dates, Distributions, DelimitedFiles

include("src/printmat.jl")
include("src/UtilityFnMFT.jl");
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Value at Risk (VaR) for a $N(\mu, \sigma^2)$ Return

$\text{VaR}_{95\%} = -(5^{th} \text{ percentile of the return distribution})$

With a $N(\mu, \sigma^2)$ distribution this gives approximately

$$\text{VaR}_{95\%} \approx -(\mu - 1.64\sigma)$$

A Remark on the Code

- The Distributions.jl package defines a normal distribution as `Normal( $\mu$ , $\sigma$ )`. Notice that it uses the standard deviation, not the variance. For instance, to calculate the 5th quantile, use `quantile(Normal( $\mu$ , $\sigma$ ),0.05)` and to calculate the pdf value at each element of a vector `x`, use `pdf.(Normal( $\mu$ , $\sigma$ ),x)`.

```
"""
    VaRNormal( $\mu$ , $\sigma$ , $\alpha$ )

Calculate VaR at a confidence level  $\alpha$  (eg. 0.95) from a N() distribution with mean  $\mu$  and standard deviation  $\sigma$ 
"""
function VaRNormal( $\mu$ , $\sigma$ , $\alpha$ )
    c = quantile(Normal(0,1),1- $\alpha$ )
    VaR = -( $\mu$  + c* $\sigma$ )          #same as -quantile(Normal( $\mu$ , $\sigma$ ),1- $\alpha$ )
    return VaR
end
```

VaRNormal

```
( $\mu$ , $\sigma$ ) = (8,16)

VaR_b = -( $\mu$  - 1.64* $\sigma$ )
VaR    = VaRNormal( $\mu$ , $\sigma$ ,0.95)

printblue("with  $\mu$ =$ $\mu$  and  $\sigma$ =$ $\sigma$ , we have approximately:\n")
printmat([ $\mu$  - 1.64485* $\sigma$ ,VaR_b,VaR];rowNames=["5th quantile"," $\approx$  VaR 95%","VaR 95%"])
```

with $\mu=8$ and $\sigma=16$, we have approximately:

5th quantile	-18.318
$\approx$ VaR 95%	18.240
VaR 95%	18.318

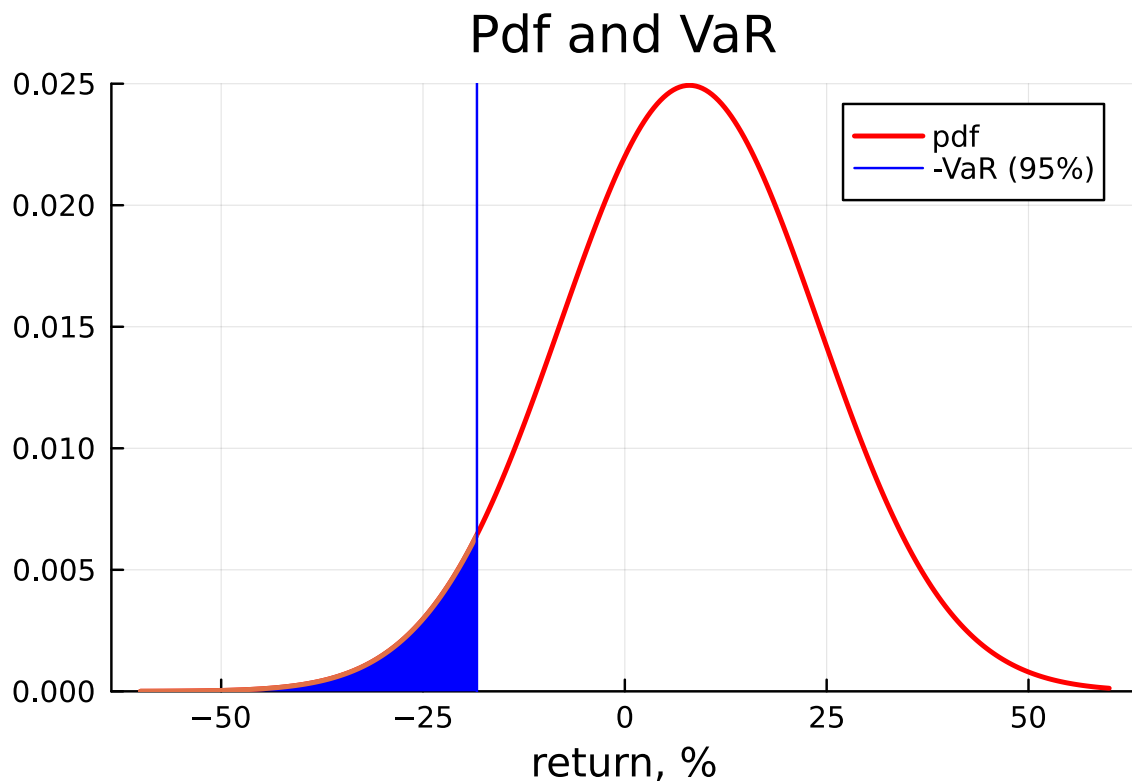
```
R      = range(-60,60,length=301)
pdfR   = pdf.(Normal( $\mu$ , $\sigma$ ),R)
Rb     = R[R .<= -VaR]          #or filter(<=(-VaR95),R)

p1 = plot( R,pdfR,
            linecolor = :red,
            linewidth  = 2,
```

```

ylims = (0,0.025),
label = "pdf",
title = "Pdf and VaR",
xlabel = "return, %" )
plot!(Rb,pdf.(Normal(μ,σ),Rb),fillcolor=:red,linewidth=2,fill=(0,:blue),label="")
vline!([-VaR],linecolor=:blue,label="-VaR (95%)")
display(p1)

```



Loading Daily S&P 500 Data

Remark on the Code

The first column of the data file contains dates in the form 02/01/1979, which will be interpreted as strings. The `x` matrix will be an `Any` matrix, so that it can hold both strings and numbers. For this reason, we convert the numerical columns to floating point numbers. This is not strictly necessary, but might speed up some calculations.

(Other packages, eg. [CSV.jl](#), might do a better job in detecting the type(s) of the data.)

```

xx = readdlm("Data/RSP500Daily.csv",',',skipstart=1)  #net returns
dN = Date.(xx[:,1])
R = Float64.(xx[:,2])
xx = nothing

(dN,R) = excise(dN,R)                                #get rid of missing values
T = length(R)

println("Days in the sample: $T")

P = cumprod(1 .+ R/100);                             #index level

```

Days in the sample: 11590

Backtesting a Static VaR from N() on Data

To backtest a VaR model, we study the relative frequency of loss $> \text{VaR}$, that is of $-R > \text{VaR}$.

Backtesting for the Full Sample

The code below does this for different confidence levels (0.95,0.96,...) of the VaR. For instance, at the 0.95 confidence levels we 1. calculate the VaR as the (negative of the) 0.05 quantile of a normal distribution (with the mean and std estimated from the sample, which is just one possible choice) 2. count the relative frequency of the loss $> \text{VaR}$ (it should be 0.05).

Then repeat for other confidence levels, eg. 0.96.

```

(μemp,σemp) = (mean(R),std(R))

VaR_95      = VaRNormal(μemp,σemp,0.95)    #mean and std of data (empirical)
VaR_99      = VaRNormal(μemp,σemp,0.99)
BreakFreq_95 = mean(R .< -VaR_95)          #freq of breaking the VaR
BreakFreq_99 = mean(R .< -VaR_99)

xut = 100*hcat([0.05,0.01],[BreakFreq_95,BreakFreq_99])
printmat(xut;colNames=["nominal","empirical"],prec=1)

```


nominal	empirical
5.0	4.1
1.0	1.7

Backtesting on a Moving Data Window

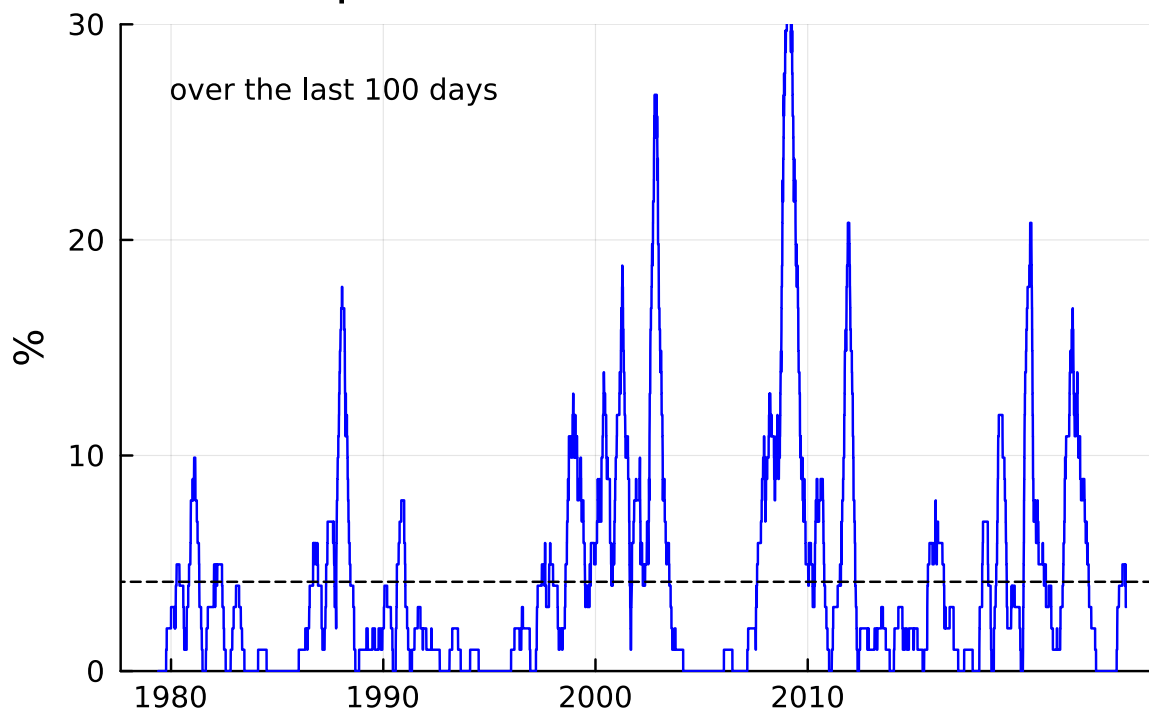
The code below also studies the relative frequency of loss > VaR, but over a *moving data window* of 100 days. This allows us to investigate if there are long periods of failures of the VaR model. To be parsimonious, we do this for a single confidence level, 0.95.

```
BreakFreq = fill(NaN,T)           #vector, freq(R < -VaR) on moving data window
for t in 101:T                     #loop over end-points of the data window
    BreakFreq[t] = mean(R[t-100:t] .< -VaR_95)
end
```

```
xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")
```

```
p1 = plot( dN,BreakFreq*100,
            linecolor = :blue,
            ylim = (0,30),
            legend = false,
            xticks = (xTicksLoc,xTicksLab),
            title = "Freq of R < (-1)static VaR 95%",
            ylabel = "%",
            annotation = (Date(1980),27,text("over the last 100 days",8,:left)) )
hline!([BreakFreq_95*100],linecolor=:black,line=(:dash,1))
display(p1)
```

Freq of $R < (-1)\text{static VaR } 95\%$



A Simple Dynamic VaR with Time-Varying Volatility

To improve the performance of the VaR model, we combine a simple time series model for volatility with an N()-based VaR calculation.

We first construct an simple estimate of σ_t^2 and μ_t as a backward looking exponential moving average (cf. RiskMetrics)

$$\sigma_t^2 = \lambda \sigma_{t-1}^2 + (1 - \lambda)(R_{t-1} - \mu_{t-1})^2, \text{ where } \mu_t = \lambda \mu_{t-1} + (1 - \lambda)R_{t-1},$$

where we use $\lambda \approx 0.94$.

We then redo the $\text{VaR}_{95\%}$ calculation using

$$\text{VaR}_t = -(\mu_t - 1.64\sigma_t)$$

and study if it has better properties than the static VaR.

```

"""
    EWMA(x,λ,μ_start,σ²_start)

Calculate an EWMA mean and variance
"""
function EWMA(x,λ,μ_start,σ²_start)
    T = length(x)
    (μ,σ²) = (fill(μ_start,T),fill(σ²_start,T))
    for t in 2:T
        μ[t] = λ*μ[t-1] + (1-λ)*x[t-1]
        σ²[t] = λ*σ²[t-1] + (1-λ)*(x[t-1]-μ[t-1])^2
    end
    return μ,σ²
end

```

EWMA

```

λ = 0.94
(μ,σ²) = EWMA(R,λ,μ_emp,σ_emp^2)           #EWMA estimates of (μ,σ²)

VaRd = VarNormal.(μ,sqrt.(σ²),0.95);        #VaR for each date

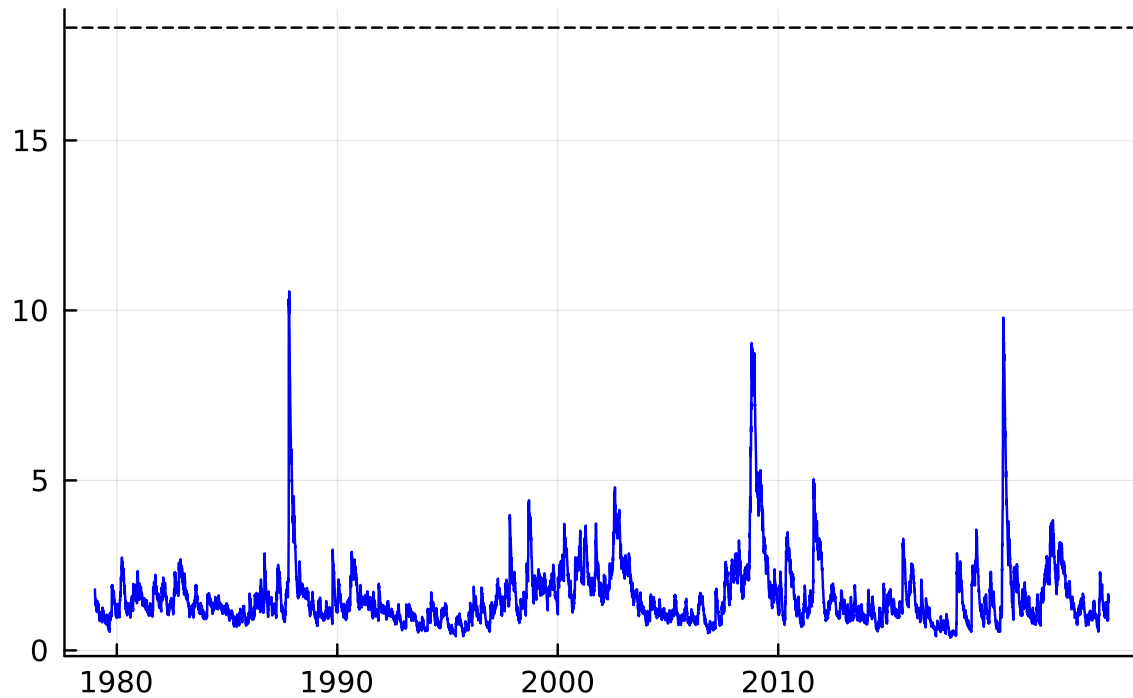
```

```

p1 = plot( dN,VaRd,
            linecolor = :blue,
            #ylim = (-1,35),
            legend = false,
            xticks = (xTicksLoc,xTicksLab),
            title = "Dynamic VaR 95%",
            ylabel= "",
            annotation = (Date(1980),32,text("over the last 100 days",8,:left)) )
hline!([VaR],linecolor=:black,line=(:dash,1))
display(p1)

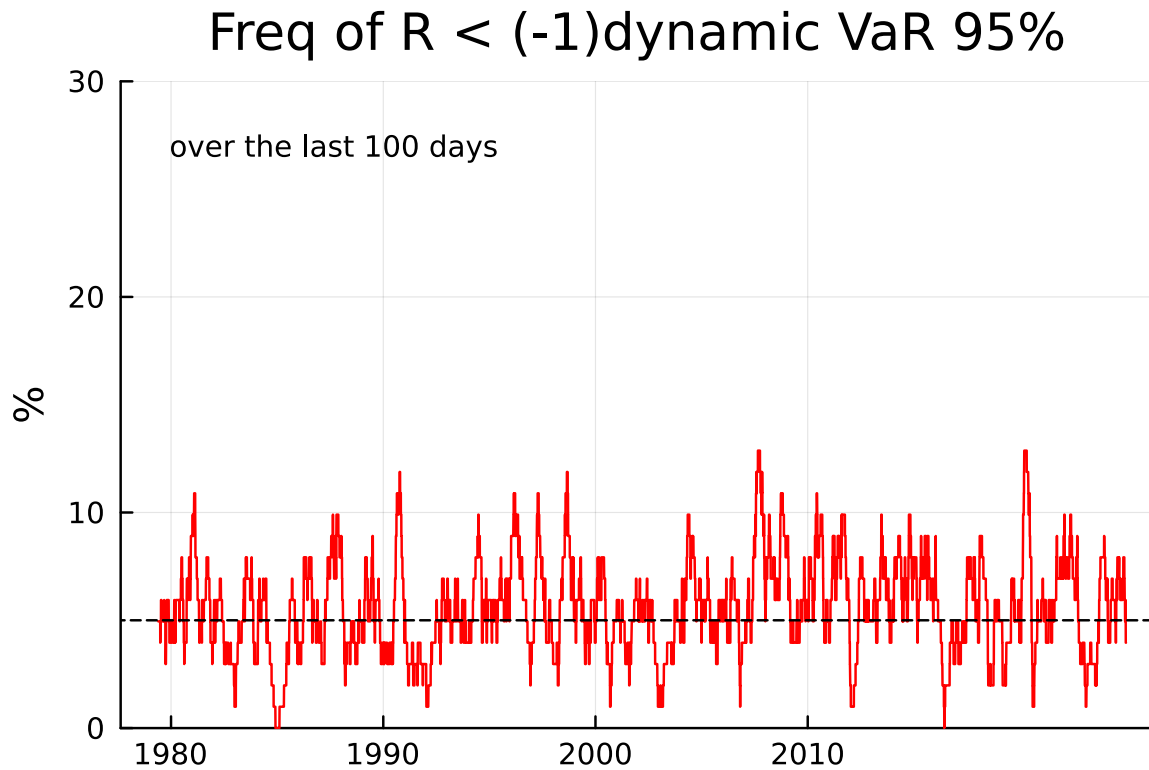
```

Dynamic VaR 95%



```
BreakFreqd = fill(NaN,T)      #backtesting, freq(loss>VaR) on moving data window
for t in 101:T
    BreakFreqd[t] = mean(R[t-100:t] .< -VaRd[t-100:t])
end
```

```
p1 = plot( dN,BreakFreqd*100,
            linecolor = :red,
            ylim = (0,30),
            legend = false,
            xticks = (xTicksLoc,xTicksLab),
            title = "Freq of R < (-1)dynamic VaR 95%",
            ylabel= "%",
            annotation = (Date(1980),27,text("over the last 100 days",8,:left)) )
hline!([5],linecolor=:black,line=:dash,1))
display(p1)
```



Expected Shortfall

The Expected Shortfall (ES) is the average loss, conditional on exceeding the VaR level. In terms of the returns (not losses) this is

$$ES_{\alpha} = -E(R|R \leq -\text{VaR}_{\alpha})$$

For a normally distributed return $R \sim N(\mu, \sigma^2)$ we have

$$ES_{95\%} = -\left(\mu - \frac{\phi(-1.64)}{0.05}\sigma\right),$$

where $\phi()$ is the pdf of a $N(0, 1)$ variable. For other confidence levels, change -1.64 and 0.05.

Alternatively, we can calculate the ES as `-mean(truncated(Normal( $\mu, \sigma$ );upper=q))` where q is the quantile of the return distribution.

```
"""
```

```
    ESNormal( $\mu, \sigma, \alpha$ )
```

```
Calculate ES at a confidence level  $\alpha$  (eg. 0.95) from a N() distribution with
```

mean μ and standard deviation σ .

Notice that this can also be calculated as

```
```  
q = quantile(Normal(μ , σ),1- α)
ES = -mean(truncated(Normal(μ , σ);upper=q))
```  
  
"""  
function ESNormal( $\mu$ , $\sigma$ , $\alpha$ )  
    c  = quantile(Normal(0,1),1- $\alpha$ )  
     $\phi$  = pdf(Normal(0,1),c)  
    ES = -( $\mu$  -  $\phi$ * $\sigma$ /(1- $\alpha$ ))  
    return ES  
end
```

ESNormal

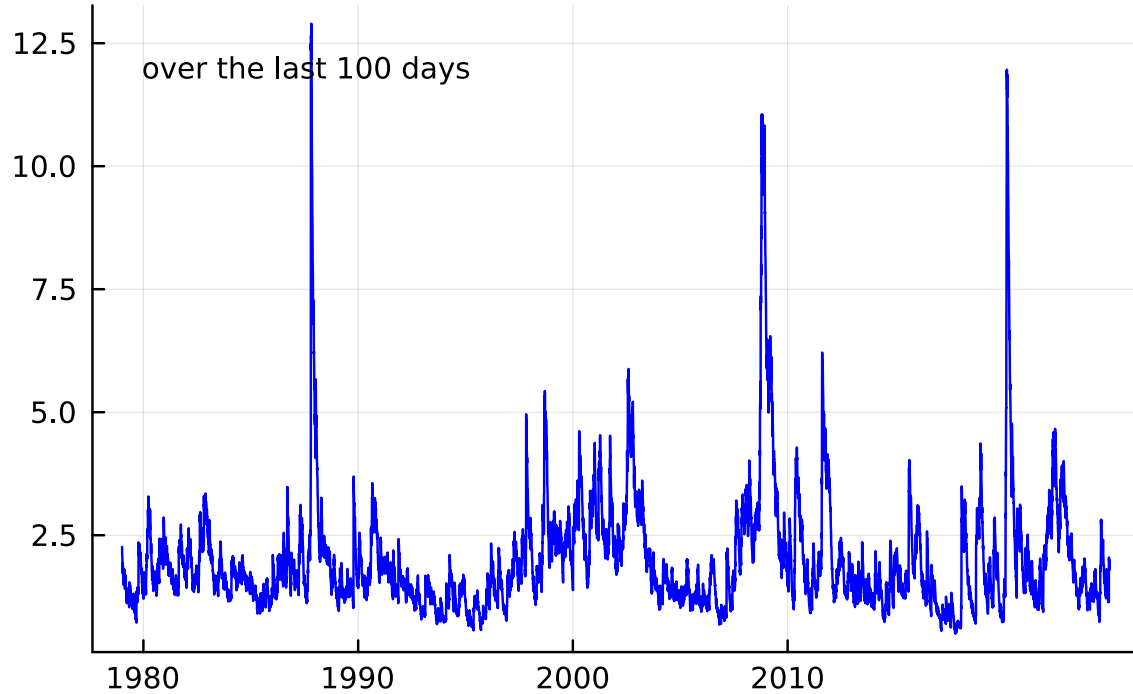
```
( $\mu$ b, $\sigma$ b) = (8,16)  
ES = ESNormal( $\mu$ b, $\sigma$ b,0.95)  
  
printblue("ES 95% with  $\mu$ =$ $\mu$ b and  $\sigma$ =$ $\sigma$ b is:\n")  
printlnPs(ES)  
  
ESd = ESNormal.( $\mu$ ,sqrt.( $\sigma^2$ ),0.95);    #for each date
```

ES 95% with $\mu=8$ and $\sigma=16$ is:

25.003

```
p1 = plot( dN,ESd,  
           linecolor = :blue,  
           legend = false,  
           xticks = (xTicksLoc,xTicksLab),  
           title = "Dynamic ES 95%",  
           ylabel= "",  
           annotation = (Date(1980),12,text("over the last 100 days",8,:left)) )  
display(p1)
```

Dynamic ES 95%



Backtesting ES

by comparing the empirical average return (conditional on loss > VaR) with the predictions from the model with $N(\mu_t, \sigma_t^2)$.

Notice that the interactive dummy δ means that we are effectively calculating results only for those observations when loss > VaR.

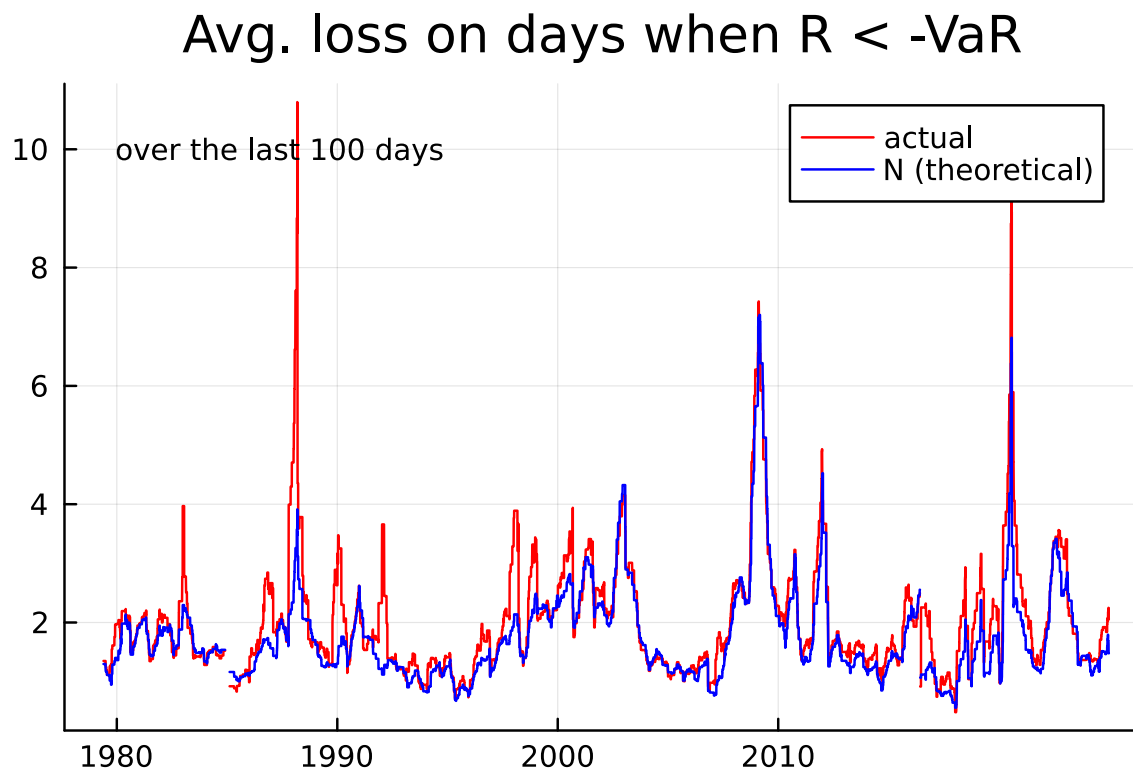
```
ES_backtest = fill(NaN,T,2)          #backtesting ES
for t in 101:T
    δ = R[t-100:t] .< -VaRd[t-100:t]    #dummy: 1 if R < -VaR
    ES_backtest[t,1] = -sum(δ.*R[t-100:t])/sum(δ)    #mean empirical loss|R<-VaR
    ES_backtest[t,2] = sum(δ.*ESd[t-100:t])/sum(δ)    #mean N()-theoretical loss|loss>VaR
end
```

```
p1 = plot( dN,ES_backtest,
            linecolor = [:red :blue],
            label = ["actual" "N (theoretical)"],
            xticks = (xTicksLoc,xTicksLab),
```

```

title = "Avg. loss on days when R < -VaR",
ylabel= "",
annotation = (Date(1980),10,text("over the last 100 days",8,:left)) )
display(p1)

```



Target Semi-Variance

The target semi-variance is the average value of $\delta \cdot (x - h)^2$ where $\delta[t] = 1$ if $x[t] \leq h$ and zero otherwise. The “Sortino” ratio is the ratio of the average $x - h$ divided by the square root of the target semi-variance. It is an alternative to the Sharpe ratio.

```

"""
    TargetSemiVar(x,h)

Calculate target semi-variance
"""
function TargetSemiVar(x,h)

```



```

     $\delta$  = x .<= h
    TSV = mean(  $\delta$ .*(x.-h).^2 )    #a function for target semi-variance
    return TSV
end

```

TargetSemiVar

```

TSV = TargetSemiVar(R,0)

xut = [ $\sigma_{emp}/2$ ,sqrt(TSV)]
printmat(xut;rowNames=[" $\sigma/\sqrt{2}$ ", "target semi-std"])

```

$\sigma/\sqrt{2}$	0.558
target semi-std	0.795

Maximum Drawdown

is the largest loss that could have been made in a sample, buy buying at the top and selling at the bottom.

```

"""
    HistMax(x)

Calculate historial max of a data series, up to and including `x[t]`
"""
function HistMax(x)
    T = length(x)
    xMax = fill(x[1],T)
    for t in 2:T
        xMax[t] = max(xMax[t-1],x[t])
    end
    return xMax
end

"""
    Drawdown(x)

Calculate the (relative) drawdown in a sample, compared to an earlier max level

```

```

"""
function Drawdown(x)
    x_HistMax = HistMax(x)
    D          = (x_HistMax - x)./x_HistMax      #relative drawdown
    return D, x_HistMax
end;

(D,P_max) = Drawdown(P)                        #calculate the relative drawdown

vv1 = argmax(D)                                #index of day with worst drawdown
vv2 = findfirst(==(P_max[vv1]),P_max)          #index of day when the max was reached

printblue("Maximum relative drawdown: $(round(Int,D[vv1]*100))% from $(dN[vv2]) to $(dN[vv1]) \n")

```

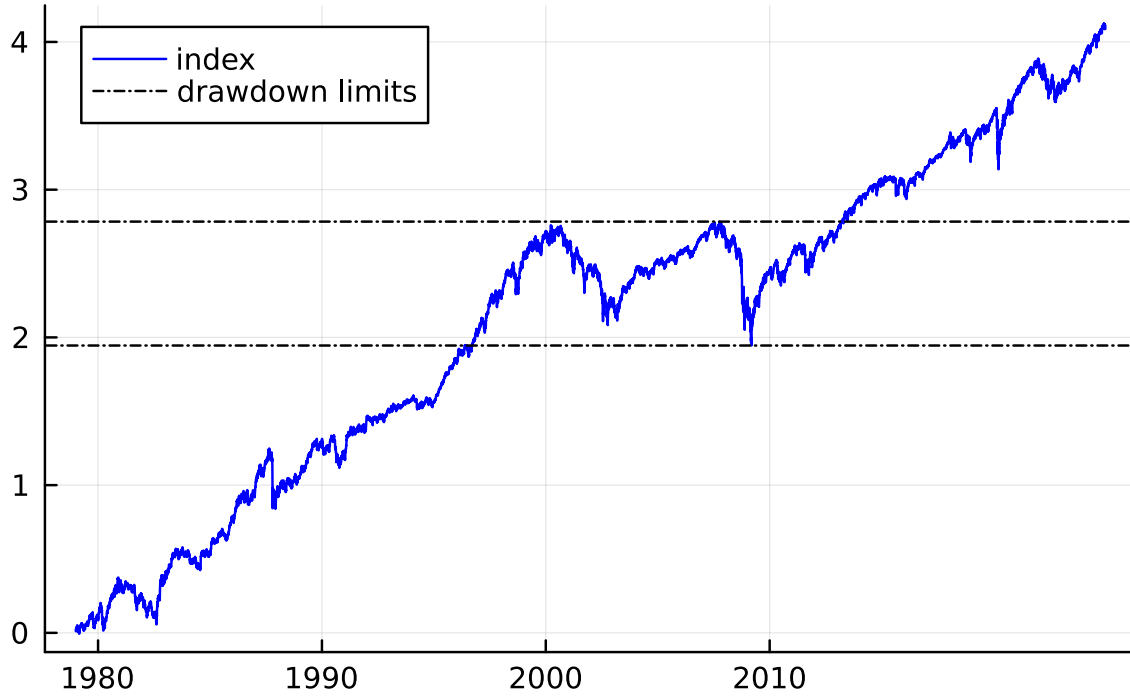
Maximum relative drawdown: 57% from 2007-10-09 to 2009-03-09

```

p1 = plot( dN,log.(P),
            linecolor = :blue,
            #legend = false,
            label = "index",
            xticks = (xTicksLoc,xTicksLab),
            title = "S&P 500, log return index",
            ylabel= "" )
hline!(log.(P[[vv1,vv2]]),linecolor=:black,line=:dashdot,label="drawdown limits")
display(p1)

```

S&P 500, log return index



Utility Theory

This notebook summarizes basic utility theory and how it can be used in portfolio optimisation.

Load Packages and Extra Functions

[Optim.jl](#) is an optimization package.

```
using Printf, Optim  
  
include("src/printmat.jl");
```

```
using Plots  
default(size = (480,320),fmt = :png)
```

Utility Functions

Several of the examples will use the CRRA utility function, which is $U(x) = \frac{x^{1-\gamma}}{1-\gamma}$, where γ is the relative risk aversion.

A Remark on the Code

1. A Julia function can be created in several ways. For a simple one-liner, it is enough to do like this: `MySum(x,y) = x + y`
2. If the function $U(x,\gamma)$ is written for a scalar x value, then we can calculate its value at each element of an array or range `x_range` by using `U.(x_range,γ)`.

```

"""
    U(x,γ)

CRRA utility function, γ is the risk aversion.
"""
U(x,γ) = x^(1-γ)/(1-γ)

"""
    U_1(u,γ)

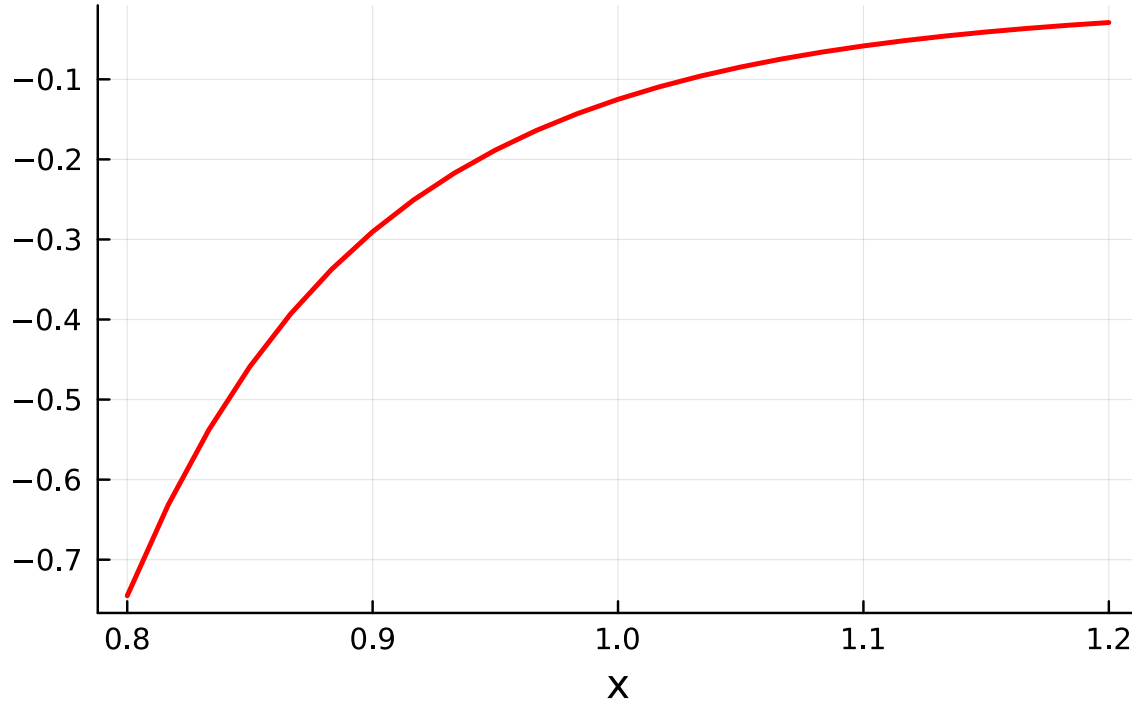
Inverse of CRRA utility function. Solves for x st. U(x,γ) = u, where u is given.
"""
U_1(u,γ) = (u*(1-γ))^(1/(1-γ));

x_range = range(0.8,1.2,length=25)          #possible outcomes
γ = 9

p1 = plot( x_range,U.(x_range,γ),           #notice the dot(.)
           linecolor = :red,
           linewidth = 2,
           legend = false,
           title = "Utility function, CRRA, γ = $γ",
           xlabel = "x" )
display(p1)

```

Utility function, CRRA, $\gamma = 9$



Expected Utility

Recall: if π_s is the probability of outcome (“state”) x_s and there are S possible outcomes, then the expected value is

$$Ex = \sum_{s=1}^S \pi_s x_s.$$

Similarly, the expected utility is

$$EU(x) = \sum_{s=1}^S \pi_s U(x_s)$$

A Remark on the Code

When x is a vector, then $U.(x, \gamma)$ is a vector of the corresponding utility values. Therefore, $E(\pi, U.(x, \gamma))$ sums $\pi[i] * U(x[i], \gamma)$ across all i .

```

"""
    E( $\pi, z$ )

Calculate the expected value from vector of outcomes `z` and
a vector of their probabilities `pi`.
"""
E( $\pi, z$ ) = sum( $\pi.*z$ );          #alternatively, dot( $\pi, z$ ) or  $\pi'z$ 

gamma = 9                      #risk aversion

(x1,x2) = (0.85,1.15)          #possible outcomes
(pi1,pi2) = (0.5,0.5)          #probabilities of outcomes

state1 = [x1,U(x1,gamma),pi1]  #for printing
state2 = [x2,U(x2,gamma),pi2]

printblue("Different states: wealth, utility and probability:\n")
printmat([state1 state2];colNames=["state 1","state 2"],
          rowNames=["wealth","utility","probability"])

Ex = E([pi1,pi2],[x1,x2])      #expected wealth
EU = E([pi1,pi2],U.([x1,x2],gamma)) #expected utility

printmat([Ex,EU];rowNames=["Expected wealth","Expected utility"])

```

Different states: wealth, utility and probability:

	state 1	state 2
wealth	0.850	1.150
utility	-0.459	-0.041
probability	0.500	0.500

Expected wealth	1.000
Expected utility	-0.250

```

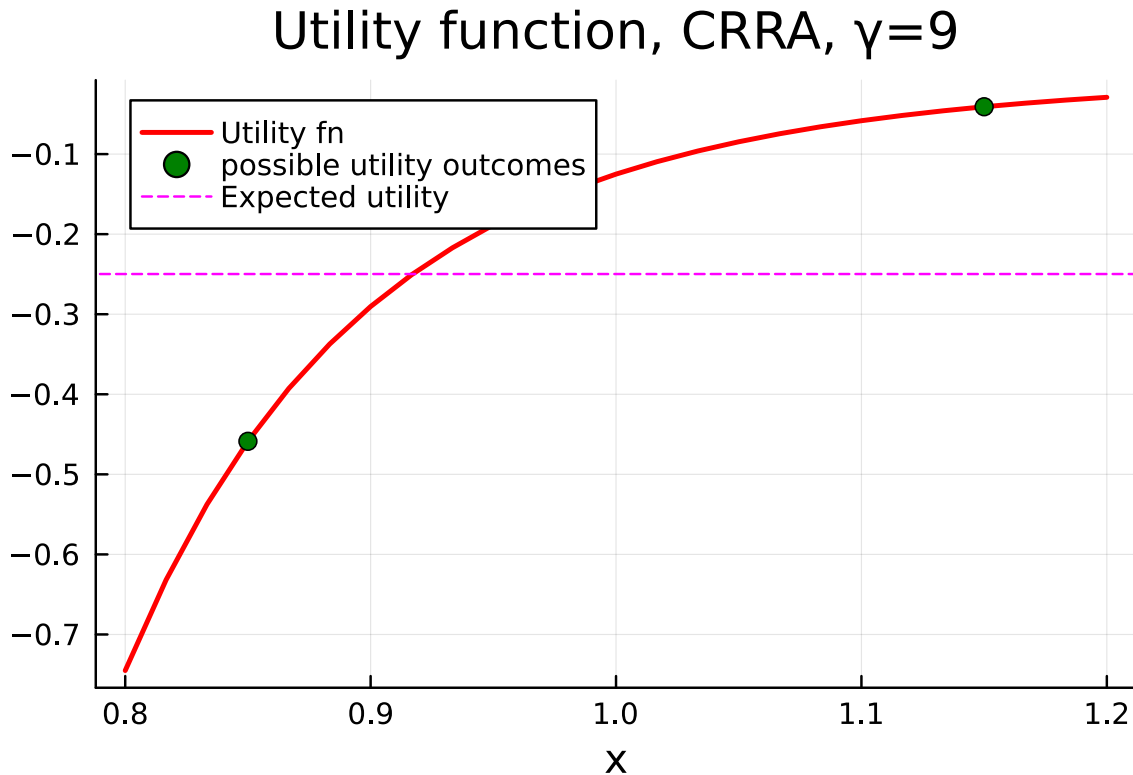
p1 = plot( x_range,U.(x_range,gamma),
            linecolor = :red,
            linewidth = 2,
            label = "Utility fn",
            #ylim = (-0.8,0),
            legend = :topleft,

```

```

title = "Utility function, CRRA,  $\gamma=\gamma$ ",
xlabel = "x" )
scatter!([x1,x2],U.([x1,x2], $\gamma$ ),markercolor=:green,label="possible utility outcomes")
hline!([EU],linecolor=:magenta,line=( :dash,1),label="Expected utility")
display(p1)

```



Certainty Equivalent

The certainty equivalent (here denoted P) is the sure value that solves

$$U(P) = EU(x),$$

where the right hand side is the expected utility from the random x . Thus, P is the highest price the investor is willing to pay for “asset” x .

The code below solves for P by inverting the utility function, first for the same risk aversion (γ) as above, and later for different values of γ .

We can think of $E(x)/P - 1$ as the expected return on x that the investor requires in order to buy the asset. It is increasing in the risk aversion γ .

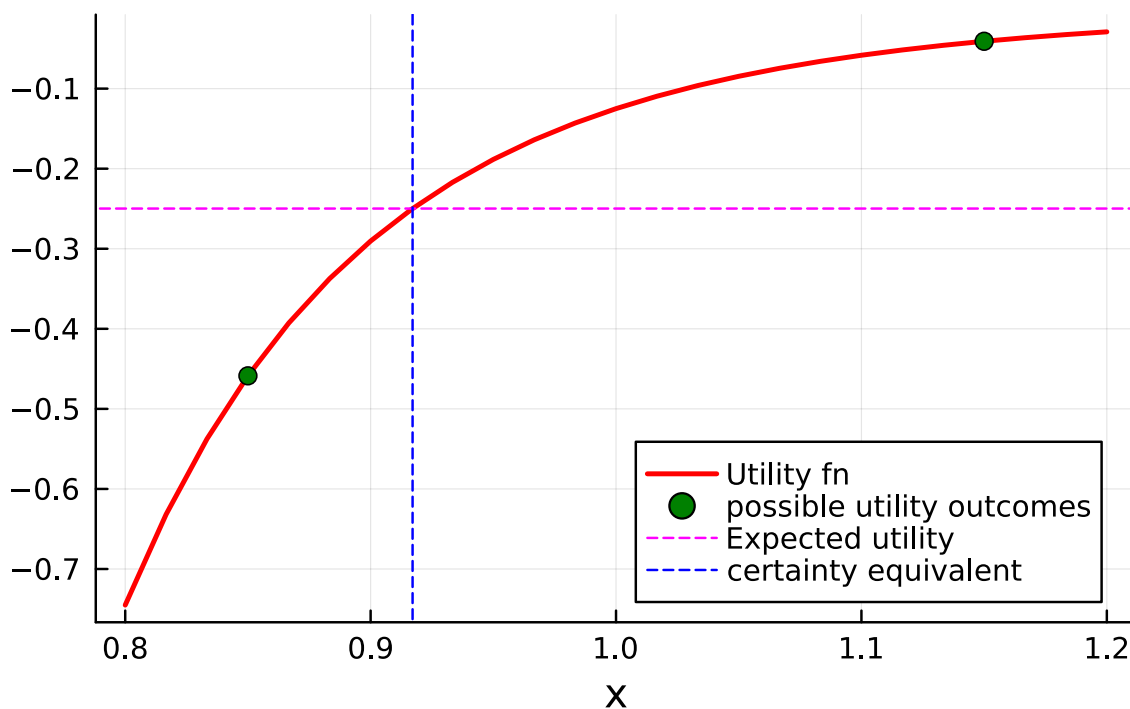

```

EU_i = E([ $\pi_1, \pi_2$ ], U.([ $x_1, x_2$ ],  $\gamma$ )) #expected utility
P     = U_1(EU_i,  $\gamma$ )                    #certainty equivalent (inverting the utility fn)

p1 = plot( x_range, U.(x_range,  $\gamma$ ),
           linecolor = :red,
           linewidth = 2,
           label = "Utility fn",
           #ylim = (-1.3, -0.7),
           legend = :bottomright,
           title = "Utility function, CRRA,  $\gamma = \$\gamma$ ",
           xlabel = "x" )
scatter!([ $x_1, x_2$ ], U.([ $x_1, x_2$ ],  $\gamma$ ), markercolor=:green, label="possible utility outcomes")
hline!([EU], linecolor=:magenta, line=(:dash, 1), label="Expected utility")
vline!([P], linecolor=:blue, line=(:dash, 1), label="certainty equivalent")
display(p1)

```

Utility function, CRRA, $\gamma=9$



```

 $\gamma$ M = [0, 2, 5, 10, 25, 50, 100]          #different risk aversions
L     = length( $\gamma$ M)

```

```

(P,ERx) = (fill(NaN,L),fill(NaN,L))
for i in 1:L                                     #loop over γ values
    #local EU_i                                  #local/global is needed in script
    EU_i = E([π1,π2],U.([x1,x2],γM[i])) #expected utility with γM[i]
    P[i] = U_1(EU_i,γM[i])                     #inverting the utility fn
    ERx[i] = Ex/P[i] - 1                        #(required) expected net return
end

printblue("risk aversion and certainly equivalent (recall: E(wealth) = $Ex):\n")
printmat(γM,P,ERx;colNames= ["γ","certainty eq","expected return"],width=20)

```

risk aversion and certainly equivalent (recall: $E(\text{wealth}) = 1.0$):

γ	certainty eq	expected return
0	1.000	0.000
2	0.977	0.023
5	0.947	0.056
10	0.912	0.097
25	0.875	0.143
50	0.862	0.160
100	0.856	0.168

Utility-Based Portfolio Choice with One Risky Asset

In the example below, the investor maximizes $E \ln(1 + R_p)$, with $R_p = vR^e + R_f$ by choosing v (the weight on the only risky asset). For simplicity, there are only two possible outcomes for R^e with equal probabilities.

This particular problem can be solved by pen and paper, but this becomes difficult when the number of states increases - and even worse when there are many assets. To prepare for these trickier cases, we apply a numerical optimization algorithm already to this simple problem.

Remark on the Code

To solve the optimization problem we use `optimize()` from the [Optim.jl](#) package. The key steps are:

1. Define a function for expected utility, $EU_{\log}(v, \pi, R^e, R_f)$. The value depends on the portfolio choice v , as well as the properties of the asset (probabilities and returns for different states, the vectors π and R^e) and the riskfree return (R_f).

2. To create data for the plot, we loop over $v[i]$ values and calculate expected utility as $EU\log(v[i], \pi, R^e, R_f)$. (Warning: you can assign a value to π provided you have not used the built-in constant π (3.14156...) first.)
3. For the optimization, we minimize the anonymous function $v \rightarrow -EU\log(v, \pi, R^e, R_f)$. This is a function of v only and we multiply by -1 since `optimize()` is a *minimization* routine.

```

"""
    EUlog(v,π,Re,Rf)

Calculate expected utility (log(1+Rp)) from investing into one risky and one riskfree asset

v: scalar
π: S vector probabilities of the different states
Re: S vector, excess returns of the risky asset in different states
Rf: scalar, riskfree rate

"""
function EUlog(v,π,Re,Rf)          #expected utility, utility fn is logarithmic
    Rp = v*Re .+ Rf                #portfolio return in each state, vector
    eu = E(π,log.(1 .+ Rp))         #expected utility
    return eu
end

```

EUlog

```

π      = [0.5,0.5]                  #probabilities for different states
Re     = [-0.10,0.10]               #excess returns in different states
Re_b   = [-0.10,0.11]
Rf    = 0                          #riskfree rate

v_range = range(-1,1.5,length=101)  #try different weights on risky asset

L        = length(v_range)
EUv      = fill(NaN,L,2)
for i in 1:L                          #loop over v values
    EUv[i,1] = EUlog(v_range[i],π,Re,Rf)
    EUv[i,2] = EUlog(v_range[i],π,Re_b,Rf)
end

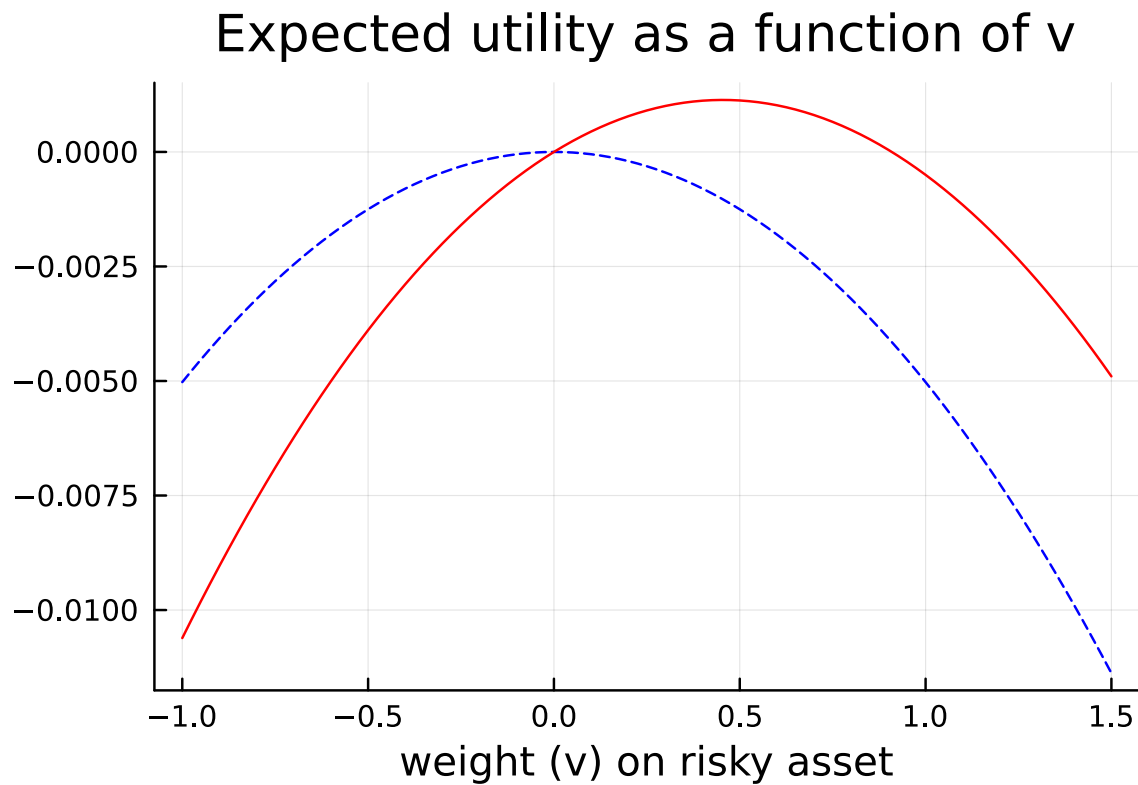
p1 = plot( v_range,EUv,
            linecolor = [:blue :red],

```

```

linestyle = [:dash :solid],
linewidth = 1,
legend = false,
title = "Expected utility as a function of v",
xlabel = "weight (v) on risky asset" )
display(p1)

```



```

Sol = optimize(v→-EUlog(v,π,Re,Rf),-1,1)      #minimize -EUlog
printlnPs("Optimum at: ",Optim.minimizer(Sol))

Sol = optimize(v→-EUlog(v,π,Re_b,Rf),-1,1)      #minimize -EUlog
printlnPs("Optimum at: ",Optim.minimizer(Sol))

printred("\nCompare with the figure")

```

Optimum at: -0.000
Optimum at: 0.455

Compare with the figure

Portfolio Choice with Several Risky Assets

This optimization problem has several risky assets and states and a CRRA utility function. Numerical optimization is still straightforward.

A Remark on the Code

- The code below is fairly similar to the log utility case solved before, but extended to handle CRRA utility and several assets and states.
- With several choice variables, the call to `optimize()` requires a vector of starting guesses as input.

```
"""
    EUcrra(v,π,Re,Rf,γ)

Calculate expected utility from investing into n risky assets and one riskfree asset

v:  n vector (weights on the n risky assets)
π:  S vector (S possible "states")
Re: nxS matrix, each column is the n vector of excess returns in one of the states
Rf: scalar, riskfree rate
γ:  scalar, risk aversion
"""
function EUcrra(v,π,Re,Rf,γ)
    S = length(π)
    Rp = Re'v .+ Rf          #portfolio return in each state, vector
    eu = E(π,U.(1 .+ Rp,γ)) #expected utility when using portfolio v
    return eu
end
```

EUcrra

```
R = [-0.03 0.08 0.20;      #2 assets, 3 states
      -0.04 0.22 0.15]    #R[i,j] is the return of asset i in state j
π = [1/3,1/3,1/3]         #probs of the states
Rf = 0.065
Re = R .- Rf              #excess returns in different states
γ = 5

Sol = optimize(v→-EUcrra(v,π,Re,Rf,γ),[-0.6,1.2],BFGS()) #minimize -EUcrra, use BFGS algorithm
v = Optim.minimizer(Sol)    #extract the solution
```

```
printblue("optimal portfolio weights from max EUcrra():\n")
printmat([v;1-sum(v)];rowNames=["asset 1","asset 2","riskfree"],prec=2)
```

optimal portfolio weights from max EUcrra():

```
asset 1      -0.73
asset 2       1.32
riskfree     0.41
```

Mean-Variance and the Telser Criterion

Let μ be a vector of expected returns of the investible assets and Σ be their covariance matrix.

The Telser criterion solves a problem of the following type:

$\max_v ER_p$ subject to $VaR_{95\%} < V^*$, where $ER_p = v' \mu + (1 - 1'v)R_f$.

If the returns are normally distributed then

$VaR_{95\%} = -[ER_p - 1.64Std(R_p)]$,

It follows that the VaR restriction can be written

$ER_p > -V^* + 1.64Std(R_p)$.

The figure below illustrates that the optimal portfolio is *on the CML* when the returns are normally distributed.

```
include("src/MvCalculations.jl");    #functions for traditional MV frontiers
```

```
V* = 0.04

μ = [11.5, 9.5, 6]/100           #expected returns
Σ = [166 34 58;                  #covariance matrix
      34 64 4;
      58 4 100]/100^2
Rf = 0.03

L = 101
μ*_range = range(Rf,0.15,length=L)    #required average returns

(σMVF,σCML) = (fill(NaN,L),fill(NaN,L))
for i in 1:L                           #loop over different required returns (μ*)
```

```

    σMVF[i] = MVCalc(μx_range[i],μ,Σ)[1]          #std of MVF (risky only) at μx
    σCML[i] = MVCalcRf(μx_range[i],μ,Σ,Rf)[1]      #std of MVF (risky&riskfree) at μx
end

VaRRestr = -Vx .+ 1.64*σCML;    #the portfolio mean return must be above this

(μT,μT,σT) = MVTangencyP(μ,Σ,Rf)          #tangency portfolio
SRT = (μT-Rf)/σT                          #Sharpe ratio

w = -(Rf+Vx)/(μT-Rf-1.64*σT)
(μopt,σopt) = (w*μT+(1-w)*Rf,abs(w)*σT)
printlnPs("Optimal weight on tangency portfolio: ",w)

```

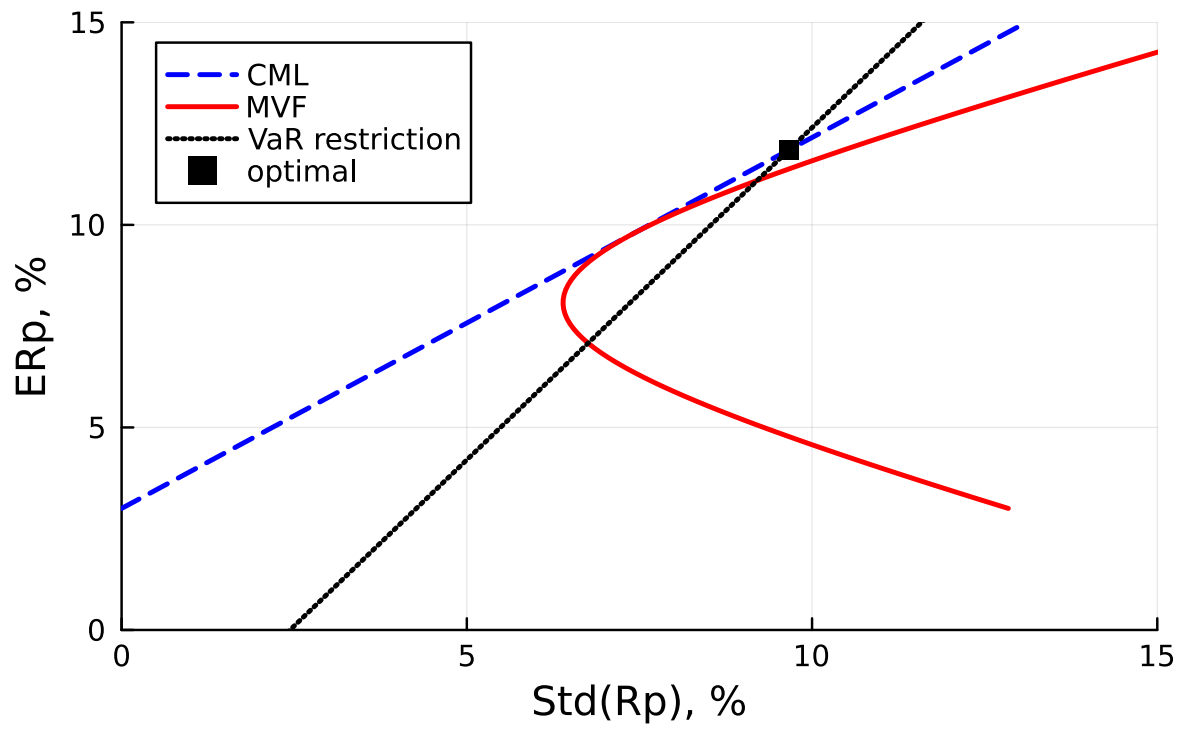
Optimal weight on tangency portfolio: 1.311

```

p1 = plot( [σCML σMVF σCML]*100,[μx_range μx_range VaRRestr]*100,
    linestyle = [:dash :solid :dot],
    linecolor = [:blue :red :black],
    linewidth = 2,
    label = ["CML" "MVF" "VaR restriction"],
    xlim = (0,15),
    ylim = (0,15),
    legend = :topleft,
    title = "Mean vs std",
    xlabel = "Std(Rp), %",
    ylabel = "ERp, %" )
    scatter!([μopt*100],[μopt*100],markercolor = :black,markershape = :square,label="optimal")
display(p1)

```

Mean vs std



Multi-Factor Models

The first part of this notebook summarizes a simple economic model that leads to a multi-factor representation of the expected returns. The second part tests a multi-factor model by OLS.

Load Packages and Extra Functions

```
using Printf, DelimitedFiles, Statistics, LinearAlgebra

include("src/printmat.jl")
include("src/OlsGMNW.jl");           #function for doing OLS
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Portfolio Choice with Background Risk

The investor maximizes $ER_p - \frac{k}{2}\text{Var}(R_p)$, where

$$R_p = (1 - \phi)R_{Fin} + \phi R_c \text{ with } R_{Fin} = w'R^e + R_f.$$

In this definition of the portfolio return, ϕ is the fraction of total wealth that is bound to non-traded “assets”: this is the background risk.

Combining the two equations and taking expectations gives $ER_p = v'\mu^e + \phi\mu_c^e + R_f$, where $v = w(1 - \phi)$ and where μ^e is the vector of expected excess returns on the risky assets and μ_c^e is the expected excess return on the background risk. Also, $\text{Var}(R_p) = v'\Sigma v + \phi^2\sigma_{cc} + 2\phi v'S_c$, where S_c is a vector of covariances of the risky assets with the background risk, and σ_{cc} is the variance of the background risk.

The optimal portfolio choice (weight on the risky assets within the financial portfolio) is

$$w = \frac{1}{k} \left(\frac{v'\Sigma v + \phi^2\sigma_{cc} + 2\phi v'S_c}{v'\Sigma v} \right) (1 - \phi).$$

The next cell defines functions for expected utility and the optimal portfolio choice.

A Remark on the Code

The code uses the subscript h (as in S_h etc) instead of c . The reason is that the Julia unicode symbols do not include a subscripted c .

```

"""
    EU

Expected utility for the case with background risk.

The notation follows the text above, except that the subscript h (as in  $S_h$ ) for the non-traded asset.
"""
function EU(w,  $\phi$ , k,  $\mu^e$ ,  $\Sigma$ ,  $S_h$ ,  $\sigma_{hh}$ ,  $\mu^e_h$ ,  $R_f$ )
    ER_fin = w'  $\mu^e$  +  $R_f$ 
    ERp    = (1- $\phi$ )*ER_fin +  $\phi$ *( $\mu^e_h$ + $R_f$ )
    v      = (1- $\phi$ )*w
    VarRp  = v'  $\Sigma$  v +  $\phi^2$ * $\sigma_{hh}$  + 2* $\phi$ *v'  $S_h$ 
    EUtil  = ERp - k/2*VarRp
    return EUtil
end

"""
    PortFOpt

Solve for the optimal weight on risky asset (w) in the financial portfolio.

The notation follows the text above.
"""
function PortFOpt( $\phi$ , k,  $\mu^e$ ,  $\Sigma$ ,  $S_h$ )
    wopt = inv( $\Sigma$ )*( $\mu^e$ /k- $\phi$ * $S_h$ )/(1- $\phi$ )
    return wopt
end

```

PortFOpt

One Risky Asset

We first consider the case when there is only one risky asset (w is a scalar). We will illustrate several cases (A, B, C) which differ with respect to the fraction of non-traded asset (denoted ϕ) and its

covariance with the risk assets (denoted S_c as in the lecture notes or S_h as in the code).

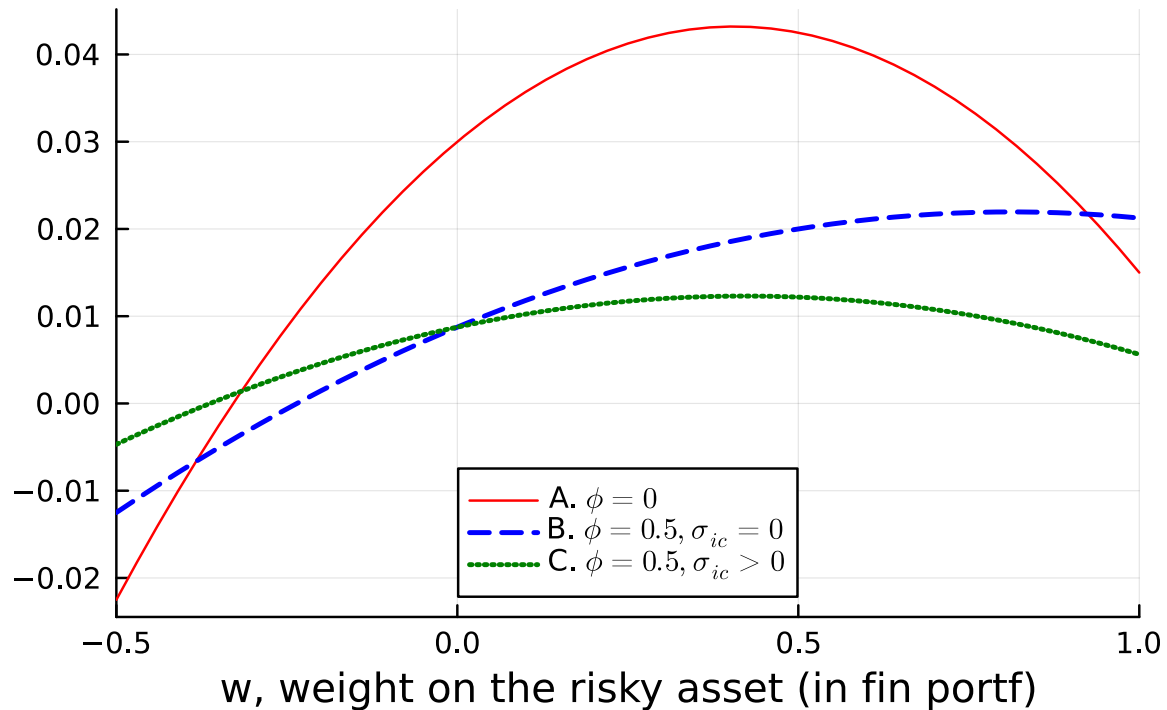
```
Σ      = 0.08^2      #variance-covariance (matrix) of risky assets, here only one
(μe,k,σhh,μeh,Rf) = (0.065,25,0.1^2,0.02,0.03)
Sh-A = 0           #covariance(s) of risky with background, case A (and B)
Sh-C = 0.3*sqrt(σhh*σhh)      #case C

L      = 51
w_range = range(-0.5,1,length=L) #weight on risky asset in financial portf

(EU_case_A,EU_case_B,EU_case_C) = (fill(NaN,L),fill(NaN,L),fill(NaN,L))
for i in 1:L                      #expected utility at different v values
    EU_case_A[i] = EU(w_range[i],0,k,μe,Σ,Sh-A,σhh,μeh,Rf)      #φ=0
    EU_case_B[i] = EU(w_range[i],0.5,k,μe,Σ,Sh-A,σhh,μeh,Rf)    #φ>0, Sh=0
    EU_case_C[i] = EU(w_range[i],0.5,k,μe,Σ,Sh-C,σhh,μeh,Rf)    #φ>0, Sh>0
end
```

```
p1 = plot( w_range,[EU_case_A EU_case_B EU_case_C],
           linecolor = [:red :blue :green],
           linestyle = [:solid :dash :dot],
           linewidth = [1 2 2],
           label = [L"A. $φ=0$ L"B. $φ=0.5, \sigma_{ic}=0$ L"C. $φ=0.5, \sigma_{ic}>0$"],
           xlims = (-0.5,1),
           legend = :bottom,
           title = "Expected utility",
           xlabel = "w, weight on the risky asset (in fin portf)" )
display(p1)
```

Expected utility



```
w_A = PortFOpt(0,k,μe,Σ,Sh-A) #optimal portfolio choice
w_B = PortFOpt(0.5,k,μe,Σ,Sh-A)
w_C = PortFOpt(0.5,k,μe,Σ,Sh-C)

printblue("\nOptimal weight on (a single) risky asset in three cases (φ ≥ 0):\n")
xx = [w_A;w_B;w_C]
colNames = ["in financial portf"]
rowNames = ["A. φ=0","B. φ=0.5, Sh=0","C. φ=0.5, Sh>0"]
printmat(xx;colNames,rowNames,width=20)

printred("\nCompare with the plot")
```

Optimal weight on (a single) risky asset in three cases ($\phi \geq 0$):

	in financial portf
A. $\phi=0$	0.406
B. $\phi=0.5, S_h=0$	0.812
C. $\phi=0.5, S_h>0$	0.422

Compare with the plot

Several Risky Assets

We now consider several risky assets. Notice that S_c (or S_h) is now a vector of covariances of each of the investable risky assets with the non-traded asset.

A Remark on the Code

- The S_h (S_c) vector is here created in such a way that we prespecify the correlations (and then scale with the product of the standard deviations of the non-traded assets and the risky assets).

```
"""
    TheoryParams()

Parameters for the
"""
function TheoryParams()
    μ = [11.5, 9.5, 6]/100           #expected returns
    Σ = [166 34 58;                 #covariance matrix
         34 64 4;
         58 4 100]/100^2
    Rf = 0.03

    φ = 0.3                        #fraction of non-traded asset
    σhh = 0.25^2
    Sh = [0.5,0.9,-0.1].*sqrt(σhh).*sqrt.(diag(Σ)) #a vector of covariances
    μh = 0.1
    k = 8
    return μ,Σ,Rf,φ,σhh,Sh,μh,k
end
```

TheoryParams

```
(μ,Σ,Rf,φ,σhh,Sh,μh,k) = TheoryParams()           #parameters

w = PortFOpt(φ,k,μ.-Rf,Σ,Sh)
```

```
printblue("optimal weights (inside the financial subportfolio):\n")
printmat([w;1-sum(w)];rowNames=["A","B","C","Rf"],prec=2)
```

optimal weights (inside the financial subportfolio):

```
A      0.24
B      0.45
C      0.49
Rf     -0.18
```

Asset Pricing Implications of a Multifactor Model

If several factors affect the portfolio choice, then they will (in equilibrium) also affect prices and returns. In fact, the expected excess return on asset i is

$$ER_i^e = \beta_i' \mu_F^e,$$

where β_i is the vector of regressions coefficients from regressing R_i^e on the factors and μ_F^e is the vector of expected excess returns on those factors.

The example below assumes there are two factors and we only consider the pricing of a single investable asset. The numbers used have nothing in particular to do with those used in the previous examples. (That link could be done, but requires some intermediate steps that we here skip.)

```
μ^eF = [5.22,19.98]           #expected excess returns of factors

β = [0.80,0.15]              #coeffs in ER^e = β'Factors
printlnPs("The multiple regression coefficients and factor risk premia: ")
printmat(β,μ^eF;colNames=["β","μ^e (factors), %"],width=18)

μ^e = β'μ^eF
printlnPs("μ^e for portfolio p according to 2-factor model,%:")
printmat(μ^e)
```

The multiple regression coefficients and factor risk premia:

```
      β      μ^e (factors), %
0.800      5.220
0.150     19.980
```

```
μ^e for portfolio p according to 2-factor model,%:
7.173
```

Factor Mimicking Portfolio: Details (extra)

Remark

A vector of regression slopes (y regressed on the vector x) can be calculated as

$$\beta = \Sigma_{xx}^{-1} S_{xy},$$

where Σ_{xx} is the variance-covariance matrix of x and S_{xy} is a vector of covariance of each x with y .

See the text in the pdf for details of these computations.

```
"""
    PortFOpt( $\phi, k, \mu^e, \Sigma, S_h$ )

Solve for the optimal weight on risky asset ( $w$ ) in the financial portfolio.

The notation follows the text above.
"""
function PortFOpt( $\phi, k, \mu^e, \Sigma, S_h$ )
    wopt = inv( $\Sigma$ )*( $\mu^e/k - \phi * S_h$ )/(1- $\phi$ )
    return wopt
end

"""
    Portf_μσ( $w, \mu^e, R_f, \Sigma$ )

Calculate expected return and portfolio standard deviation
"""
function Portf_μσ( $w, \mu^e, R_f, \Sigma$ )
     $\mu^e\_p = w' * \mu^e$ 
     $\mu\_p = w' * \mu^e + R_f$ 
     $\sigma\_p = \text{sqrt}(w' * \Sigma * w)$ 
    return  $\mu^e\_p, \mu\_p, \sigma\_p$ 
end
```

Portf_μσ

```
( $\mu, \Sigma, R_f, \phi, \sigma_{hh}, S_h, \mu_h, k$ ) = TheoryParams()           #parameters
 $\mu^e = \mu - R_f$ 
 $k = 8.691$ 
```

```

wm = PortFOpt(φ,k,μe,Σ,Sh)           #market portfolio
(μem,-,σm) = Portf_μσ(wm,μe,0,Σ)

wk = inv(Σ)*Sh                         #factor mimicking portfolio
(μek,-,σk) = Portf_μσ(wk,μe,0,Σ)     #using k instead of λ as subscript

wp = [0.5,0.4,0.1]                     #portfolio p
μep,- = Portf_μσ(wp,μe,0,Σ)

println("portfolios:\n")
printmat(wm,wk,wp;rowNames=["A","B","C","Rf"],colNames=["m","k (λ)","p"],prec=3)

```

portfolios:

	m	k (λ)	p
A	0.191	0.735	0.500
B	0.333	2.470	0.400
C	0.476	-0.775	0.100

```

σmk = wk'Σ*wm
Ψ = [σm2 σmk;
      σmk σk2]           #vcv of (m,k)

σpm = wp'Σ*wm           #cov(p, and [m,k])
σpk = wp'Σ*wk

β = inv(Ψ)*[σpm,σpk]     #Betas

xx = hcat([μem,μek]*100,Ψ*1002,[σpm,σpk]*1002,β)
printmat(xx;rowNames=["m","k (λ)"],colNames=["μe", "", "vcv(m,k)", "Cov(x,p)", "β"])

printlnPs("Implied and actual μep: ",β'*[μem,μek]*100,μep*100)

```

	μ ^e		vcv(m,k)	Cov(x,p)	β
m	5.217	51.981	78.807	53.230	0.797
k (λ)	19.980	78.807	582.438	150.026	0.150
Implied and actual μ ^e _p :			7.150	7.150	

Empirical Test of a 3-Factor Model: Loading Data

Load Data

```
x = readlm("Data/FFmFactorsPs.csv",',',skipstart=1)
Rme = x[:,2]          #market excess return
RSMB = x[:,3]         #small minus big firms
RHML = x[:,4]         #high minus low book-to-market ratio
Rf = x[:,5]           #interest rate

x = readlm("Data/25_Portfolios_5x5.CSV",',',skipstart=1)
R = x[:,2:end]        #returns for 25 FF portfolios
Re = R .- Rf          #excess returns for the 25 FF portfolios
#Re = Re[:,[1,7,13,19,25]] #use just 5 assets to make the printing easier

(T,n) = size(Re)      #no. obs and no. test assets
```

(660, 25)

OLS Estimation and Testing $\alpha_i = 0$

Recall: estimate (α_i, b_i) in the factor model

$$R_{it}^e = \alpha_i + b_i' f_t + \varepsilon_{it},$$

where f_t is a vector of excess returns of the factors.

Test if $\alpha_i = 0$

```
x = [ones(T) Rme RSMB RHML]      #regressors
K = size(x,2)                    #no. of regressors

(alpha,beta,tstat) = (fill(NaN,n,K),fill(NaN,n))
for i in 1:n                      #loop over the different test assets
    #local b_i,Covb                #local/global is needed in script
    (b_i,_,_,Covb,_) = OlsGM(Re[:,i],x)
    alpha[i,:] = b_i
    tstat[i] = (alpha[i,1]-0)/sqrt(Covb[1,1]) #t-stat for alpha[i]=0?
end

printblue("Regression of Re on constant and 3 factors:\n")
```

```
rowNames = [string("asset ",i) for i=1:n]
printmat([αβ[:,1]*12 tstat];rowNames,colNames=["α*12","t-stat"])
```

Regression of R_e on constant and 3 factors:

	$\alpha*12$	t-stat
asset 1	-6.741	-5.613
asset 2	0.089	0.098
asset 3	-0.356	-0.536
asset 4	1.696	2.735
asset 5	2.173	2.290
asset 6	-2.419	-2.950
asset 7	0.405	0.595
asset 8	0.326	0.462
asset 9	0.778	1.281
asset 10	-0.160	-0.250
asset 11	-1.349	-1.798
asset 12	0.841	1.134
asset 13	-0.476	-0.629
asset 14	0.100	0.140
asset 15	0.199	0.221
asset 16	0.736	0.991
asset 17	-0.459	-0.576
asset 18	-0.547	-0.663
asset 19	0.094	0.113
asset 20	-1.060	-1.078
asset 21	1.716	3.303
asset 22	0.527	0.754
asset 23	-0.195	-0.242
asset 24	-2.921	-3.839
asset 25	-2.107	-1.720

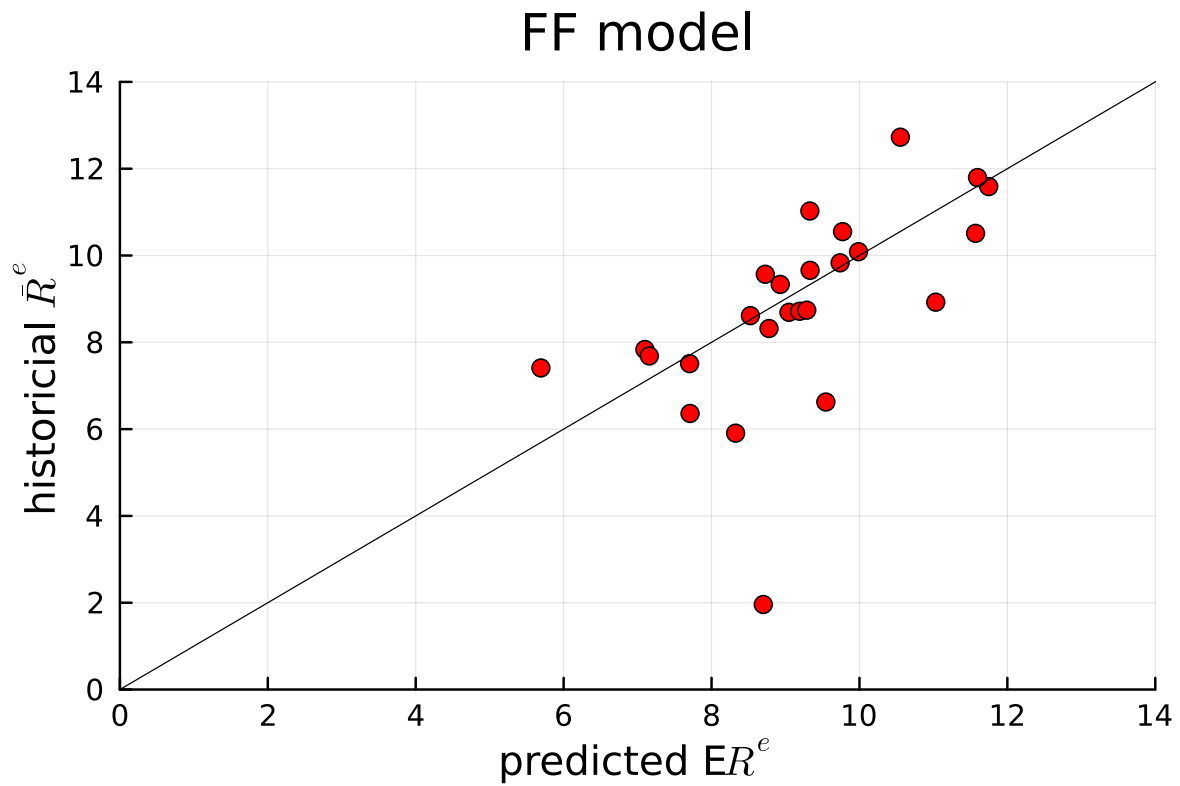
```
R^ePred = αβ[:,2:end]*vec(mean(x[:,2:end],dims=1)) #predicted avg R^e with α=0
R^eAvg = vec(mean(Re,dims=1)) #historical avg R^e

p1 = scatter( R^ePred*12,R^eAvg*12,
              xlim = (0,14),
              ylim = (0,14),
              legend = false,
              markercolor = :red,
              markershape = :circle,
```

```

title = "FF model",
xlabel = L"predicted  $ER^e$ ",
ylabel = L"historical  $\bar{R}^e$ ")
Plots.abline!(1,0,linewidth=0.5, line=:solid)
display(p1)

```



Efficient Markets

This notebook describes and tests the predictability of asset returns (autocorrelations, autoregressions, out-of-sample R², Mariano-Diebold test) and implements a simple trading strategy.

Load Packages and Extra Functions

```
using Printf, Dates, DelimitedFiles, Statistics, LinearAlgebra, StatsBase

include("src/printmat.jl")
include("src/OlsGMNW.jl") #OLS with covariance matrix etc
include("src/OlsM.jl");   #OLS, basic
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data

The data set contains daily data of the equity market return, riskfree rate and the returns of the 25 Fama-French portfolios. All returns are in percent.

```
x = readdlm("Data/MomentumSR.csv",',')
dN = Date.(x[:,1],"yyyy-mm-dd")           #Julia dates
y = convert.(Float64,x[:,2:end])

(Rm,Rf,R25) = (y[:,1],y[:,2],y[:,3:end])
R           = R25[:,end]                  #R are returns of 'large value' firms
                                           #R25 will be used later

println("\nThe first few rows of data")
printmat(dN[1:4],Rm[1:4],Rf[1:4],R[1:4];colNames=["dN","Rm","Rf","R"])
```

```
println("size of dN, Rm, Rf, R")
println(size(dN),"\n",size(Rm),"\n",size(Rf),"\n",size(R))

T = length(R);                                #number of periods
```

The first few rows of data

	dN	Rm	Rf	R
1979-01-02	0.615	0.035	1.420	
1979-01-03	1.155	0.035	1.750	
1979-01-04	0.975	0.035	1.560	
1979-01-05	0.685	0.035	1.430	

```
size of dN, Rm, Rf, R
(9837,)
(9837,)
(9837,)
(9837,)
```

Autocorrelations

The s th autocorrelation is

$$\rho_s = \text{Corr}(R_t, R_{t-s})$$

In large samples, $\sqrt{T} \hat{\rho}_s \sim N(0, 1)$ if the true value is $\rho_s = 0$ for all s .

A Remark on the Code

The [StatsBase.jl](#) package contains methods for estimating autocorrelations (see `autocor()` below).

```
plags = 1:5          #different lags to calculate autocorrelations for
ρ      = autocor(R,plags)

printblue("autocorrelations and their t-stats:\n")
printmat(ρ,sqrt(T)*ρ;colNames=["ρ","t-stat"],rowNames=plags,cell00="lag")
```

autocorrelations and their t-stats:

lag	ρ	t-stat
1	-0.027	-2.697
2	-0.028	-2.747
3	-0.024	-2.343
4	-0.014	-1.350
5	-0.020	-2.015

Autoregressions

An AR(1) is

$$R_t = c + a_1 R_{t-1} + \varepsilon_t.$$

We also consider an asymmetric AR(1)

$$R_t = \alpha + \beta Q_{t-1} R_{t-1} + \gamma (1 - Q_{t-1}) R_{t-1} + \varepsilon_t,$$

where $Q_{t-1} = 1$ if $R_{t-1} < 0$ and zero otherwise.

Both models can be estimated by OLS.

A Remark on the Code

- The function `olsGMFn()` does OLS and reports, among other things, the variance-covariance matrix of the estimates (called `Covb` below).
- To get standard errors, we first extract the variances from the variance-covariance matrix (`diag(Covb)`) and then calculate the square roots (`sqrt().()`). Notice the dot. in the latter calculation: to calculate the square root of each element in the vector of variances.
- The t-stats are then just the point estimates (minus the null hypothesis, which is here 0) divided by the standard errors.

```
x = [ones(T-1) R[1:end-1]]          #R(t) is regressed on (1,R(t-1))
(b,-,-,Covb,) = olsGMFn(R[2:end],x)
Stdb = sqrt.(diag(Covb))            #diag() picks out the diagonal of the variance-covariance mat
tstat = b./Stdb

printblue("Results from an AR(1):\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope"])
```

Results from an AR(1):

	coef	t-stat
constant	0.061	4.314
slope	-0.027	-2.698

```
Q = R[1:end-1] .< 0          #dummy: 1 if R(t-1) < 0
x = [ones(T-1) Q.*R[1:end-1] (1.0.-Q).*R[1:end-1]]  #R(t) is regressed on (1,Q(t-1)R(t-1),[1-Q(t-1)R(t-1)])

(b,_,_,Covb,) = OlsGMFn(R[2:end],x)
Stdb = sqrt.(diag(Covb))
tstat = b./Stdb

printblue("Results from an AR(1) with dummies:\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope (down)","slope (up)"])
```

Results from an AR(1) with dummies:

	coef	t-stat
constant	-0.007	-0.385
slope (down)	-0.102	-6.134
slope (up)	0.048	2.872

Recursive Estimation and Out-of-Sample R²

The next cell does recursive estimation (longer and longer sample) and predicts one period ahead (out-side of the sample). The performance of this prediction model is measured by an “out-of-sample” R_{OOS}^2 defined as

$$R_{OOS}^2 = 1 - \frac{\text{MSE}(\text{forecasting model})}{\text{MSE}(\text{benchmark forecast})}$$

A Remark on the Code

- The code loops over $t=100:T_b$ where T_b is the effective length of the sample. Inside the loop we use data on $y[1:t-1]$ and $x[1:t-1,:]$. This means that the first estimation uses the first 99 observations, the second estimation the first 100 observations and so forth. After the loop, we discard the first 100 observations from the output since they are just NaN.
- For estimation we use the basic OLS function `olsM()`. Clearly, `olsGMFn()` gives the same results (but is a tiny bit slower).

```

y = R[2:end]      #traditional symbol for the LHS variable
dNb = dN[2:end]   #corresponding dates, used for plotting
Tb = length(y)    #length of the effective sample

x = [ones(Tb) Q.*R[1:end-1] (1.0.-Q).*R[1:end-1]]    #predictors

(ε,e) = (fill(NaN,Tb),fill(NaN,Tb))
for t in 100:Tb
    local b
    b, = olsM(y[1:t-1],x[1:t-1,:])    #estimate on sample 1:t-1
    e[t] = y[t] - x[t,:]'b            #forecast error for t, model
    e[t] = y[t] - mean(y[1:t-1])      #forecast error for t, historical average
end
(ε,e,dNb) = (ε[100:end],e[100:end],dNb[100:end])    #drop periods without forecasts

MSE_Model = mean(abs2,ε)    #MSE for out-of-sample forecasts, same as mean(ε.^2) but quicker
MSE_Bench = mean(abs2,e)
R2oos      = 1 - MSE_Model/MSE_Bench

printblue("Performance of out-of-sample forecasting:\n")
printmat([MSE_Model;MSE_Bench;R2oos];rowNames=["MSE of AR(1)","MSE of hist avg","R2_oos"])

```

Performance of out-of-sample forecasting:

MSE of AR(1)	2.014
MSE of hist avg	2.008
R2_oos	-0.003

```

xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")    #formatting of date axis

p1 = plot( dNb,[ε.^2 e.^2],
    linecolor = [:blue :red],
    linestyle = [:solid :dash],
    label = ["AR(1) model" "historical mean"],
    legend = :topleft,
    xticks = (xTicksLoc,xTicksLab),
    title = "forecast errors^2" )
display(p1)

```



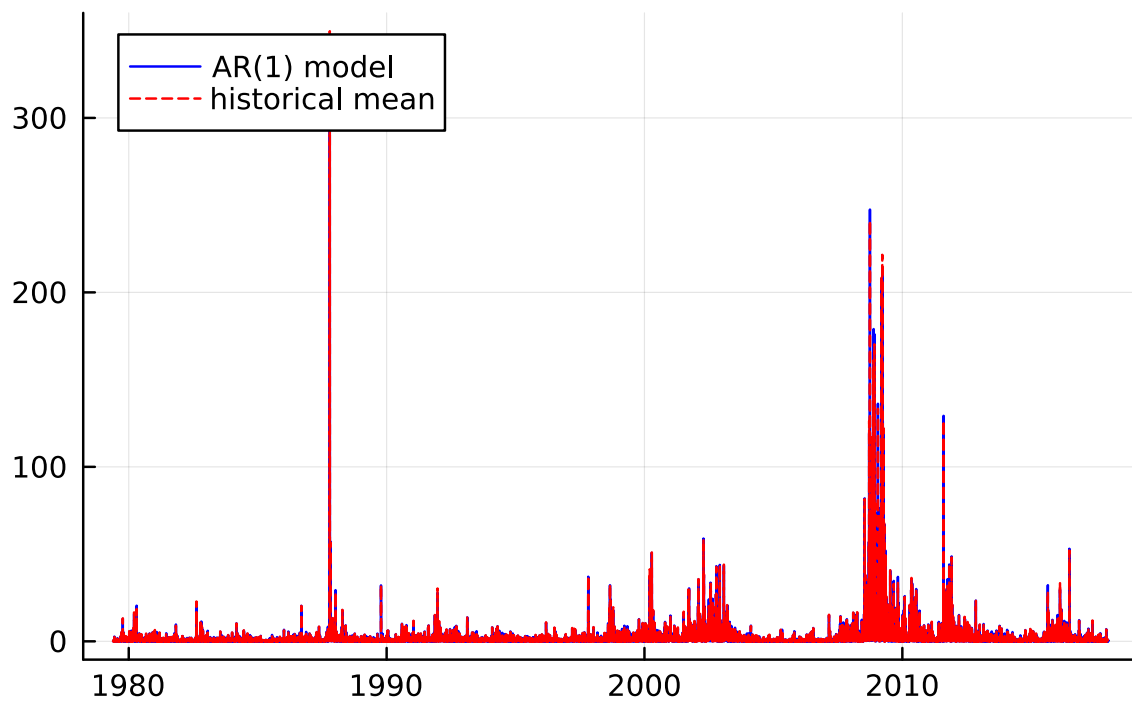
```

p2 = plot( dNb,[cumsum(e.^2) cumsum(e.^2)],
    linecolor = [:blue :red],
    linestyle = [:solid :dash],
    label = ["AR(1) model" "historical mean"],
    legend = :topleft,
    xticks = (xTicksLoc,xTicksLab),
    title = "Cumulated forecast errors^2" )
display(p2)

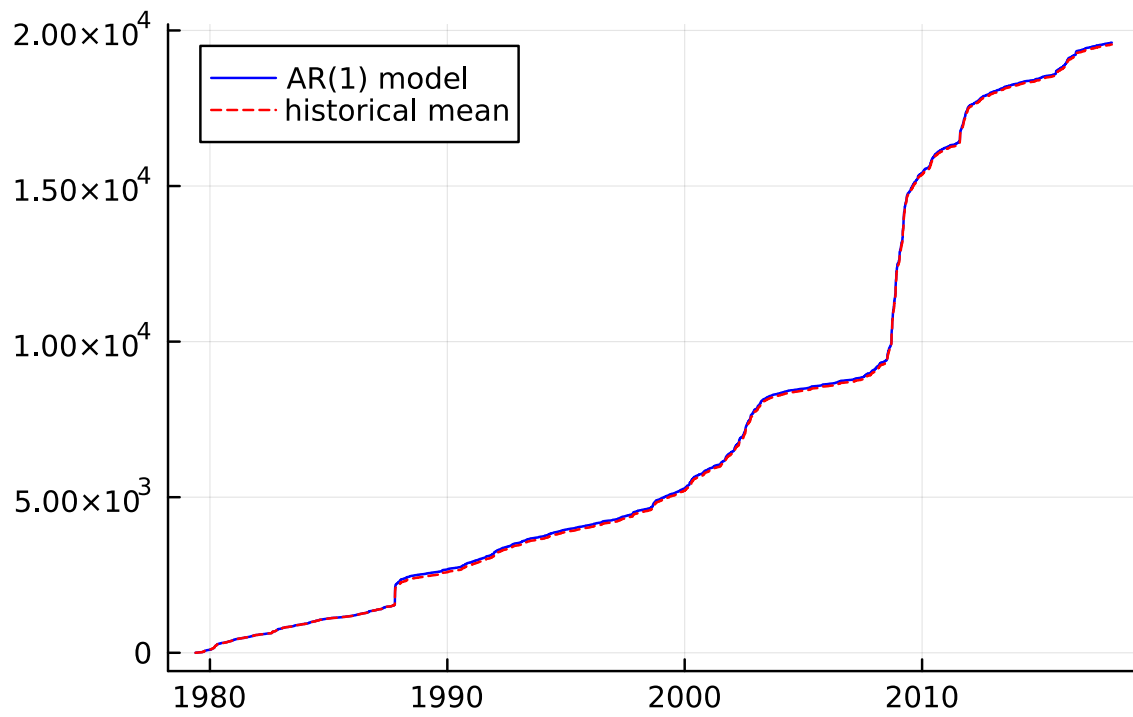
p3 = plot( dNb,cumsum(e.^2) - cumsum(e.^2),
    linecolor = :black,
    linestyle = :solid,
    label = ["AR(1) model - historical mean"],
    xticks = (xTicksLoc,xTicksLab),
    title = "Difference of cumulated forecast errors^2" )
display(p3)

```

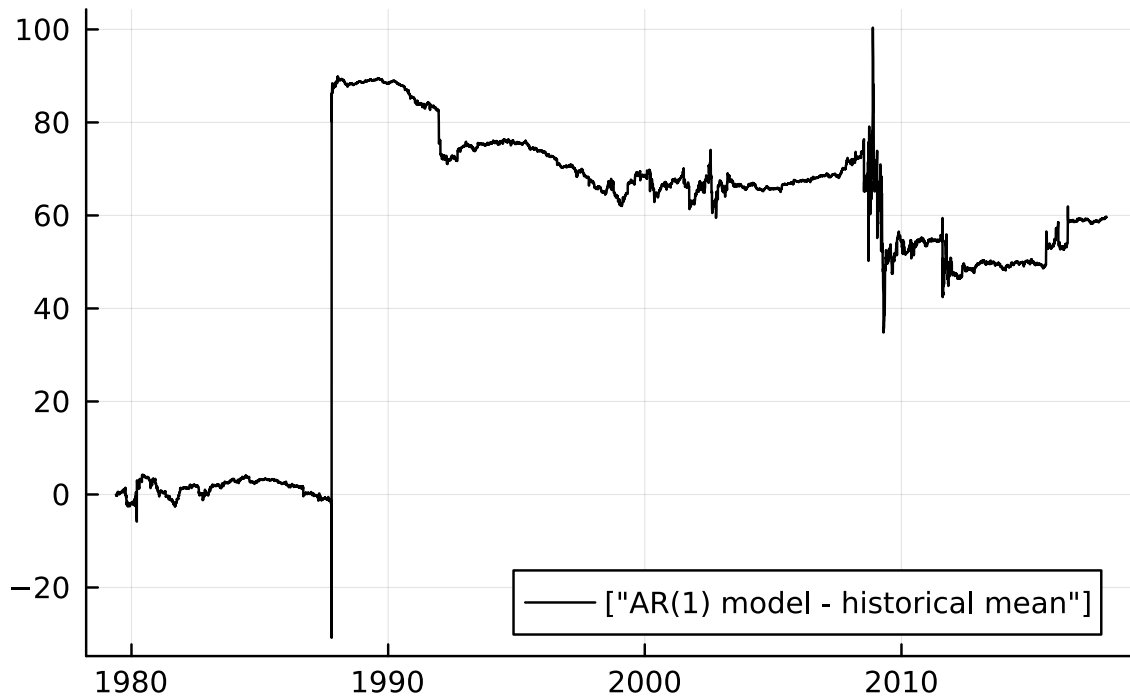
forecast errors²



Cumulated forecast errors²



Difference of cumulated forecast errors²



Mariano-Diebold and Clark-West Tests (extra)

The Mariano-Diebold and Clark-West tests compare the prediction errors of two models (e : benchmark; ϵ : your model). Notice that the Mariano-Diebold test is not well suited for nested models (when your model is an augmented version of the baseline model). Use the Clark-West in that case.

If $g_1 = e^2$ and $g_2 = \epsilon^2$ are not autocorrelated, then the standard deviation of the sample average is $\text{Std}(g_i)/\sqrt{T}$. For simplicity, this assumption is used below. (Otherwise, use a Newey-West estimator or similar.)

```
"""
    MDCW(e,ε)

Moment conditions doing MD or CW tests of two series of forreast errors (e: benchmark, ε: from the
"""
MDCW(e,ε) = hcat(e.^2 - ε.^2, 2*e.*(e - ε)) #Mariano-Diebold and Clark&West
```

MDCW

```

g = MDCW(e,e)          #e,e are from the recursive estimation (above)
Tg = size(g,1)

μ      = mean(g,dims=1)
Stdμ   = std(g,dims=1)/sqrt(Tg)
tstats = μ./Stdμ

printblue("t-stats of tests of difference in performance (errors^2)\n")
printmat(tstats';rowNames=["Mariano-Diebold","Clark-West"])

```

t-stats of tests of difference in performance (errors^2)

Mariano-Diebold	-0.430
Clark-West	0.814

A Trading Strategy

This section implements a momentum strategy (buy past winners, short sell past losers), which is rebalanced daily. For simplicity, we disregard trading costs.

Implementing the Strategy

1. Rank the 25 assets according to the lagged return R_{t-1} .
2. (In the evening of) period $t - 1$ buy $1/5$ of each of the 5 best assets based on the ranking in point 1. Similarly, buy $-1/5$ (short-sell) each of the 5 worst assets. Collect these portfolio weights in a vector w_t .
3. In period t , the return on the portfolio is $R_{p,t} = w_t' R_t$.
4. Repeat for all periods

A Remark on the Code

- With $x = [9, 7, 8]$, the `rankPs(x)` function (see below) gives the output `[3,1,2]`. This says, for instance, that 7 is the lowest number (rank 1).
- The portfolio weights are not stored for use outside the loop. Only the strategy (portfolio) return is.

```

"""
    rankPs(x)

Calculates the ordinal rank of each element in a vector 'x'. As an alternative,
use 'ordinalrank' from the 'StatsBase.jl' package.

"""
rankPs(x) = invperm(sortperm(x))

```

rankPs

```

m = 5 #number of assets in long/short leg of portfolio

R = copy(R25) #we recygly the 'R' notation: it is now a Tx25 matrix
(T,n) = size(R)

Rp = fill(NaN,T)
for t in 2:T #loop over periods, save portfolio returns
    #local r,w #local/global is needed in script
    r = rankPs(R[t-1,:])
    w = (r.<=m)*(-1/m) + ((n-m+1).<=r)*(1/m)
    Rp[t] = w'R[t,:] #same as sum(w.*R[t,:])
end

Rp = Rp[2:end];

```

We calculate the mean (excess) return, its standard deviation and the Sharpe ratio. Annualising is done by assuming 250 trading days per year (means are multiplied by 250 and standard deviation by $\sqrt{250}$). Compare with the excess return on passively holding an equity market index.

```

μ = mean(Rp) #strategy
σ = std(Rp)

Rme = Rm - Rf #market
μm = mean(Rme[2:end])
σm = std(Rme[2:end])

printblue("Annualised results:\n")
result = [μ*250;σ*sqrt(250);μ/σ*sqrt(250)]
resultm = [μm*250;σm*sqrt(250);μm/σm*sqrt(250)]
printmat(result,resultm,colNames=["Strategy","market"],rowNames=["mean","std","SR"])

```

Annualised results:

	Strategy	market
mean	16.134	8.307
std	9.871	16.770
SR	1.634	0.495

To cumulate the returns to a return index, use $(1 + R_1)$, $(1 + R_1)(1 + R_2)$, etc. However, this does not work for excess returns, so convert them to net returns by adding the riskfree rate.

It is often more useful to show the logarithm of the return index. The slope can then be interpreted as a return.

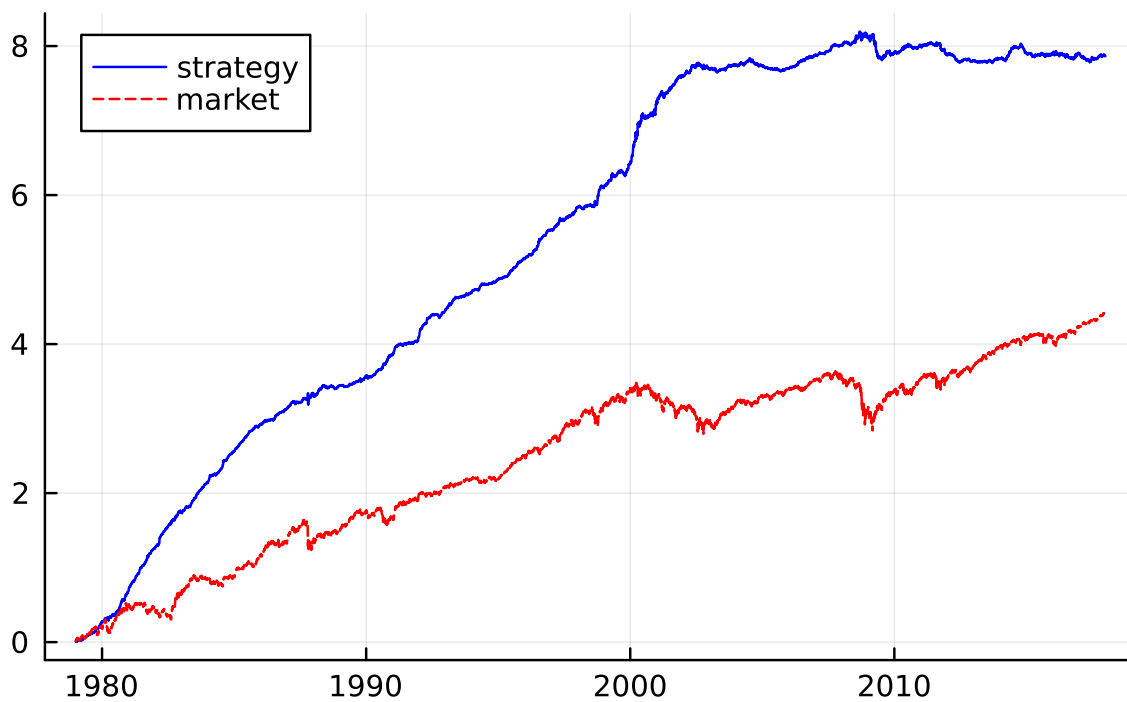
```
Rbp = Rp + Rf[2:end]           #add Rf to make it a net return

Vp = cumprod(1 .+ Rbp/100)      #cumulate to return index ("value")
Vm = cumprod(1 .+ Rm[2:end]/100); #notice /100 since percentage returns
```

```
xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")
```

```
plot( dN[2:end],log.([Vp Vm]),
      linecolor = [:blue :red],
      linestyle = [:solid :dash],
      label = ["strategy" "market"],
      legend = :topleft,
      xticks = (xTicksLoc,xTicksLab),
      title = "log return indices" )
```

log return indices



Efficient Markets

This notebook describes and tests the predictability of asset returns (autocorrelations, autoregressions, out-of-sample R^2 , Mariano-Diebold test) and implements a simple trading strategy.

Load Packages and Extra Functions

```
using Printf, LinearAlgebra, Dates, DelimitedFiles, Statistics, StatsBase

include("src/printmat.jl")
include("src/UtilityFnMFT.jl")
include("src/OlsGMNW.jl");           #function for doing OLS
```

```
using Plots
default(size = (480,320),fmt = :png)
```

Load Data

Daily excess return on S&P 500.

```
xx      = readlm("Data/ReSP500Daily.csv",' ',skipstart=1)
y_data  = Float64.(xx[:,2])
dN      = Date.(xx[:,1],"yyyy-mm-dd")
xx      = nothing

(Re,dN) = excise(y_data,dN) #drop dates with missing values
T       = size(Re,1)
println("Sample size: $T")
```

Sample size: 11479

Autocorrelations

Autocorrelation s is

$$\rho_s = \text{Corr}(R_t, R_{t-s})$$

In large samples, $\sqrt{T} \hat{\rho}_s \sim N(0, 1)$ if the true value is $\rho_s = 0$ for all s .

A Remark on the Code

The [StatsBase.jl](#) package contains methods for estimating autocorrelations (see `autocor()` below).

```
plags = 1:5          #different lags to calculate autocorrelations for
ρ      = autocor(Re,plags)

printblue("autocorrelations and their t-stats:\n")
printmat(ρ,sqrt(T)*ρ;colNames=["ρ","t-stat"],rowNames=plags,cell00="lag")
```

autocorrelations and their t-stats:

lag	ρ	t-stat
1	-0.054	-5.767
2	-0.006	-0.624
3	-0.007	-0.786
4	-0.032	-3.435
5	-0.003	-0.278

Autoregressions

An AR(1) is

$$R_t = c + a_1 R_{t-1} + \varepsilon_t.$$

We also consider an asymmetric AR(1)

$$R_t = \alpha + \beta \delta_{t-1} R_{t-1} + \gamma (1 - \delta_{t-1}) R_{t-1} + \varepsilon_t,$$

where $\delta_{t-1} = 1$ if $R_{t-1} < 0$ and zero otherwise.

Both models can be estimated by OLS.

A Remark on the Code

- The function `OlsGM()` does OLS and reports, among other things, the variance-covariance matrix of the estimates (called `Covb` below).
- To get standard errors, we first extract the variances from the variance-covariance matrix (`diag(Covb)`) and then calculate the square roots (`sqrt().()`). Notice the dot. in the latter calculation: to calculate the square root of each element in the vector of variances.
- The t-stats are then just the point estimates (minus the null hypothesis, which is here 0) divided by the standard errors.

```
x = [ones(T-1) Re[1:end-1]]          #R(t) is regressed on (1,R(t-1))
(b,--,Covb,) = OlsGM(Re[2:end],x)
Stdb = sqrt.(diag(Covb))             #diag() picks out the diagonal of the variance-covariance mat
tstat = b./Stdb

printblue("Results from an AR(1):\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope"])
```

Results from an AR(1):

	coef	t-stat
constant	0.029	2.777
slope	-0.054	-5.775

```
δ = Re[1:end-1] .< 0                #dummy: 1 if R(t-1) < 0
x = hcat(ones(T-1), δ.*Re[1:end-1], (1.0.-δ).*Re[1:end-1])  #R(t) is regressed on (1,Q(t-1)R(t-1),

(b,--,Covb,) = OlsGM(Re[2:end],x)
Stdb = sqrt.(diag(Covb))
tstat = b./Stdb

printblue("Results from an AR(1) with dummies:\n")
printmat([b tstat];colNames=["coef","t-stat"],rowNames=["constant","slope (down)","slope (up)"])
```

Results from an AR(1) with dummies:

	coef	t-stat
constant	0.004	0.266
slope (down)	-0.086	-5.666
slope (up)	-0.018	-1.131

Recursive Estimation and Out-of-Sample R²

The next cell does recursive estimation (longer and longer sample) and predicts one period ahead (out-side of the sample). The performance of this prediction model is measured by an “out-of-sample” R_{OOS}^2 defined as

$$R_{OOS}^2 = 1 - \frac{\text{MSE}(\text{forecasting model})}{\text{MSE}(\text{benchmark forecast})}$$

A Remark on the Code

- The code loops over $t=100:Tb$ where Tb is the effective length of the sample. Inside the loop we use data on $y[1:t-1]$ and $x[1:t-1,:]$. This means that the first estimation uses the first 99 observations, the second estimation the first 100 observations and so forth. After the loop, we discard the first 100 observations from the output since they are just NaN.
- For estimation we use the basic OLS function `olsM()`. Clearly, `olsGMFn()` gives the same results (but is a tiny bit slower).

```
y = Re[2:end]      #traditional symbol for the LHS variable
Tb = length(y)     #length of the effective sample
dNb = dN[2:end]

x = hcat(ones(Tb), delta.*Re[1:end-1], (1.0.-delta).*Re[1:end-1])    #predictors

(e,e) = (fill(NaN,Tb),fill(NaN,Tb))
for t in 100:Tb
    local b
    b,    = olsGM(y[1:t-1],x[1:t-1,:])    #estimate on sample 1:t-1
    e[t] = y[t] - x[t,:]'b                #forecast error for t, model
    e[t] = y[t] - mean(y[1:t-1])          #forecast error for t, historical average
end
(e,e,dNb) = (e[100:end],e[100:end],dNb[100:end])    #drop periods without forecasts

MSE_Model = mean(abs2,e)    #MSE for out-of-sample forecasts, same as mean(e.^2) but quicker
MSE_Bench = mean(abs2,e)
R2oos      = 1 - MSE_Model/MSE_Bench

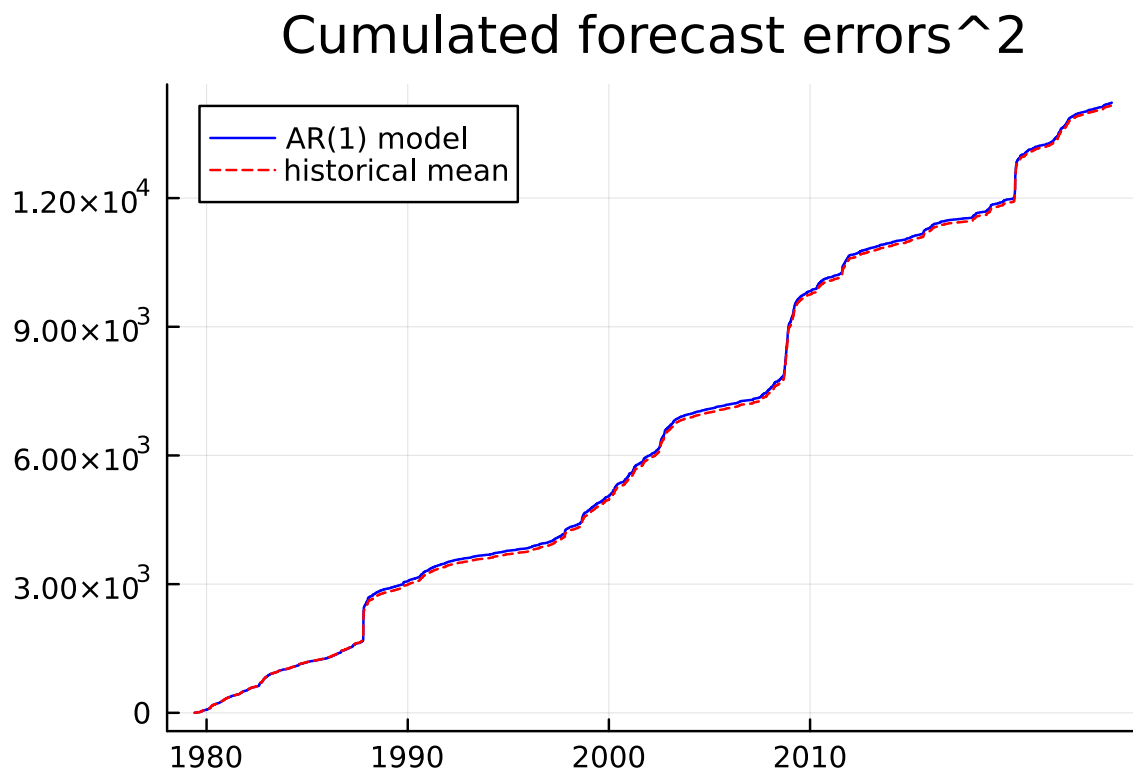
printblue("Performance of out-of-sample forecasting:\n")
printmat([MSE_Model;MSE_Bench;R2oos];rowNames=["MSE of AR(1)","MSE of hist avg","R2_oos"])
```

Performance of out-of-sample forecasting:

MSE of AR(1)	1.250
--------------	-------

MSE of hist avg 1.245
R2_oos -0.004

```
xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]  
xTicksLab = Dates.format.(xTicksLoc,"Y")      #formatting of date axis  
  
p2 = plot( dNb,[cumsum(e.^2) cumsum(e.^2)],  
          linecolor = [:blue :red],  
          linestyle = [:solid :dash],  
          label = ["AR(1) model" "historical mean"],  
          legend = :topleft,  
          xticks = (xTicksLoc,xTicksLab),  
          title = "Cumulated forecast errors^2" )  
display(p2)
```



A Trading Strategy

This section implements a momentum strategy (buy past winners, short sell past losers), which is rebalanced daily. For simplicity, we disregard trading costs.

Implementing the Strategy

1. Rank the 25 assets according to the lagged return R_{t-1} .
2. (In the evening of) period $t - 1$ buy $1/5$ of each of the 5 best assets based on the ranking in point 1. Similarly, buy $-1/5$ (short-sell) each of the 5 worst assets. Collect these portfolio weights in a vector w_t .
3. In period t , the return on the portfolio is $R_{p,t} = w_t' R_t$.
4. Repeat for all periods

A Remark on the Code

- With $x = [9, 7, 8]$, the `rankPs(x)` function (see below) gives the output `[3, 1, 2]`. This says, for instance, that 7 is the lowest number (rank 1).
- The portfolio weights are not stored for use outside the loop. Only the strategy (portfolio) return is.

Loading Data

The data set contains daily data of the equity market return, riskfree rate and the returns of the 25 Fama-French portfolios. All returns are in percent.

```
x = readlm("Data/MomentumSR.csv",',')
dN = Date.(x[:,1],"yyyy-mm-dd")
x = Float64.(x[:,2:end])
(Rm,Rf,R) = (x[:,1],x[:,2],x[:,3:end])

println("size of dN, Rm, Rf, R")
println(size(dN),"\n",size(Rm),"\n",size(Rf),"\n",size(R))

Re25 = R .- Rf           #excess returns
Rme = Rm .- Rf

(T,n) = size(Re25);      #number of periods and assets
```

```

size of dN, Rm, Rf, R
(11598,)
(11598,)
(11598,)
(11598, 25)

```

```

"""
    rankPs(x)

Calculates the ordinal rank of each element in a vector 'x'. As an alternative,
use 'ordinalrank' from the 'StatsBase.jl' package.

"""
rankPs(x) = invperm(sortperm(x))

```

rankPs

```

m = 5 #number of assets in long/short leg of portfolio
L = 22 #window of signal starts here

Rp = fill(NaN,T)
for t in L+1:T #loop over periods, save portfolio returns
    #local r,w,z #local/global is needed in script
    z = vec(mean(Re25[t-L:t-1,:],dims=1))
    r = rankPs(z)
    w = -(r.<=m)/m + ((n-m+1).<=r)/m #short, long leg
    Rp[t] = w'Re25[t,:] #same as sum(w.*R[t,:])
end

Rp = Rp[L+1:end];

```

We calculate the mean (excess) return, its standard deviation and the Sharpe ratio. Annualising is done by assuming 250 trading days per year (means are multiplied by 250 and standard deviation by $\sqrt{250}$). Compare with the excess return on passively holding an equity market index.

```

μ = mean(Rp) #strategy
σ = std(Rp)

Rme = Rm - Rf #market
μm = mean(Rme[L+1:end])
σm = std(Rme[L+1:end])

```

```

printblue("Annualised results:\n")
result = [ $\mu \times 250$ ;  $\sigma \times \text{sqrt}(250)$ ;  $\mu / \sigma \times \text{sqrt}(250)$ ]
result_m = [ $\mu_m \times 250$ ;  $\sigma_m \times \text{sqrt}(250)$ ;  $\mu_m / \sigma_m \times \text{sqrt}(250)$ ]
printmat(result,result_m,colNames=["Strategy","market"],rowNames=["mean","std","SR"])

```

Annualised results:

	Strategy	market
mean	9.510	8.836
std	11.160	17.334
SR	0.852	0.510

To cumulate the returns to a return index, use $(1 + R_1)$, $(1 + R_1)(1 + R_2)$, etc. However, this does not work for excess returns, so convert them to net returns by adding the riskfree rate.

It is often more useful to show the logarithm of the return index. The slope can then be interpreted as a return.

```

Rb_p = R_p + Rf[L+1:end]          #add Rf to make it a net return

V_p = cumprod(1 .+ Rb_p/100)      #cumulate to return index ("value")
V_m = cumprod(1 .+ Rm[L+1:end]/100); #notice /100 since percentage returns

```

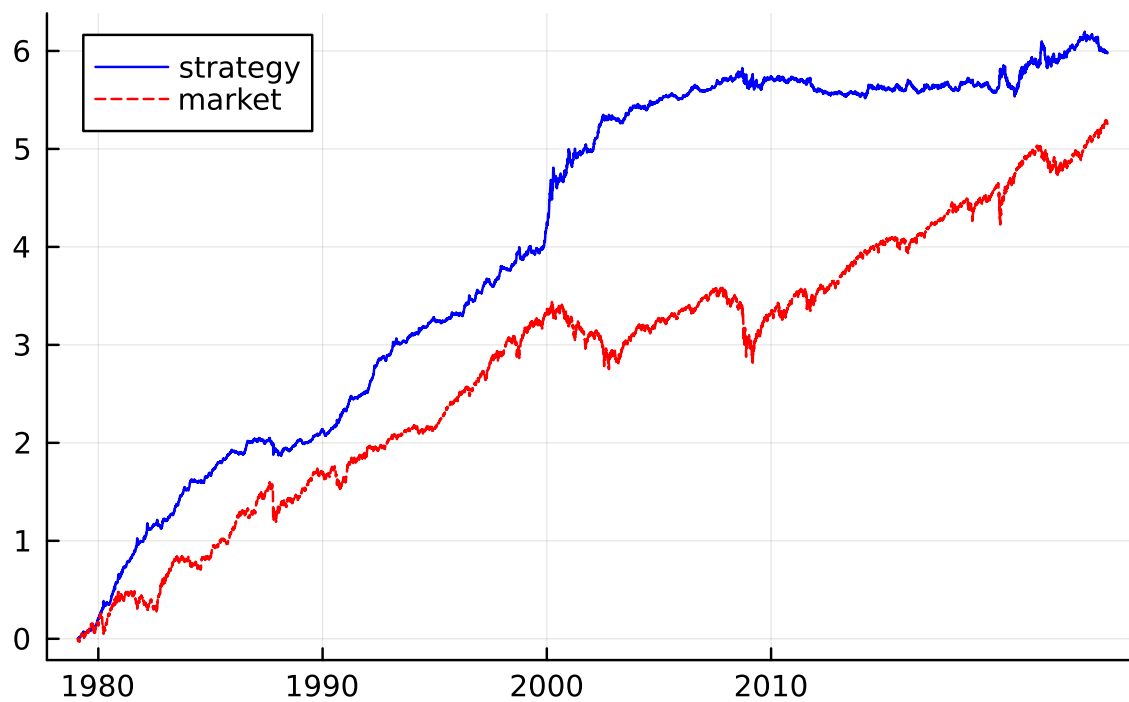
```

xTicksLoc = [Date(1980);Date(1990);Date(2000);Date(2010)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

plot( dN[L+1:end],log.([V_p V_m]),
      linecolor = [:blue :red],
      linestyle = [:solid :dash],
      label = ["strategy" "market"],
      legend = :topleft,
      xticks = (xTicksLoc,xTicksLab),
      title = "log return indices" )

```

log return indices



Performance Evaluation

The first part of this notebook summarizes the conventional performance measures for an investment fund. The second part does a “style analysis” to find out how the fund has changed its portfolio weights over time.

Load Packages and Extra Functions

```
using Printf, Dates, DelimitedFiles, Statistics

include("src/printmat.jl")
include("src/OlsM.jl");
```

```
using Plots

gr(size=(480,320))
default(fmt = :png)
```

Loading Data

The weekly return data for the mutual funds that we will evaluate are in the matrix R , the benchmarks (a number of returns on different asset indices) in R_b and the riskfree rate in the vector R_f .

A Remark on the Code

In the code below, x from `readdlm()` is an Any matrix (the first column are strings, the other columns are numbers). It often helps to convert to the right type. For the numbers in column 2:end we can do that by `Float64.(x[:,2:end])` (or alternatively by `convert.(Float64,x[:,2:end])`).

```

(x,header) = readlm("Data/Fin1StyleAnalysis.csv",'',header=true)
(IndNames,FundNames) = (header[2:9],header[11:12])    #names of variables

dN          = Date.(x[:,1],"yyyy-mm-dd")
(Rb,Rf,R)   = (Float64.(x[:,2:9]),Float64.(x[:,10]),Float64.(x[:,11:12])) #benchmarks, riskfree, fun

printblue("The data is weekly. The first 4 observations are:")
printmat(first(dN,4))

printmat(IndNames)
printmat(FundNames)

```

The data is weekly. The first 4 observations are:

```

1999-01-15
1999-01-22
1999-01-29
1999-02-05

```

```

    S&P 500
S&P Mid Cap
S&P US 600
DJ GlobalDevxUS
DJ GlobalEmerg
    Corporate
    US Tbills
U.S. Treasury

```

```

Putnam Asset Allocation: Growth A
Vanguard Wellington

```

Performance Evaluation

The next few cells report a number of different performance measures for the funds. We annualize the weekly return statistics by multiplying by $\sqrt{52}$ or 52, depending on the measure. In many cases, we also convert the numbers to percentage (%) by multiplying by 100.

A Remark on the Code

We use R_e to denote excess returns for the funds and R_{e_m} the excess return of the market. (R^e_m looks ugly, so we avoid it.) Similarly for μ_e and μ_{e_m} .

```
Re = R .- Rf          #excess returns of the funds
Rem = Rb[:,1] - Rf;    #excess returns of the marketm (S&P 500)
```

Average Returns, Sharpe Ratio and M²

The Sharpe ratio (SR) is μ^e/σ , where μ^e is the average excess return and σ the standard deviation of the excess return. The M^2 is the difference of the SRs of a fund and the market, scaled by the standard deviation of the market.

Mean returns are annualized by $\times 52$ and standard deviations by $\times \sqrt{52}$, so a SR by $\times \sqrt{52}$.

```
"""
    SRFn(Re,Rem,AnnFactor=[])

Calculate Sharpe ratios and M2
"""
function SRFn(Re,Rem,AnnFactor=Int[])
    μe = mean(Re,dims=1)
    μm = mean(Rem)
    σ = std(Re,dims=1)          #std, 1xn
    σm = std(Rem)
    SR = μe./σ
    SRm = μm/σm
    MM = (SR .- SRm)*σm
    MMm = 0                    #(SRm.-SRm)*σm=0
    if !isempty(AnnFactor)
        (μe,μm,MM) = (μe,μm,MM).*AnnFactor
        (SR,SRm) = (SR,SRm).*sqrt(AnnFactor)
    end
    return μe, μm, SR, SRm, MM
end
```

SRFn

```
(μe, μm, SR, SRm, MM) = SRFn(Re,Rem,52)

xut = hcat([μm;μe']*100,[SRm;SR'],[0;MM']*100) #returns and MM in percent
printmat(xut;colNames=["ERe","SR","M2"],rowNames=["Market";FundNames])
```

	ERe	SR	M ²
Market	7.186	0.403	0.000
Putnam Asset Allocation: Growth A	5.456	0.369	-0.611
Vanguard Wellington	5.978	0.530	2.260

Appraisal Ratio, Traynor's Ratio and

Appraisal Ratio

Estimate a single index model

$$R_t^e = \alpha + \beta R_{mt}^e + \epsilon_t$$

The appraisal ratio (AR) is $\alpha / \text{Std}(\epsilon)$.

Treynor's Ratio and T²

Treynor's ratio is μ^e / β , where β is the slope from the single index regression.

A Remark on The Code

- The `OLsM(y,x)` function (included above) does a regression for each of the n columns in y on the same set of K regressors in x . It returns $(K \times n)$ coefficients b and $(1 \times n)$ standard deviations of the residuals σ .

```

"""
    AppraisalRatioFn(Re,Re_m,AnnFactor=52)

Calculate appraisal ratio, Treynor's ratio and  $\alpha$ .
"""
function ARTRFn(Re,Re_m,AnnFactor=Int[])
     $\mu^e$  = mean(Re,dims=1)
     $\mu_{em}$  = mean(Re_m)
    T = size(Re,1)
    (b, $\sigma_e$ ) = OLsM(Re,[ones(T) Re_m]) #OLS regression for each column in Re_p
    ( $\alpha$ , $\beta$ ) = (b[1:1,:],b[2:2,:]) #intercept and slopes (1xn)
    AR =  $\alpha$ ./ $\sigma_e$  # $\alpha$  and  $\sigma_e$  are 1xn
    (Re == Re_m) && (AR=0) #impose this because  $\alpha$ , $\sigma_e$  close to 0 may give stange ratio
    TR =  $\mu^e$ ./ $\beta$ 
    TT = TR .-  $\mu_{em}$ 

```

```

if !isempty(AnnFactor)
    (α,TR,TT) = (α,TR,TT).*AnnFactor
    AR        = AR.*sqrt(AnnFactor)
end
return AR, TR, TT, α
end

```

ARTRFn

```

(AR,TR,TT,α)      = ARTRFn(Re,Rem,52)
(ARm,TRm,TTm,αm) = ARTRFn(Rem,Rem,52)

xut = hcat([ARm;AR'],[TRm;TR']*100,[TTm;TT']*100,[αm;α']*100) #TR, TT and α in percent
printmat(xut,colNames=["AR","TR","T²","α"],rowNames=["Market";FundNames])

```

	AR	TR	T ²	α
Market	0.000	7.186	0.000	-0.000
Putnam Asset Allocation: Growth A	-0.071	6.831	-0.355	-0.284
Vanguard Wellington	0.455	9.999	2.813	1.682

Style Analysis

The regression is $Y = \sum_{j=1}^K b_j X_j + \varepsilon$, where $0 \leq b_j$ and $\sum_{j=1}^K b_j = 1$.

Write the sum of squared residuals of the regression as

$$(Y - Xb)'(Y - Xb) = Y'Y - 2Y'Xb + b'X'Xb.$$

Only the two last terms matter for the choice of b .

A Remark on the Code

- The code uses the [OSQP.jl](#) package which solves problems of the type: $\min 0.5b'Pb + q'b$ subject to $l \leq Ab \leq u$.
- The restricted regression (above) can easily be written in this form by letting $P = X'X/T$ and $q = -X'Y/T$. These choices give the loss function $0.5b'X'Xb/T - Y'Xb/T$, which is proportional $(0.5/T)$ to the last two terms in the sum of squared residuals. Dividing by T improves the numerical stability.
- The A matrix is used to construct the restrictions (b sums to 1 and $0 \leq b \leq 1$).

```
using LinearAlgebra, SparseArrays, OSQP #OSQP needs SparseArrays
```

```
"""
    StyleAnalysisPs(Y,X)

# Input:
- `Y::Vector`:      T-vector, returns of a fund
- `X::Matrix`:      TxK matrix, returns on m benchmarks

# Output:
- `b_sa::Vector`:   K-vector, restricted regression coefficients
- `b_ls::Vector`:   K-vector, OLS regression coefficients
- `R2_sa::Number`:  pseudo-R2

"""
function StyleAnalysisPs(Y,X)

    (T,K) = (size(X,1),size(X,2))
    b_ls = X\Y #LS estimate, no restrictions

    settings = Dict(:verbose => false)
    P = X'X/T
    q = -X'Y/T
    A = [ones(1,K);I] #1st restriction: sum(b)=1,
    l = [1;zeros(K)] #the rest: 0<=b<=1
    u = [1;ones(K)]

    settings = Dict(:verbose => false)
    model = OSQP.Model() # (P,A) must be `Sparse`, (q,l,u) vectors of `Float64`
    OSQP.setup!(model; P=sparse(P), q=q, A=sparse(A), l=l, u=u, settings...)
    result = OSQP.solve!(model)
    b_sa = result.info.status == :Solved ? result.x : NaN

    Yhat = X*b_sa
    R2_sa = cor(Y,Yhat)^2 #pseudo-R2

    return b_sa, b_ls, R2_sa
end
```

StyleAnalysisPs

The next cell makes a “style analysis regression” based on the entire sample. The dependent variable is the first mutual fund in R (see data loading) and the regressors include all indices (again, see data loading).

```
(b,b_ls,R2) = StyleAnalysisPs(R[:,2],Rb)

printblue("OLS and style analysis coeffs:")
colNames = ["OLS","Restricted LS"]
xut      = [b_ls b;sum([b_ls b],dims=1)]
printmat(xut;colNames,rowNames=[IndNames,"sum"],width=15)

printred("Notice that the restricted LS has (approximately) no negative coeffs and that sum(coeffs)
```

OLS and style analysis coeffs:

	OLS	Restricted LS
S&P 500	0.506	0.508
S&P Mid Cap	0.026	0.014
S&P US 600	-0.011	-0.000
DJ GlobalDevxUS	0.087	0.081
DJ GlobalEmerg	-0.006	-0.000
Corporate	0.280	0.276
US Tbills	0.753	0.117
U.S. Treasury	-0.007	0.004
sum	1.628	1.000

Notice that the restricted LS has (approximately) no negative coeffs and that sum(coeffs) = 1

Redo the Style Analysis on a Moving Data Window

of size WinSize (see below). Then plot to see how the coefficients change over time.

```
(T,K) = size(Rb)
WinSize = 104

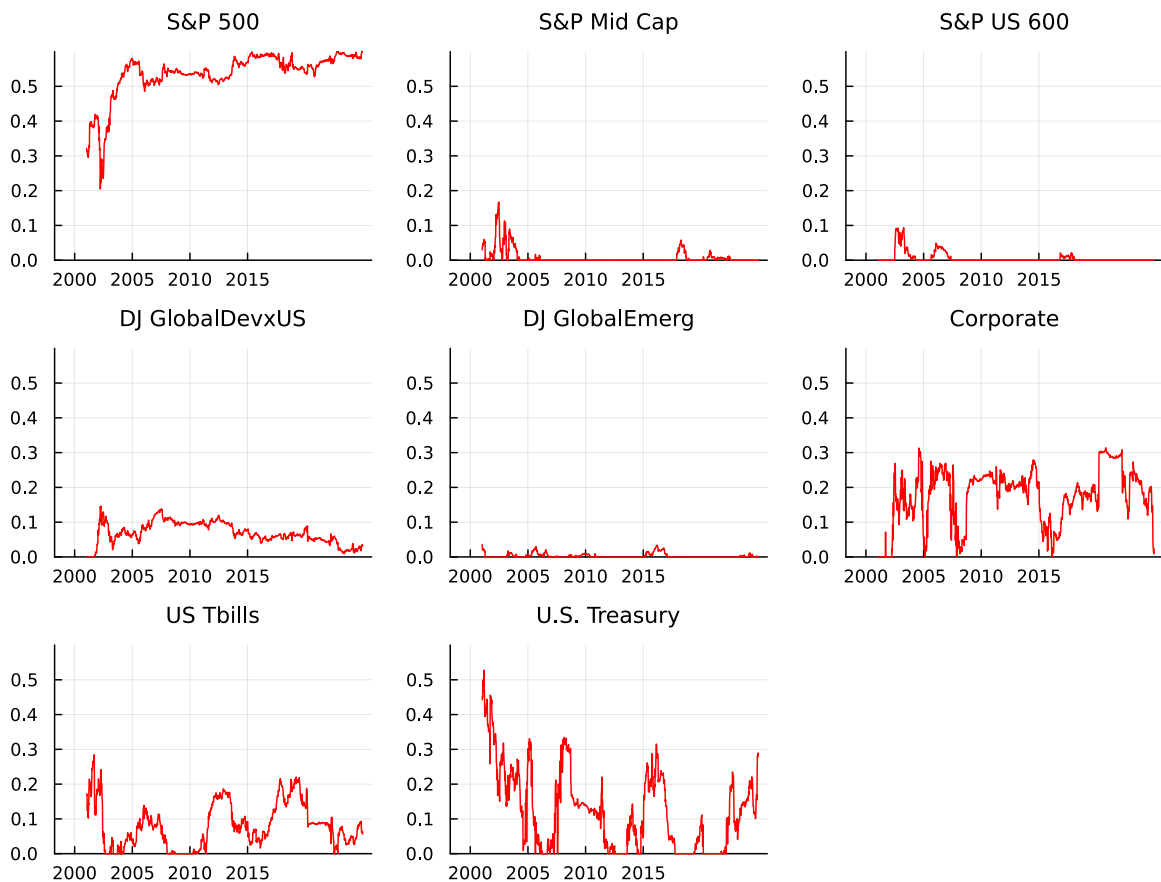
(b,R2) = (fill(NaN,T,K),fill(NaN,T))
for t in WinSize+1:T
    #local vv          #local/global is needed in script
    vv      = (t-WinSize):t    #moving data window
    (b[t,:],-,R2[t]) = StyleAnalysisPs(R[vv,2],Rb[vv,:])
end
```

```

xTicksLoc = [Date(2000);Date(2005);Date(2010);Date(2015)]
xTicksLab = Dates.format.(xTicksLoc,"Y")

p1 = plot( dN,b,
           layout = @layout[a a a;a a a;a a _],    #_ to get blank subplot
           legend = false,
           size = (800,600),
           linecolor = :red,
           xticks = (xTicksLoc,xTicksLab),
           ylims = (0,0.6),
           title = reshape(string.(IndNames),1,:),
           titlefontsize = 10 )
display(p1)

```



Long Run Portfolio Choice

This notebook summarizes how the distribution of returns is affected by the investment horizon.

Load Packages and Extra Functions

```
using Printf, LinearAlgebra, Distributions

include("src/printmat.jl")
include("src/UtilityFnMFT.jl");
```

```
using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Distribution of Long-Run Returns in the iid Case

If the excess log return over one period r^e is iid $N(\mu, \sigma^2)$, then excess log return over q periods is z_q^e is $N(q\mu, q\sigma^2)$. We use this result to draw the density function and to calculate the probability of $z_q^e < 0$.

A Remark on the Code

The Distributions.jl package defines a normal distribution by `Normal( $\mu, \sigma$ )`, that is, using the standard deviation (not the variance) as the 2nd argument. For horizon q we thus use `Normal( $q*\mu, \sqrt{q}*\sigma$ )`.

Pdfs of Long-Run Returns (for Different Horizons)

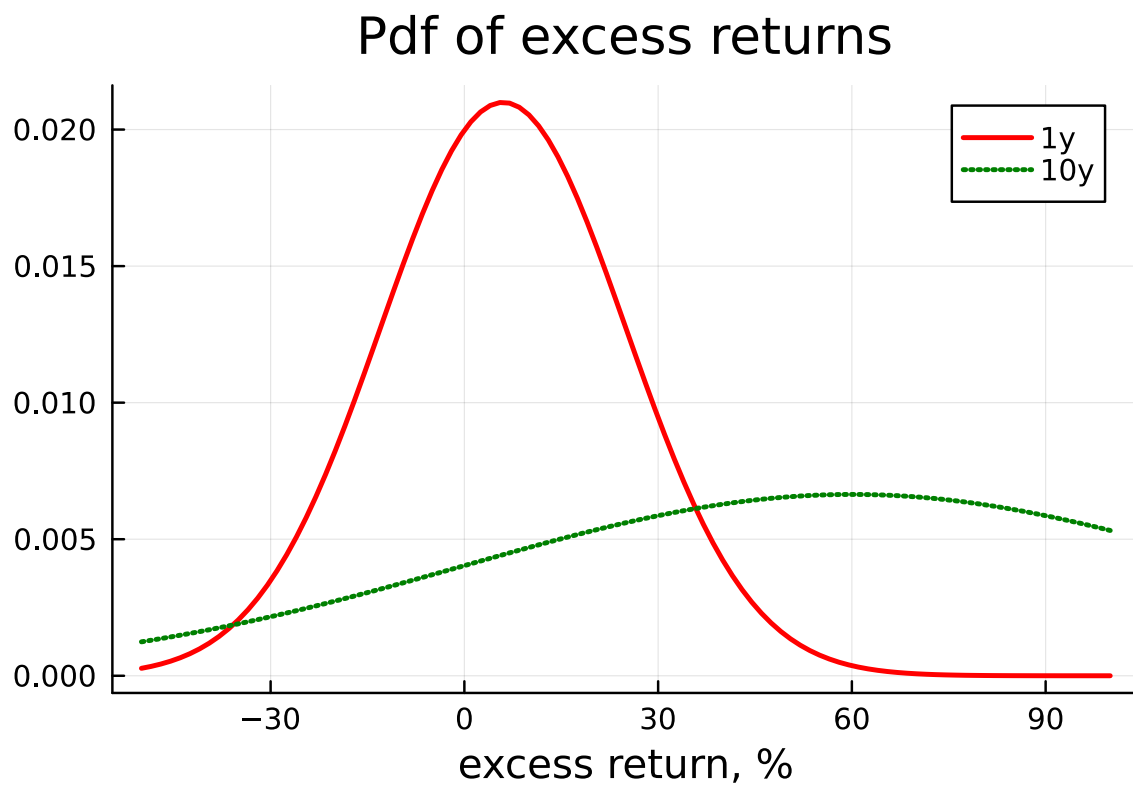
```

ze_range = range(-50,100,length=101)
μe = 0.06*100                                     #average excess return of annual data
σ = 0.19*100                                       #std of annual data

pdf_1y = pdf.(Normal(μe,σ),ze_range)             #pdf of 1-year returns
pdf_10y = pdf.(Normal(10*μe,sqrt(10)*σ),ze_range); #pdf of 10-year returns

p1 = plot( ze_range,[pdf_1y pdf_10y],
           linecolor = [:red :green],
           linestyle = [:solid :dot],
           linewidth = 2,
           label = ["1y" "10y"],
           title = "Pdf of excess returns",
           xlabel = "excess return, %" )
display(p1)

```



Prob($z < 0$) for Different Horizons

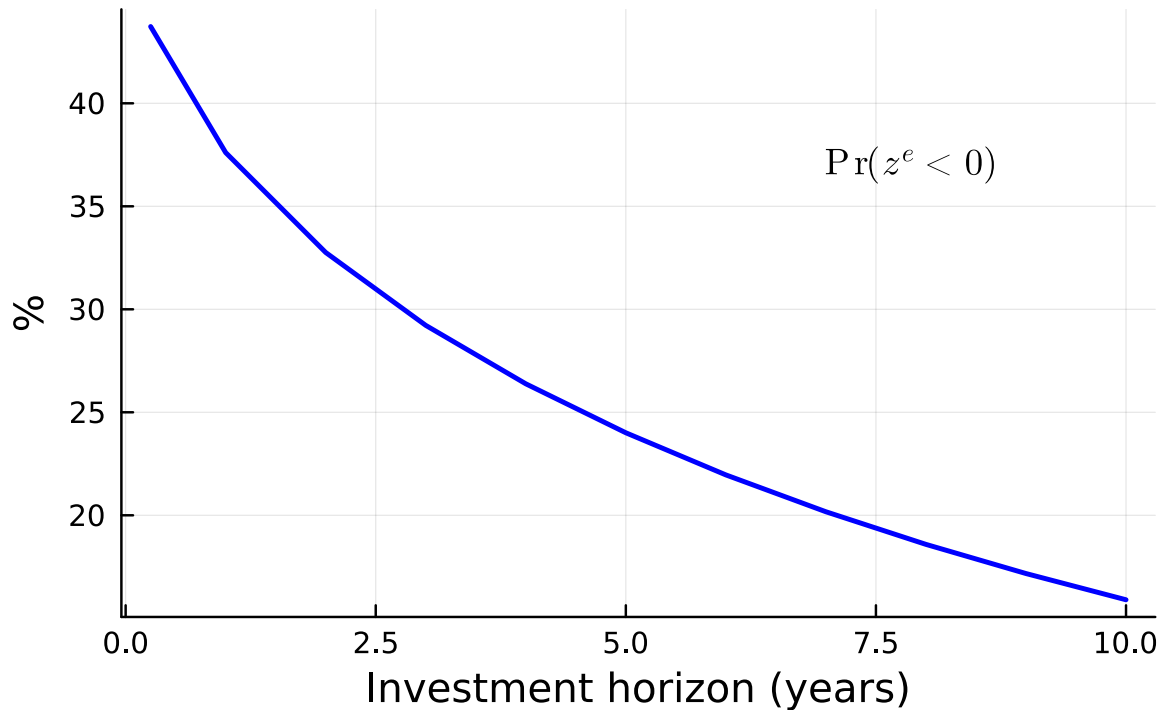
A Remark on the Code

code like `y = [exp(q) for q in 1:10]` creates a vector with 10 elements (`exp(1),exp(2)...`). It's a short form of a loop. In some cases, this is more conveniently written as `exp.(1:10)`, but in other cases a loop works better (here it does since we have to change the distribution for each `q`).

```
q_max      = 10      #longest horizon considered in the calculations
qM = vcat(0.25,1:q_max)
ProbNegReturn = [cdf(Normal(q*μ^e,sqrt(q)*σ),0) for q in qM]  #Pr(z<=0) for different horizons

txt = L"\mathrm{Pr}\{z^e<0\}"
p1 = plot( qM,ProbNegReturn*100,
           linecolor = :blue,
           linewidth = 2,
           legend = false,
           title = "Probability of negative excess return",
           xlabel = "Investment horizon (years)",
           ylabel = "%",
           annotation = (7,37,text(txt,10,:left)) )
display(p1)
```

Probability of negative excess return



$E(z | z < 0)$ for Different Horizons

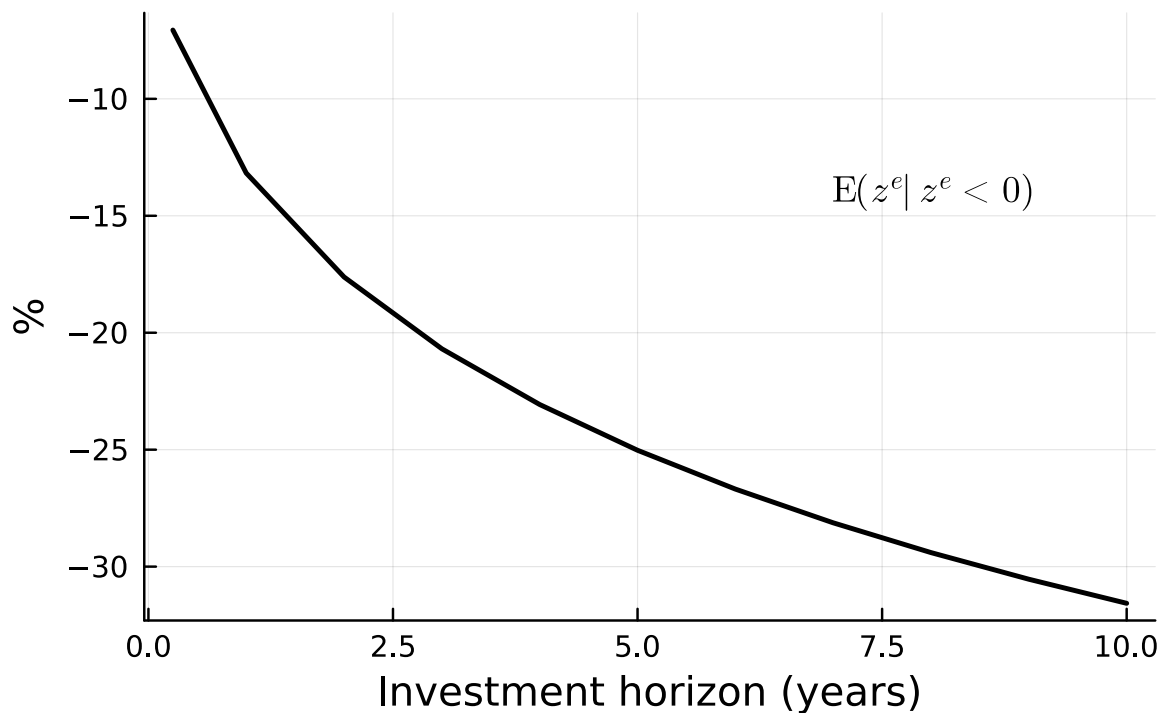
The Distributions.jl package has methods for truncated distributions. We calculate $E(z^e | z^e < 0)$ for horizon q by using

```
mean(truncated(Normal(q*μe, sqrt(q)*σ); upper=0)
```

```
CondER = [mean(truncated(Normal(q*μe, sqrt(q)*σ); upper=0)) for q in qM]

txt = L"\mathrm{E}\{z^e | z^e < 0\}"
p1 = plot( qM, CondER,
           linecolor = :black,
           linewidth = 2,
           legend = false,
           title = "Conditional expectation of excess return",
           xlabel = "Investment horizon (years)",
           ylabel = "%",
           annotation = (7, -14, text(txt, 10, :left)) )
display(p1)
```

Conditional expectation of excess return



Mean-Variance Portfolio Choice

This section runs the time series model presented in the lecture notes. It therefore needs more code than usual for the notebooks.

```
include("src/Var1IR.jl"); #impulse response of VAR(1), Cov(x) from MA(q)
```

```
"""
    xzModel3(φ,θ,T)
"""
function xzModel3(φ,θ,T)
    A = [0 1;
         0 φ]
    B = [1 0;
         0 1]
    C = VAR1IR(A,T,B)
```

```

    r = C[1, :, :]'
    z = C[2, :, :]'
    return r, z
end

"""
    xzModel3Cov( $\phi, \theta, \sigma_u, \sigma_{\eta}, q$ )

Calculate conditional variance-covariance matrix of q future
returns (r_1, r_2, ..., r_q) by first finding the MA representation
and then apply MACov.
"""
function xzModel3Cov( $\phi, \theta, \sigma_u, \sigma_{\eta}, q$ )

    (r_IR, z_IR) = xzModel3( $\phi, \theta, q+10$ )
    r_IR .*= [ $\sigma_u \ \sigma_{\eta}$ ]
    z_IR .*= [ $\sigma_u \ \sigma_{\eta}$ ]

    k = 2                                #no. variables
    C = fill(NaN, q, k, q)
    for t in 1:q
        r_t = vcat(zeros(q-t, k), r_IR[1:t, :])    #t=1: [zeros(9,2); [1 0]]
        C[t, :, :] = r_t'                          #t=2: [zeros(8,2); [-0.5 1]; [1 0]]
    end

    CovM = MACov(C, 1.0I(k))

    return CovM, r_IR, C
end

```

xzModel3Cov

```

( $\phi, \sigma_u, \sigma_{\eta}, T$ ) = (0.25, 0.05, 0.02, 12)

(r_0, z_0) = xzModel3(0, 0, T)
(r_m, z_m) = xzModel3( $\phi, -0.5, T$ )
(r_p, z_p) = xzModel3( $\phi, 0.5, T$ )

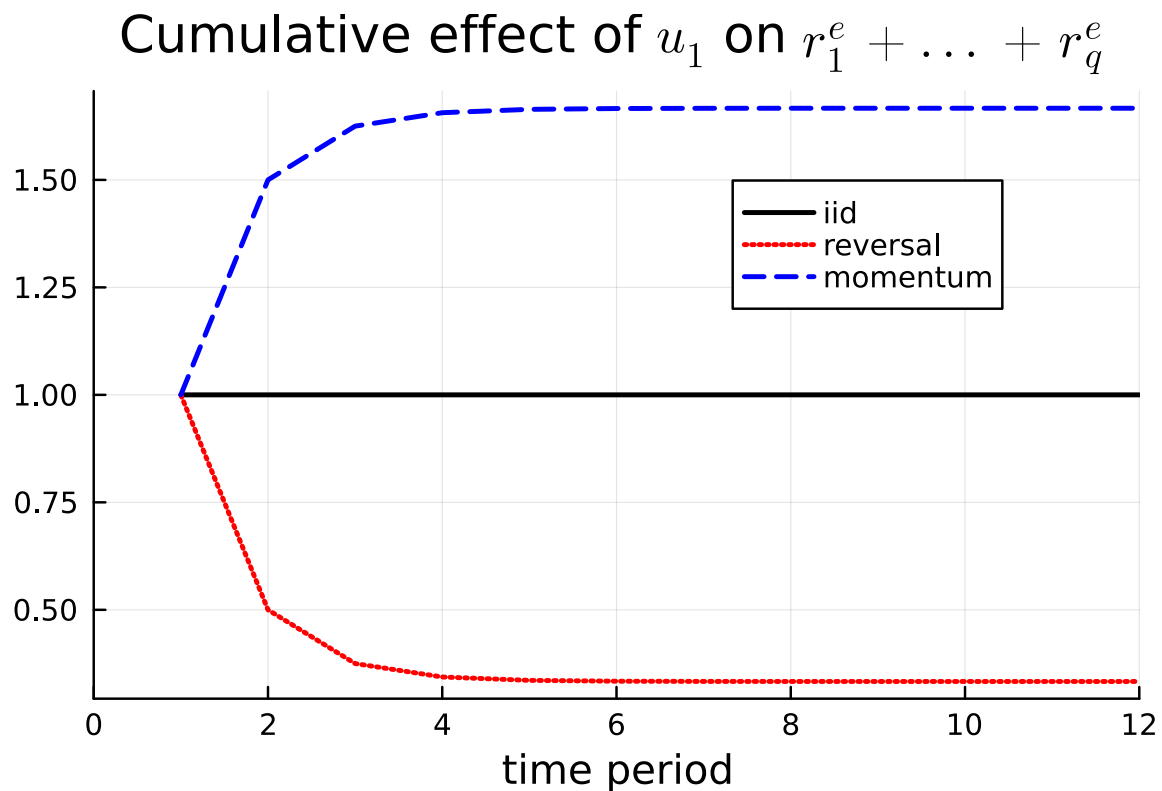
p1 = plot( 1:T, cumsum([r_0[:, 1] r_m[:, 1] r_p[:, 1]]), dims=1),
          xlims = (0, T),
          linestyle = [:solid :dot :dash],

```

```

linecolor = [:black :red :blue],
linewidth = 2,
label = ["iid" "reversal" "momentum"],
legend = (0.7,0.8),
title = L"Cumulative effect of $u_1$ on $r^e_1+\dots+r^e_q$",
xlabel = "time period" )
display(p1)

```



```

qM = 1:T

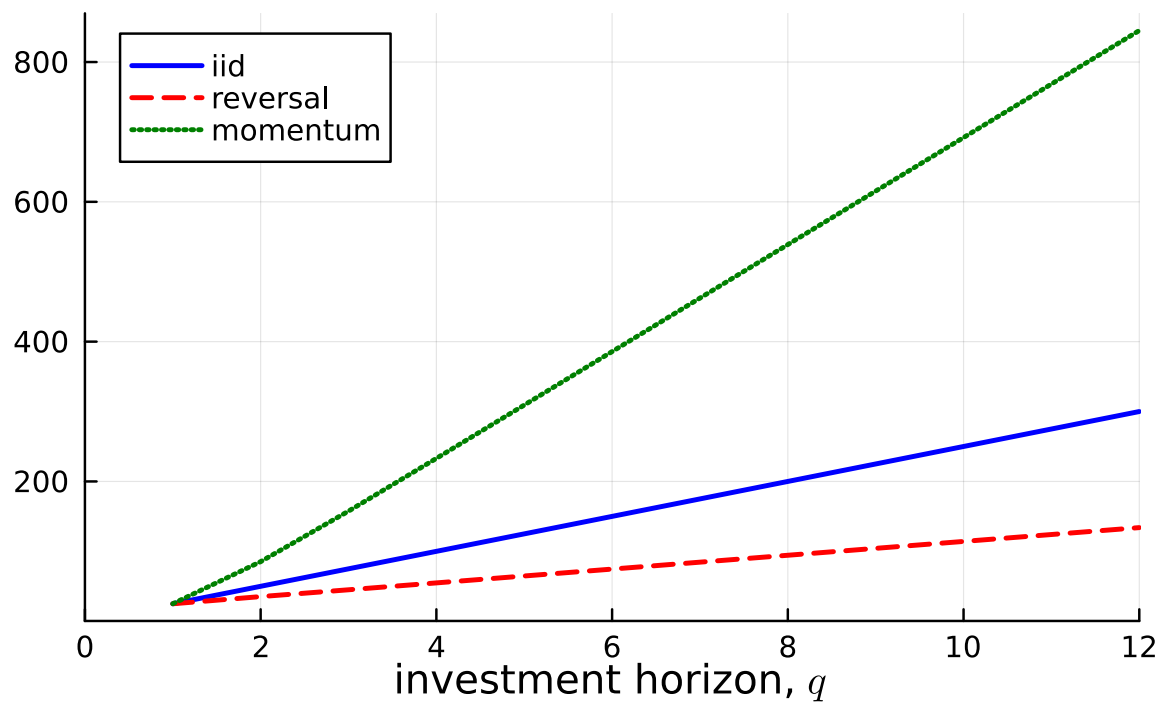
Var_q = fill(NaN,T,3)
for q in qM
    local CovM_m, CovM_0, CovM_p
    CovM_0, = xzModel3Cov(phi,0,sigma_u,0,q)          #iid
    CovM_m, = xzModel3Cov(phi,-0.5,sigma_u,sigma_eta,q) #reversal
    CovM_p, = xzModel3Cov(phi,0.5,sigma_u,sigma_eta,q)  #momentum
    Var_q[q,:] = [sum(CovM_0) sum(CovM_m) sum(CovM_p)] #var(r_1+r_2+...+r_q)
end
printmat(Var_q*10_000);

```

25.000	25.000	25.000
50.000	35.250	85.250
75.000	45.016	157.516
100.000	54.860	232.985
125.000	64.737	309.269
150.000	74.623	385.756
175.000	84.511	462.295
200.000	94.400	538.846
225.000	104.289	615.400
250.000	114.178	691.956
275.000	124.067	768.511
300.000	133.956	845.067

```
p1 = plot( qM, Var_q*10_000,
           xlims = (0,T),
           linestyle = [:solid :dash :dot],
           linecolor = [:blue :red :green],
           linewidth = 2,
           title = L"Variance of  $r^e_1 + \dots + r^e_q$ ",
           label = ["iid" "reversal" "momentum"],
           xlabel = L"investment horizon,  $q$ " )
```


Variance of $r_1^e + \dots + r_q^e$



```
"""
    vWeight( $\mu^e, \sigma^2, k$ )

Optimal portfolio weight on the risky asset, so (1-v) is in the risk-free asset
"""
```

```
vWeight( $\mu^e, \sigma^2, k$ ) =  $\mu^e / ((1+k) * \sigma^2) + 1 / (2 * (1+k))$ 
```

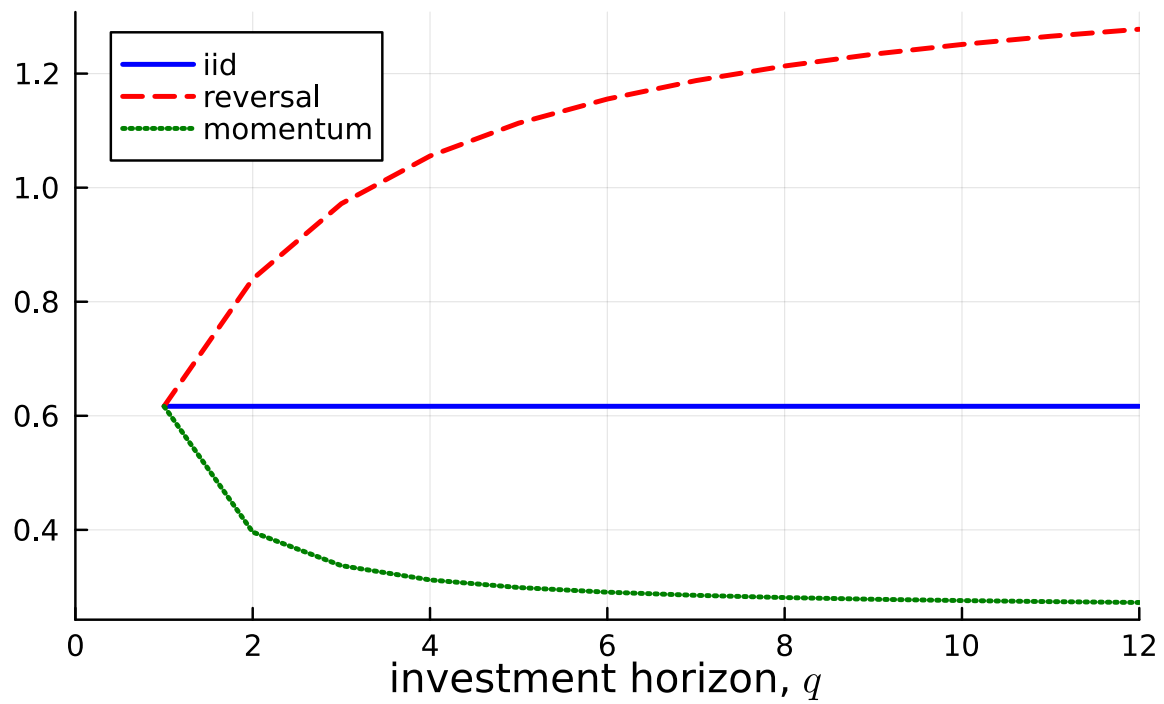
vWeight

```
( $\mu^e, k$ ) = (0.8/100, 5)
v_q = vWeight.( $\mu^e$ . * qM, Var_q, k)

p1 = plot( qM, v_q,
           xlims = (0, T),
           linestyle = [:solid :dash :dot],
           linecolor = [:blue :red :green],
           linewidth = 2,
           title = "portfolio weight on risky asset",
```

```
label = ["iid" "reversal" "momentum"],  
xlabel = L"investment horizon,  $q$ " )
```

portfolio weight on risky asset



Dynamic Portfolios

and intertemporal hedging in a (relatively) simple discrete time model.

Load Packages and Extra Functions

```
using Printf, LinearAlgebra
include("src/printmat.jl");

using Plots, LaTeXStrings
default(size = (480,320),fmt = :png)
```

Log Utility

We first find the *myopic* (period by period) optimal portfolio when the investor has log utility,

$$U(R_p) = \ln(1 + R_p),$$

where R_p is the portfolio return.

In all examples discussed below, the log returns are normally distributed, but the vector of *expected* returns changes across time. In particular, the vector of expected returns take on two different values (state A and state B).

```
"""
Calculate optimal portfolio for log utility case, when the log returns are N(μe+rf,Σ),
Campbell&Viceira.
"""
function OptPortLogUtil(Σ,μe,rf)
    v      = inv(Σ)*(μe+diag(Σ)/2)
    Erp    = rf + v'μe + v'diag(Σ)/2 - v'Σ*v/2
    Varrp  = v'Σ*v
```

```

return v, Erp, Varrp
end

```

OptPortLogUtil

```

Σ = [83 17 29;           #3 risky assets
      17 32 2;
      29 2 50]/10000
μeA = [0.8, 0.9, 0.3]/100    #expected excess returns in state A
μeB = [0.4, 0.45, 0.15]/100  #expected excess returns in state B

vA, = OptPortLogUtil(Σ,μeA,0) #myopic portfolio choice in each state
vB, = OptPortLogUtil(Σ,μeB,0)

printblue("Portfolio weights in the two states (several risky assets):")
rowNames3 = ["asset 1","asset 2","asset 3","riskfree"]
printmat([vA vB;1-sum(vA) 1-sum(vB)];colNames=["state A","state B"],rowNames=rowNames3,prec=2)

```

Portfolio weights in the two states (several risky assets):

	state A	state B
asset 1	0.65	0.49
asset 2	2.93	1.62
asset 3	0.60	0.45
riskfree	-3.18	-1.56

Myopic Portfolio Choice, CRRA

We now turn to the *myopic* (period by period) optimal portfolio when the investor has CRRA utility

$$U(R_p) = (1 + R_p)^{1-\gamma} / (1 - \gamma),$$

where R_p is the portfolio return. As $\gamma \rightarrow 1$, this becomes the same as log utility (need to take the limit to prove that).

```

"""
Calculate optimal portfolio for CRRA utility case, when the log returns are N(μe+rf,Σ),
Campbell&Viceira.
"""
function OptPortCRRA(Σ,μe,rf,γ)
    v      = inv(Σ)*(μe+diag(Σ)/2)/γ
    Erp    = rf + v'μe + v'diag(Σ)/2 - v'Σ*v/2
    Varrp  = v'Σ*v

```

```

return v, Erp, Varrp
end

```

OptPortCRRRA

```

γ = 6

vA, = OptPortCRRRA(Σ,μeA,0,γ)           #with several risky assets
vB, = OptPortCRRRA(Σ,μeB,0,γ)

printblue("Portfolio weights in the two states (several risky assets):")
printmat([vA vB;1-sum(vA) 1-sum(vB)];colNames=["state A","state B"],rowNames=rowNames3,prec=2)

```

Portfolio weights in the two states (several risky assets):

	state A	state B
asset 1	0.11	0.08
asset 2	0.49	0.27
asset 3	0.10	0.07
riskfree	0.30	0.57

Intertemporal Hedging, CRRA

We now consider the more difficult case when the CRRA investor considers several periods.

In the case of log utility ($\gamma = 1$), this actually gives a myopic solution: in each period, $\ln(1 + R_p)$ is maximized, where R_p is the one period portfolio return.

With CRRA this may no longer hold. In particular, if there are some predictable (non-iid) features of the asset returns, then today's investment may be influenced by how the return over the next period is correlated with the investment opportunities in the subsequent periods.

The optimal solution used in the next few cells (see lecture notes for details) is for the case when the vector (n assets) of excess returns follow

$$r_{t+1}^e = a + z_t + u_{t+1},$$

where the vector z_t follows the VAR(1)

$$z_{t+1} = \phi z_t + \eta_{t+1}, \text{ with } \eta_{t+1} \text{ being } N(\mathbf{0}, \Sigma_\eta).$$

The covariance matrix of u_{t+1} and η_{t+1} , which plays a key role of the analysis, is denoted by $\Sigma_{u\eta}$.

Notice that the first equation implies that (*a*) returns are predictable, that is, the expected returns change over time; (*b*) and that those expectations potentially correlates with today's return. This may lead to

intertemporal hedging, where the portfolio weights for an investment between t and $t + 1$ is affected by how the return in $t + 1$ correlates with the investment opportunity set in $t + 1$, that is, with the return distribution in $t + 2$.

The next cell solves the three optimization problems (myopic, no rebalancing, rebalancing) as in the lecture notes.

```
function CRRAPortOpt(Σ_uu::Number,Σ_ηη::Number,z::Number,a::Number,φ::Number,θ::Number,γ)
    v_myop = inv(Σ_uu)*(a + z + Σ_uu/2)/γ
    Σ_2    = 2*Σ_uu*(1+θ) + θ*Σ_uu*θ + Σ_ηη      #for (r1+r2)
    v_noreb = inv(Σ_2)*(2*a + (1+φ)*z + Σ_2/2)/γ
    Ev1     = inv(Σ_uu)*(a + φ*z + Σ_uu/2)/γ
    v_rebal = inv(Σ_uu)*(a + z + Σ_uu/2 + (1-γ)*Σ_uu*θ*Ev1)/γ
    return v_myop, v_noreb, v_rebal, Σ_2
end

function CRRAPortOpt(Σ_uu::Matrix,Σ_ηη::Matrix,z::Vector,a::Vector,φ::Matrix,θ::Matrix,γ)
    v_myop = inv(Σ_uu)*(a + z + diag(Σ_uu)/2)/γ
    Σ_2    = 2*Σ_uu*(I+θ') + θ*Σ_uu*θ' + Σ_ηη      #for (r1+r2)
    v_noreb = inv(Σ_2)*(2*a + (I+φ)*z + diag(Σ_2)/2)/γ
    Ev1     = inv(Σ_uu)*(a + φ*z + diag(Σ_uu)/2)/γ
    v_rebal = inv(Σ_uu)*(a + z + diag(Σ_uu)/2 + (1-γ)*Σ_uu*θ'*Ev1)/γ
    return v_myop, v_noreb, v_rebal, Σ_2
end
```

CRRAPortOpt (generic function with 2 methods)

A Single Risky Asset

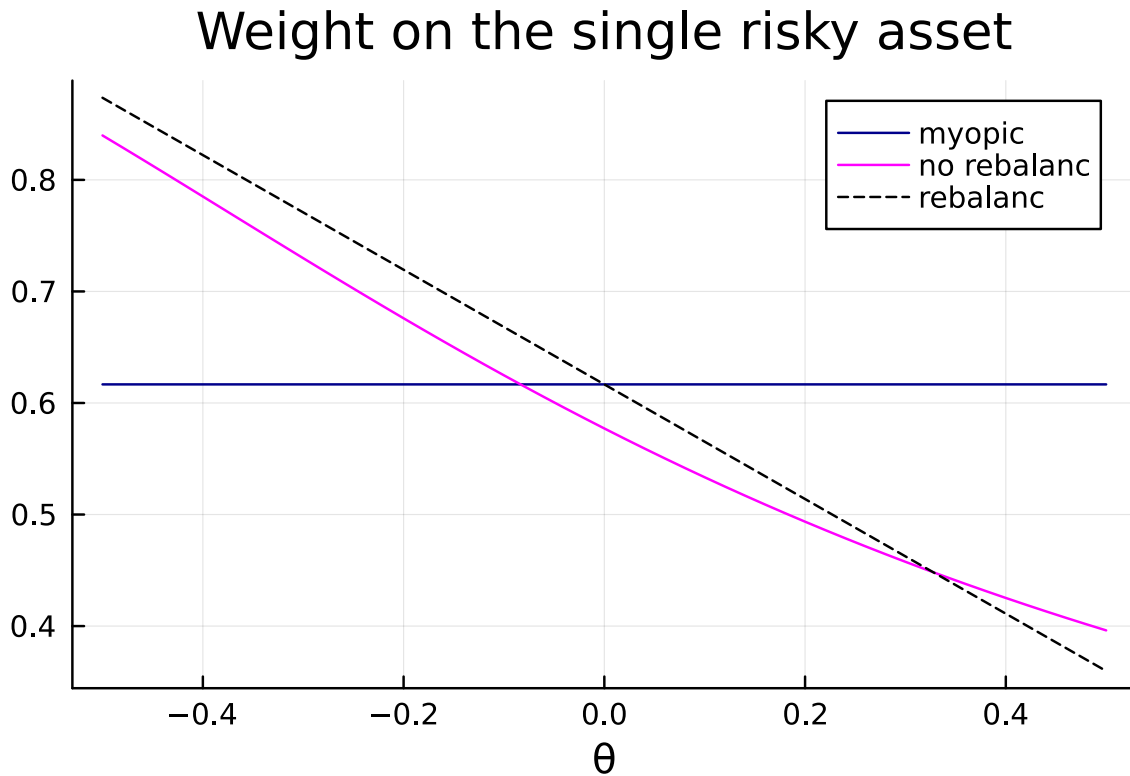
and a riskfree.

The next cell calculates the portfolio weight on the risky asset in (a) the myopic case; (b) the 2-period investor with no rebalancing; and (c) the 2-period investor with rebalancing, for different values of the mean reversion coefficient θ .

```
(φ,σ_u,σ_η,a,γ) = (0.25,0.05,0.02,0.8/100,6)      #parameters in example with one risky asset

L = 51
θM = range(-0.5,0.5,length=L)                      #different mean reversion coeff
(v_myopH,v_norebH,v_rebH) = [fill(NaN,L) for i in 1:3]
for (i,θ) in enumerate(θM)
    (v_myopH[i],v_norebH[i],v_rebH[i],_) = CRRAPortOpt(σ_u^2,σ_η^2,θ,a,φ,θ,γ)
end
```

```
plot(θM, hcat(v_myopH, v_norebH, v_rebH),
     linestyle = [:solid :solid :dash],
     linecolor = [:darkblue :magenta :black],
     title = "Weight on the single risky asset",
     xlabel = "θ",
     label = ["myopic" "no rebalanc" "rebalanc"] )
```



Several Risky Assets

and a riskfree.

The next cells specify the parameter values, solves for optimal portfolio weights (in the myopic case, the 2-period investor with no rebalancing and the 2-period investor with rebalancing) for different values of θ_2 .

```
Σ_uu = [ 83  17  29;
         17  32   2;
         29   2  50]/10_000
```

#parameter values

```
 $\Sigma_{\eta\eta} = \text{diagm}([0, 0.02^2, 0])$ 
```

```
a = [0.8, 0.9, 0.3]/100
```

```
 $\phi = \text{diagm}([0, 0.25, 0]);$ 
```

```
(v_myop, v_noreb, v_reb) = [fill(NaN, L, 3) for i in 1:3]
for (i,  $\theta_i$ ) in enumerate( $\Theta M$ )
    local  $\Theta M_i$ 
     $\Theta M_i = \text{diagm}([0, \theta_i, 0])$ 
    (v_myop[i, :], v_noreb[i, :], v_reb[i, :],) = CRRAPortOpt( $\Sigma_{uu}, \Sigma_{\eta\eta}, \text{zeros}(3), a, \phi, \Theta M_i, \gamma$ )
end
```

```
labels = string("asset ", (1:3)')
```

```
p1 = plot(  $\Theta M$ , v_myop,
           linestyle = [:solid :dot :dash],
           linecolor = [:red :blue :green],
           title = "Weight, myopic",
           xlabel = " $\theta_2$ ",
           label = labels )
```

```
p2 = plot(  $\Theta M$ , v_noreb,
           linestyle = [:solid :dot :dash],
           linecolor = [:red :blue :green],
           title = "Weight, 2-period inv., no rebalancing",
           xlabel = " $\theta_2$ ",
           legend = false)
```

```
p3 = plot(  $\Theta M$ , v_reb,
           linestyle = [:solid :dot :dash],
           linecolor = [:red :blue :green],
           title = "Weight, 2-period inv., rebalancing",
           xlabel = " $\theta_2$ ",
           legend = false)
```

```
pAll = plot( p1, p2, p3,                                     #combine the subplots
             layout = @layout[a a; a _],
             size = (600*1.5, 400*1.5) )
display(pAll)
```